

Banco de Dados

Introdução aos bancos de dados relacionais

Professor

Prof. Dr. Rafael Queiroz Gonçalves, PMP



Formação acadêmica – Ciência da Computação

- Doutorado (2013-2017 - UFSC)
- Mestrado (2011-2012 - UFSC)
- Bacharelado (2006-2010 - UNIVALI)

Formação profissional (certificações)

- Gerente de projetos profissional (PMP)
- Professional Scrum Master (PSM)
- ITIL (AXELOS)
- COBIT (ISACA)
- Arquiteto de sistemas JEE (SUN)
- Profissional UML (OMG)
- Programador JAVA (SUN/ORACLE)
- Programador PHP (Zend technologies)
- Programador Python (Python Institute)
- Administrador de sistemas LINUX (LPI)
- Administrador de sistemas Windows Server (MTA)
- DBA MySQL (ORACLE)
- DBA MONGODB (MongoDB Inc)
- Microsoft PowerBI Data Science
- IBM Lotus Notes Application Developer
- Docker Certified Professional

Atuação docente

- **2018:** Graduação e pós graduação
- **2010:** Tecnólogos e Técnicos

Atuação profissional

- **2013 – atual**: Auditor de Controle Externo e Gestor de TI - Tribunal de Contas do Estado
  - Gerente de projetos
  - Assessor de Governança de TI
  - CTO - Diretor de TI
- **2008-2012**: Desenvolvedor de Software – Beck et al. Services GmbH
- **2006-2008:** Desenvolvedor de Software - Dualline

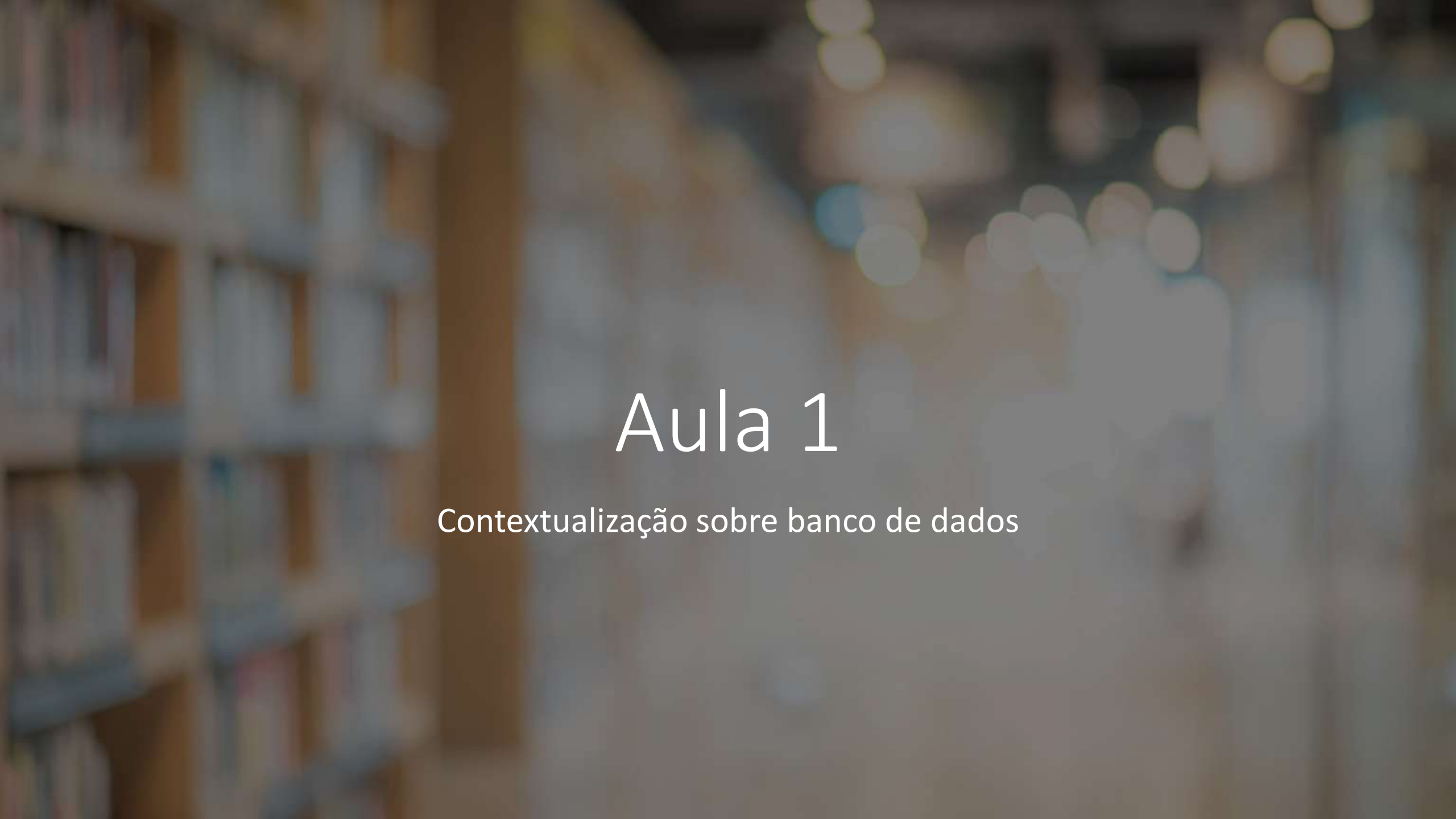
Lattes: http://lattes.cnpq.br/212895803559843
LinkedIn: https://www.linkedin.com/in/rafaelqg

Plano de aula

- 1: Contextualização
- 2: Modelo ER
- 3: Álgebra relacional e SQL
- 4: Normalização: Formas normais
- 5: Trabalho M1
- 6: Avaliação M1

- 7: Índices e DTL
- 8: Check e subqueries
- 9: Agregações e Junções
- 10: DCL (GRANT/REVOKE)
- 11: Trabalho M2
- 12: Avaliação M2

- 13: Functions/Procedures
- 14. Views e Triggers
- 15. Conectando em SGBD por aplicação – JAVA
- 16. Conectando em SGBD por aplicação – Node.JS
- 17: Trabalho M3
- 18: Avaliação M3



Aula 1

Contextualização sobre banco de dados

Persistência de dados

- Bancos de dados são repositórios onde dados são armazenados, ou seja, “persistidos”.
- Dados armazenados de modo que possam ser posteriormente recuperados.
- Para prover a durabilidade tipicamente utiliza-se um meio de memória secundária como HDs, SSDs, memória flash, etc.
- Sistemas operacionais de uso geral costumam prover serviços (APIs) de persistência por meio do seu sistema de arquivos.

Persistência de dados

- O acesso direto à arquivos pelo sistema operacional para persistência de dados é uma maneira possível, porém de muito baixo nível.

- Esta abordagem delega para própria aplicação que está acessando os dados gerenciar a busca, a escrita, o acesso concorrente, etc.

- Seguindo esta abordagem diferentes aplicações podem ter dificuldade de acessar o mesmo “banco de dados”, seja devido ao acesso concorrente ou ao formato de acesso e manipulação do dado.

Persistência de dados

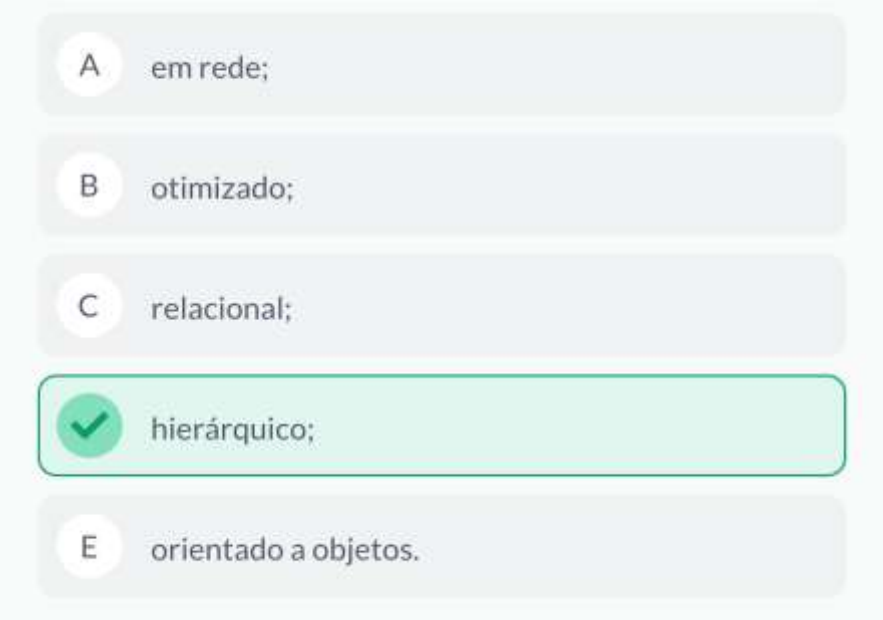
- Exemplos de desafios ao implementar um banco de dados direto com sistema de arquivos.
  - Quando um processo do sistema operacional abre um arquivo para escrita, outros processos não conseguem ler e nem escrever deste arquivo.
  - Algum formato precisará ser definido (CSV, XML, JSON, formato próprio) e cada aplicação que necessitar acessar o dado necessitará estudar a estrutura antes de poder operar a mesma.
  - O controle de acesso é dificultado, normalmente o acesso é em nível de arquivo, tem havendo mecanismos simples de configurar os diretos de acesso de forma mais granular (ex. pode incluir novos itens mas não pode editar existentes).

Sistemas gerenciadores de banco de dados

- Os Sistemas Gerenciadores de Banco de Dados (SGBD) ou Database Management System (DBMS) buscaram facilitar e padronizar a definição, a manipulação e o controle de acesso aos dados.
- Um SGBD é uma aplicação que é executada como um serviço do sistema operacional, normalmente ficando disponível para receber requisições por meio de uma porta lógica.
- O SGBD centraliza todo o acesso a informação, este software que faz acesso ao sistema de arquivos do sistema operacional, ficando a aplicação cliente dependente exclusivamente do SGBD.

Tipos de bancos de dados

O modelo de banco de dados em que a associação entre dois registros é feita por meio do relacionamento entre um registro pai e vários registros-filhos com cardinalidade 1:N, e a navegação é feita da raiz para as folhas, é o:



Sistemas gerenciadores de banco de dados

- SGBD são conhecidos como bancos de dados **transacionais**.
- Mas o que é uma transação?

Uma transação é uma unidade única de trabalho, às vezes composta de várias operações. Qualquer cálculo lógico feito de um modo consistente em um banco de dados é conhecido como uma transação.

Exemplo: é a transferência de uma conta bancária para outra: a transação completa requer a subtração do valor a ser transferido de uma conta e a adição da mesma quantia à outra.

Sistemas gerenciadores de banco de dados

- O principal o objetivo de um SGBD é garantir o atendimento as seguintes princípios (ACID):
  - **Atomicidade:** todas as transações devem ser “indivisíveis”, ou seja, devem ser executadas em sua totalidade. Se ocorrer algum erro, todas as operações que compõem a transação serão descartadas.
  - **Consistência:** uma transação deve levar o banco de dados de um estado consistente para outro estado de consistência, ou seja, toda transação deve respeitar as regras de integridade dos dados (tipo de dado, chave primária etc).
  - **Isolamento:** Mais de uma operação pode ser realizada concorrentemente e uma não deve ser afetada por outras (multiusuário).
  - **Durabilidade:** os resultados de uma operação devem ser persistidos e não desfeitos. Para “desfazer” é realizada uma nova transação que leve ao estado de anterior.

Banco de dados relacional exemplo

Indique a alternativa que não apresenta um banco de dados classificado como relacional:

A) Banco de dados SQL do Azure.

B) MySQL.

C) MongoDB. (gabarito)

D) MariaDB.

E) PostgreSQL.

Exemplo: SGBD

A respeito de sistemas gerenciadores de banco de dados, julgue o item subsequente.

A arquitetura de bancos de dados cliente-servidor é caracterizada pela divisão do processamento entre sistemas, e sua implementação requer **soluções de software** que possibilitem confirmar **transações**, desfazer transações e disponibilizar **linguagens de consultas**.

- **Certo (gabarito)**
- Errado

ACID

Analise as afirmações abaixo sobre gerenciamento de transações em bancos de dados:

1. O controle de concorrência assegura que múltiplas transações possam ocorrer simultaneamente sem causar inconsistências.

2. A recuperação de dados é utilizada para restaurar o banco de dados a um estado consistente após uma falha.

3. A integridade referencial garante que as relações entre tabelas permanecem válidas.

4. A indexação melhora a velocidade de consulta de dados no banco de dados.

5. O isolamento das transações impede que as operações de uma transação interfiram nas operações de outra transação.

O resultado da somatória dos números correspondentes às afirmações corretas é:

A) 06.

B) 10.

C)11.

D) 13.

D) 15. (gabarito)

Atividade – Entendendo SGBDs

- Dada a entidade serviço (data:datetime, descrição:varchar(255), valor:float), faça um programa que utilizando como meio de persistência arquivos texto, faça:
  - A) Insira registros desta entidade.
  - B) Imprima (apresente em tela) todos os registros desta entidade.

- Experimente executar simultaneamente duas instâncias deste programa, e reproduza um cenário de erro por concorrência de recursos ou bloqueio de escrita pelo segundo programa.

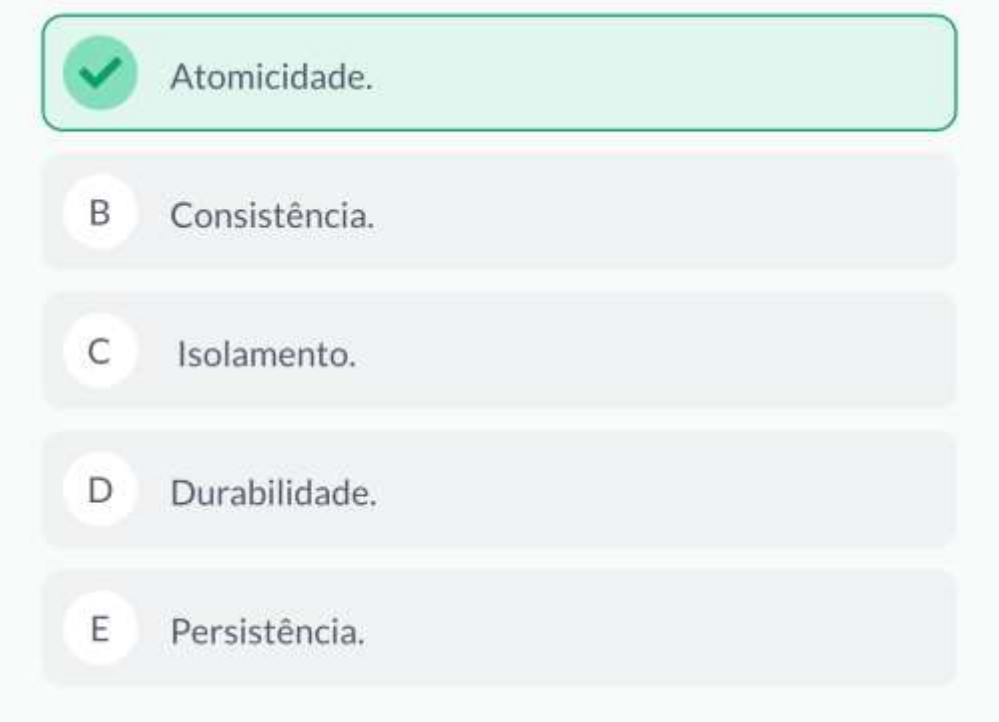
Utilizar threads para emular execuções paralelas e concorrentes.

Atividade – Entendendo SGBDs

- Dando continuidade a atividade anterior, agora cada registro também deve ter uma chave única, auto incrementável.
- Simule situação de erro devido a concorrência.

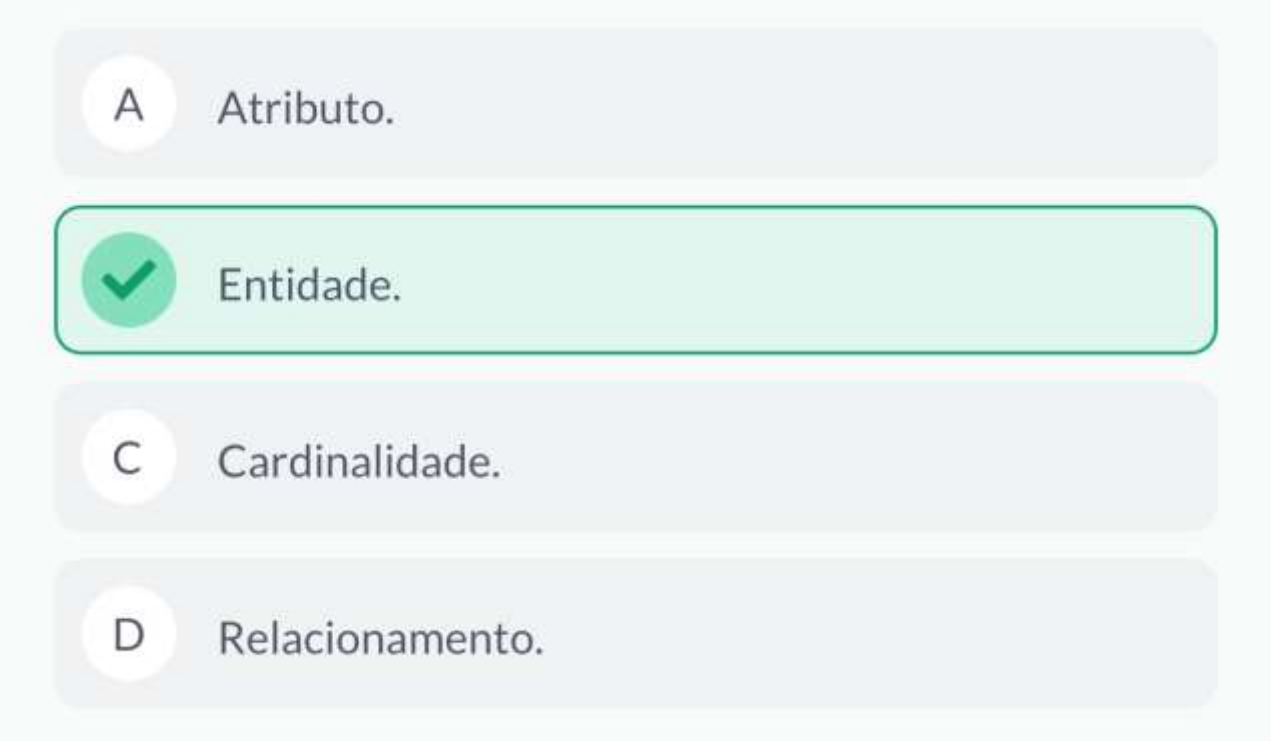
ACID

- Um sistema de gerenciamento de banco de dados (SGBD) deve garantir quatro propriedades durante as transações. Desta forma, marque a propriedade na qual os usuários devem ter a certeza de que todas as ações serão executadas ou nenhuma delas é executada.



Modelo lógico

- Um banco de dados é uma coleção de dados persistentes, usada pelos sistemas de aplicação de uma determinada empresa. No projeto de um banco de dados, o termo geralmente empregado para indicar qualquer objeto distinguível que deva ser representado no banco de dados é conhecido como:



Modelo lógico e físico

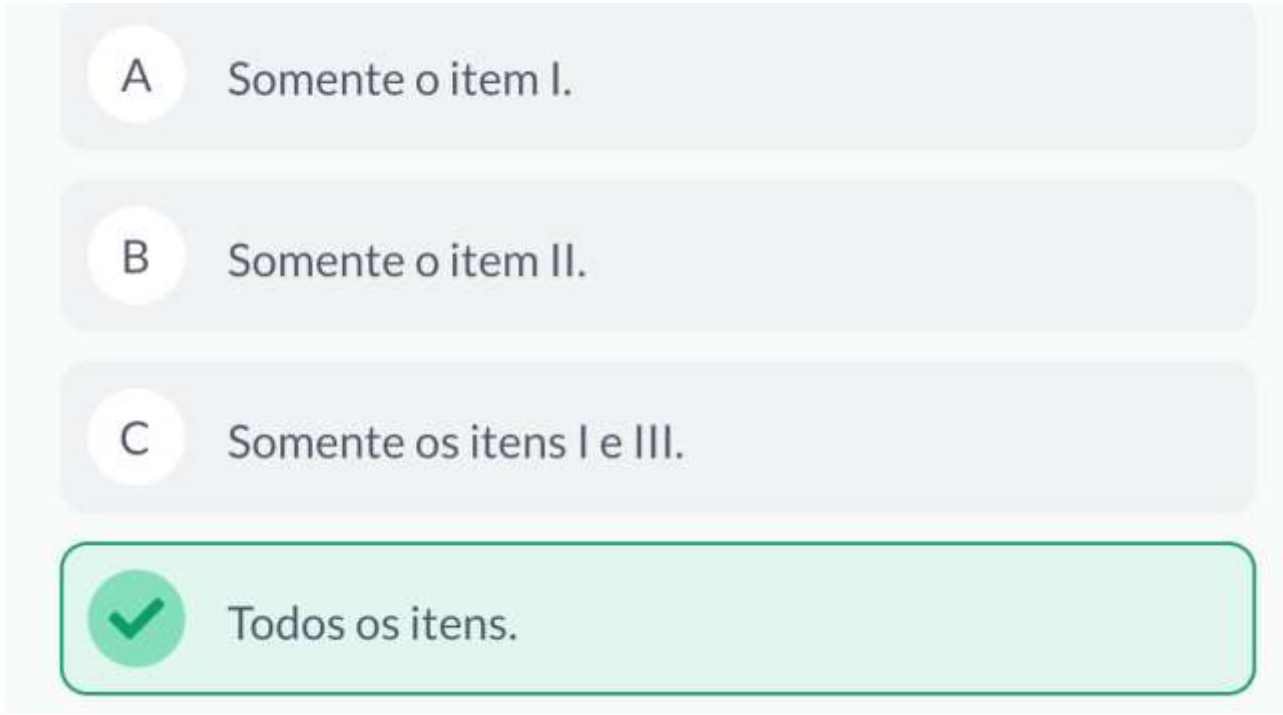
Um técnico de laboratório analisou um modelo relacional de dados de um sistema gerenciador de banco de dados (SGBD) e observou que os dados são apresentados por meio de:



Diferença entre: Dado, informação e conhecimento

Sobre os conceitos de dados na tecnologia da informação, analisar os itens abaixo:

- I. Os sistemas computacionais trabalham basicamente com **dados** e sua função é o processamento desses dados.
- II. Pode-se entender o **dado** como um elemento da informação (um conjunto de letras ou dígitos) que, tomado isoladamente, não transmite nenhum conhecimento e não contém um significado intrínseco.
- III. **Dados** são materiais brutos que precisam ser manipulados e colocados em um contexto compreensivo antes de se tornarem úteis.

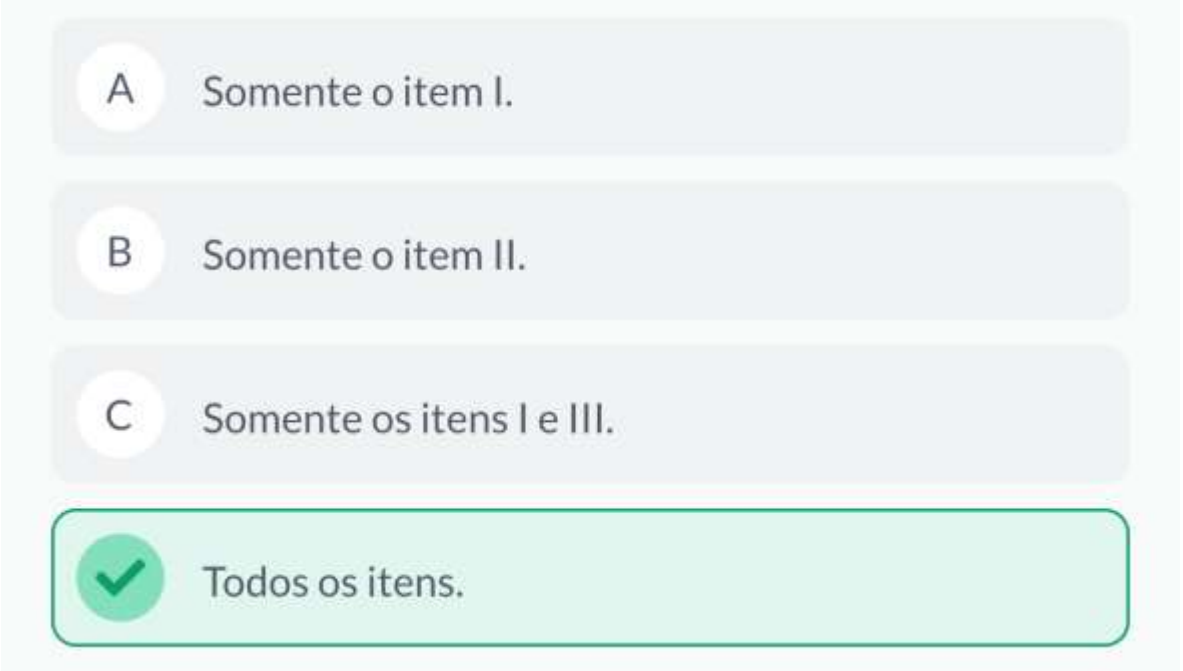


Exemplo

Sobre os conceitos de dados na tecnologia da informação, analisar os itens abaixo:

- I. Os sistemas computacionais trabalham basicamente com dados e sua função é o processamento desses dados.
- II. Pode-se entender o dado como um elemento da informação (um conjunto de letras ou dígitos) que, tomado isoladamente, não transmite nenhum conhecimento e não contém um significado intrínseco.
- III. Dados são materiais brutos que precisam ser manipulados e colocados em um contexto compreensivo antes de se tornarem úteis.

Está(ão) CORRETO(S)



Exemplo - ACID

Um sistema de banco de dados distribuído (BDD) consiste em um relação de nós, cada qual podendo participar na execução de transações que acessam dados em um ou mais nós. O acesso a diversos itens de dados em um sistema distribuído é normalmente acompanhado de transações que têm de preservar a propriedade da atomicidade, que implica na seguinte premissa:

- A) Depois da transação completar-se com sucesso, as mudanças que ela faz no banco de dados persistem.
- B) Cada transação não toma conhecimento de outras transações concorrentes.
- **C) Todas as operações da transação são refletidas corretamente no BD ou nenhuma será (gabarito).**
- D) A transparência para leitura é mais fácil de se conseguir e manter do que a transparência para atualização.
- E) A execução de uma transação isolada preserva a consistência do banco de dados.

Aula 2

SGBDs – Sistemas Gerenciadores de Bancos de Dados e modelo ER

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Exemplos de SGBDs













SGBDs relacionais

- São baseados na álgebra relacional.
- Normalmente estão presentes tabelas e relações entre registros destas tabelas, por meio de chaves primárias e estrangeirais.
- Regras de consistência de dados estão fortemente presentes na definição das estruturas de dados.

Características de um SGBD

As características de um SGBD (Sistema de Gerenciamento de Banco de Dados):

- **Velocidade de acesso:** o sistema deve fornecer as respostas aos pedidos o mais rapidamente possível, o que implica algoritmos de busca ágeis e bem elaborados;
- **Administração centralizada**: o SGBD deve permitir ao administrador poder manipular os dados, inserir elementos e verificar a sua integridade de maneira centralizada;
- **Limitação da redundância**: o SGBD deve poder evitar, na medida do possível, informações redundantes, a fim de evitar desperdício do espaço memória e erros de armazenamento (incosistências);
- **Verificação da integridade**: os dados devem ser coerentes entre eles, ainda mais quando os elementos fazem referência a outros, estes últimos devem estar presentes;
- **Compartilhamento de dados**: o SGBD deve permitir o acesso simultâneo à base de dados por vários usuários;
- **Segurança dos dados:** o SGBD deve apresentar mecanismos que permitam gerenciar as permissões de acesso aos dados de acordo com os usuários

Qual SGBD escolher?

Apesar de existir uma grande variedade de produtos, todos buscam seguir um padrão mínimo de compatibilidade, sendo este conhecido como SQL ANSI. O padrão foi evoluindo com o decorrer do tempo:

- **SQL-86:** Primeira versão da linguagem, lançada em 1986, consiste basicamente na versão inicial da linguagem criada pela IBM.
- **SQL-92:** Lançada em 1992, inclui novos recursos tais como tabelas temporárias, novas funções, expressões nomeadas, valores únicos, instrução CASE etc.
- **SQL-1999 (SQL3)** - Lançada em 1999, foi a versão que teve mais recursos novos significativos, entre eles: a implementação de expressões regulares, recursos de orientação a objetos, queries recursivas, triggers, novos tipos de dados (boolean, LOB, array e outros), novos predicados etc.
- **SQL-2003:** Lançada em 2003, inclui suporte básico ao padrão XML, sequências padronizadas, instrução MERGE, colunas com valores auto-incrementais etc.
- **SQL:2006** - Lançada em 2006, não inclui mudanças significativas para as funções e comandos SQL. Contempla basicamente a interação entre SQL e XML.
- **SQL:2008** – Melhorias em recursos de paginação.
- **SQL:2011** – Suporte à tabelas temporárias.
- **SQL:2016** – Recursos para tratar dados semi-estruturados (ex. JSON).
- **SQL:2023** – melhorias para tratar dados sem-estruturados (ex. JSON, arrays, grafos)

Source: https://exerciciosresolvidos.com.br/o-que-e-SQL-ANSI/

Transação - Concorrência

Em SGBDs relacionais, uma transação pode executar:

A. Somente uma operação de consulta, inserção, atualização ou exclusão.

B. Várias operações de consulta, mas apenas uma operação de inserção, atualização ou exclusão.

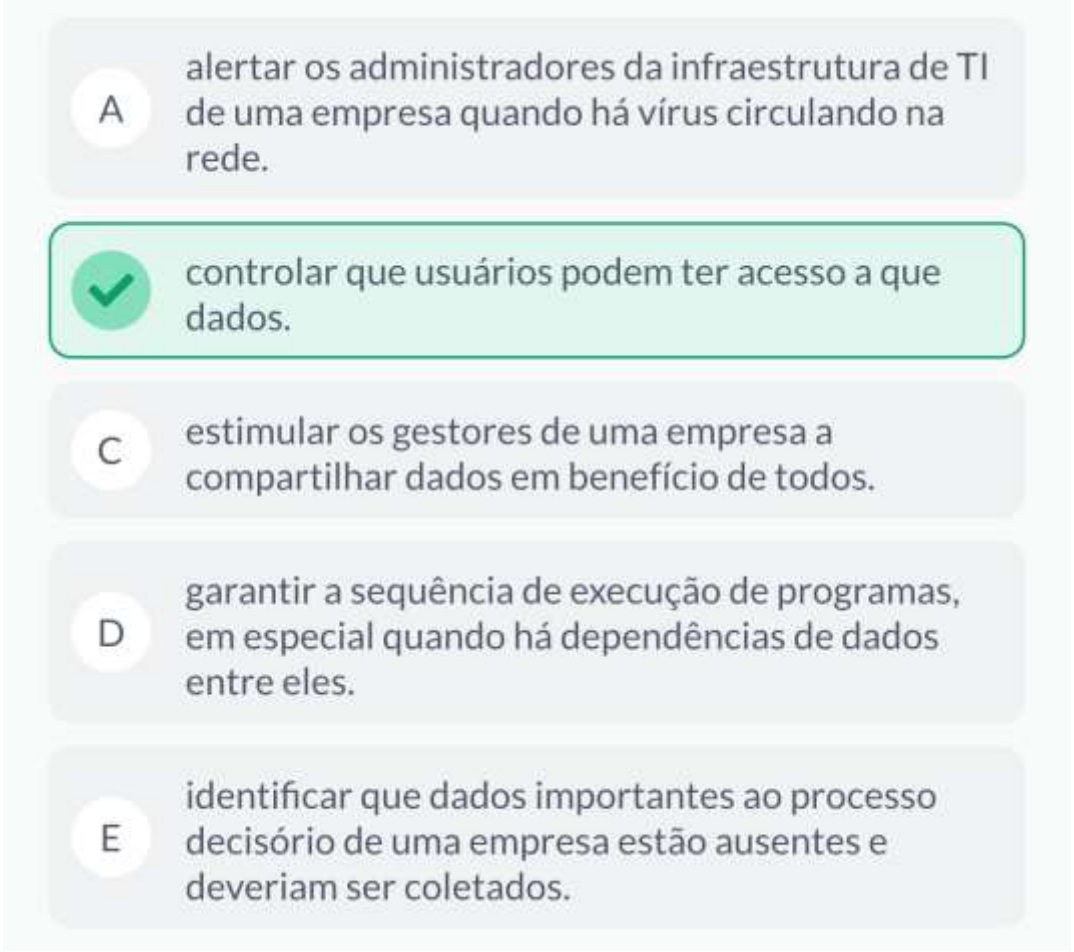
C. No máximo duas operações, sendo que uma delas deve ser de consulta.

D. Somente uma operação de atualização ou exclusão, mas várias operações de consulta e inserção.

✓ Várias operações de consulta, inserção, atualização e exclusão.

SGBD - Finalidade

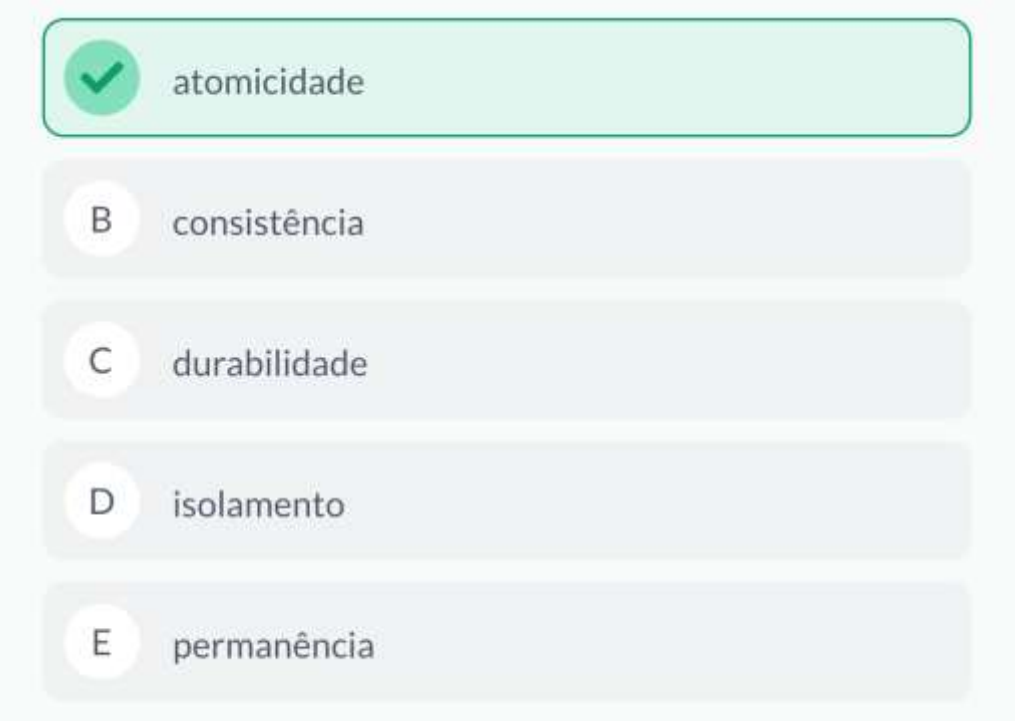
Um Sistema Gerenciador de Banco de Dados (SGBD) é um software bastante utilizado em empresas que precisam armazenar, tratar e utilizar dados em geral.
O SGBD é especializado em realizar atividades relacionadas aos dados.
Uma das várias funcionalidades que um SGBD pode executar é:



Atomicidade

Um programador de Back-End precisava construir uma transação longa, que alterava várias tabelas de um Banco de Dados, de forma que uma das alternativas ocorresse: ou a transação fosse completada, ou todos os dados permanecessem em seu estado original.

Para isso, contou com a propriedade de transações conhecida como



Atomicidade:

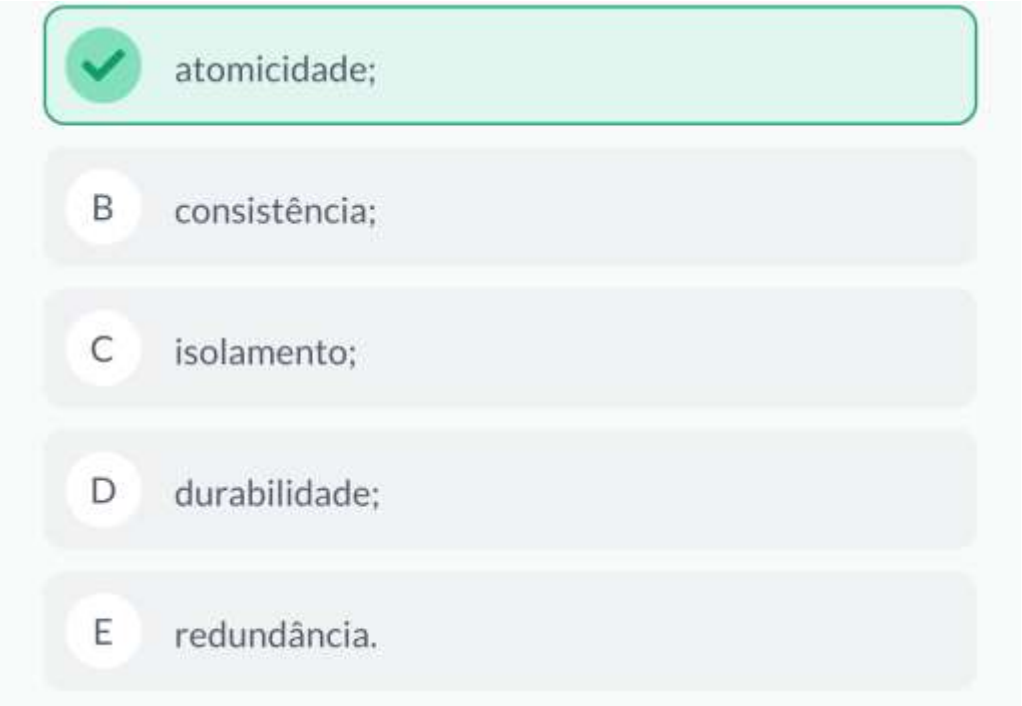
Em banco de dados, uma transação é composta por diversas operações sobre os dados, como leitura, alteração, inserção e remoção. Contudo, a transação é percebida como uma operação lógica única sobre os dados.

Com isso, para garantir a consistência de um banco de dados, as transações devem satisfazer algumas propriedades, como:

(i) todas as operações da transação são executadas em caso de sucesso; ou

(ii) em caso de falhas, todas as operações são desfeitas e o banco de dados deve voltar ao mesmo estado em que estava antes do início da transação.

A propriedade de uma transação de banco de dados descrita é a:



Atomicidade

Para um Sistema Gerenciador de Banco de Dados (SGBD), uma transação é um recurso que possui múltiplas finalidades no momento da execução de um conjunto de comandos (por exemplo, em SQL) pelo SGBD.

Dentre essas finalidades, verifica-se que uma transação:

A) realiza a transferência de recursos de uma conta SQL para outra.

B) permite agrupar vários comandos SQL de modo que ou todos são executados ou nenhum deles é executado.

C) coordena a troca de prioridades entre uma transação principal e outra secundária.

D) combina vários comandos SELECT da SQL em apenas um antes da sua execução.

E) atesta que nenhum dos comandos SQL informados foi executado anteriormente, no mesmo dia.

MySQL

Analise as afirmações a seguir sobre o MySQL.

- I. O MySQL suporta diversos tipos de tipos de dados, tais como dados numéricos e de data e hora. Contudo, não suporta dados do tipo JSON.
- II. O MySQL é um software gerenciador de banco de dados desenvolvido e comercializado pela Oracle, não dispondo de uma versão Open Source.
- III. O MySQL é um software gerenciador de banco de dados desenvolvido pela Oracle, mas que não é comercializado, sendo disponibilizado como software Open Source para ser utilizado sob os termos da GNU General Public License.
- IV. Até o momento, além de plataformas Microsoft Windows e macOS, o MySQL tem compatibilidade com o Linux em distribuições como Ubuntu e Debian.

Quais afirmações são VERDADEIRAS?

A) Apenas I está correta.

B) I, III e IV.

C) I e IV.

D) I, II e IV.

E) Apenas IV está correta. (gabarito)

Exemplo – SGBD on cloud

Durante o processo de desenvolvimento de um sistema WEB no Tribunal de Justiça do Estado de Rondônia (TJRO), o DBA responsável pelo projeto optou pela utilização do serviço Amazon Relational Database Service (Amazon RDS). O motivo dessa decisão se deve ao fato de se tratar de um serviço de banco de dados relacional de fácil gerenciamento e otimizado para o custo total de propriedade, sendo simples de configurar, operar e escalar de acordo com a demanda. O RDS possibilita a automatização de tarefas genéricas de gerenciamento de banco de dados, como provisionamento, configuração, backup e aplicação de patches, sendo ideal para as características do projeto a ser desenvolvido pelo TJRO. Segundo o fabricante do serviço, Amazon, o serviço suporta alguns mecanismos de banco de dados, EXCETO:

A) RDS para Db2.

B) Amazon Aurora.

C) RDS para Oracle.

D) RDS para Firebird. (gabarito)

E) RDS para MariaDB.

O Firebird é um sistema de gerenciamento de banco de dados relacional (SGBD) de código aberto (open source), leve e gratuito, derivado do código-fonte do InterBase da Borland. Ele é muito usado em softwares comerciais e sistemas de gestão (ERP) por sua alta estabilidade e facilidade de manutenção.

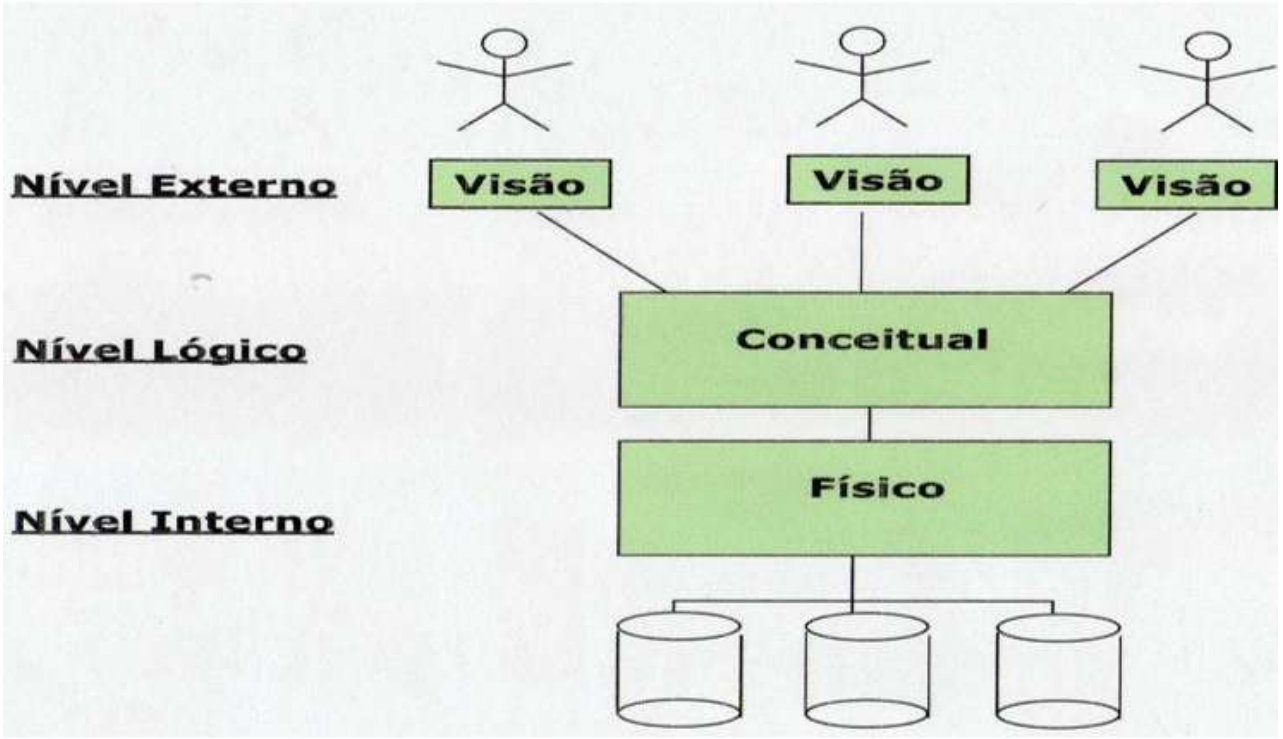
Arquitetura ANSI/SPARC

A arquitetura ANSI/SPARC, criada em meados de 1975, define níveis de abstração para um sistema de gestão de bases de dados:

Nível externo: camada de alto nível em que são feitas interações com o usuário (visão do usuário). Aqui as informações já são consolidadas e agregadas se necessário.

Nível conceptual (ou lógico): chamado também MCD (modelo conceptual dos dados) ou MLD (**modelo lógico dos dados**). Define a disposição das informações na base de dados (os dados são mapeados para tabelas).

Nível interno (ou físico) : define a estrutura de como são armazenados os dados e os métodos para acessa-los.



source: https://social.msdn.microsoft.com/Forums/getfile/756872

Arquitetura ANSI/SPARC

A arquitetura a três níveis definida pelo standard ANSI/SPARC permite ter uma INDEPENDÊNCIA entre os dados e os tratamentos.

- **Independência física** : o nível físico pode ser alterado independentemente do nível conceitual. Isso significa que todos os aspectos físicos da base de dados não são visíveis ao usuário, trata-se simplesmente de uma estrutura transparente de representação das informações.
- **Independência lógica**: o nível conceitual poderá ser alterado sem trazer riscos ao nível físico, ou seja, o administrador da base deve poder aplicar melhorias e manuseá-lo sem impactar os usuários.
- **Maneabilidade** : pessoas que não conhecem a base de dados devem ser capazes de “fazer o seu pedido” sem fazer referência a elementos técnicos da base de dados.

Source: https://social.msdn.microsoft.com/Forums/getfile/756872

Arquitetura ANSI/SPARC

Modelo lógico de dados:

Aquele em que os objetos, suas características e relacionamentos têm a representação de acordo com as regras de implementação e limitantes impostos por algum tipo de tecnologia. Essa representação, por sua vez, é independente dos dispositivos ou meios de armazenamento físico das estruturas de dados por ela definidas."

Source: https://social.msdn.microsoft.com/Forums/getfile/756872

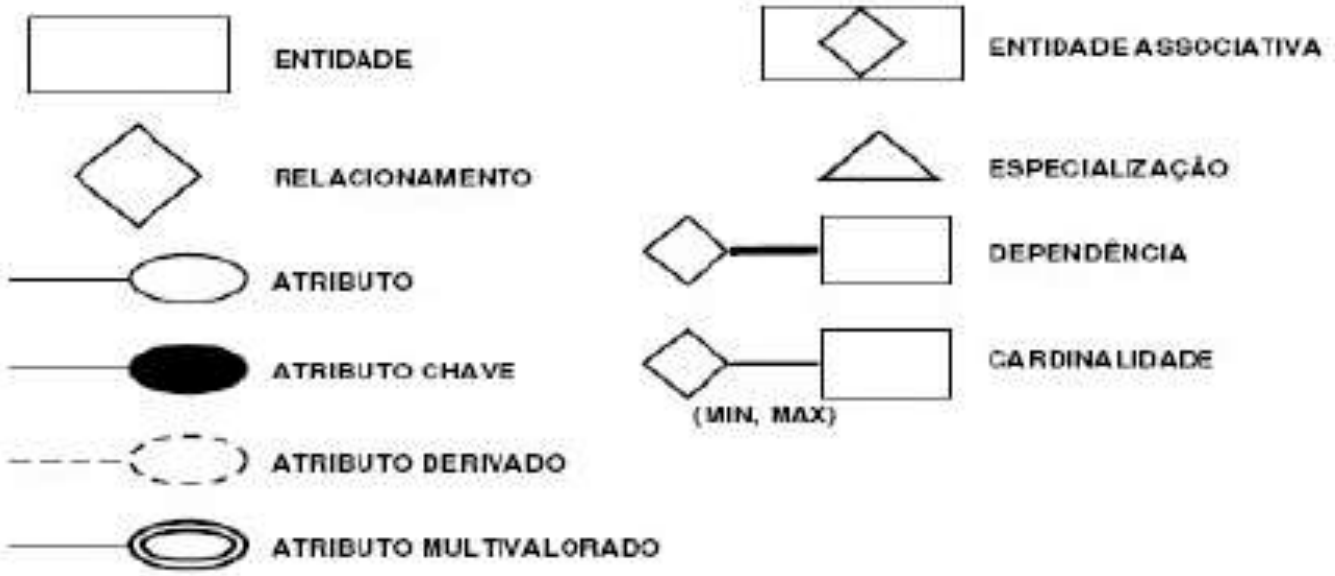
Modelo ER

- O Modelo Entidade Relacionamento (Modelo ER ou MER) é um modelo conceitual para descrever os objetos (**entidades**) envolvidos em um domínio de negócios, com suas características (atributos) e como elas se relacionam entre si (**relacionamentos**).

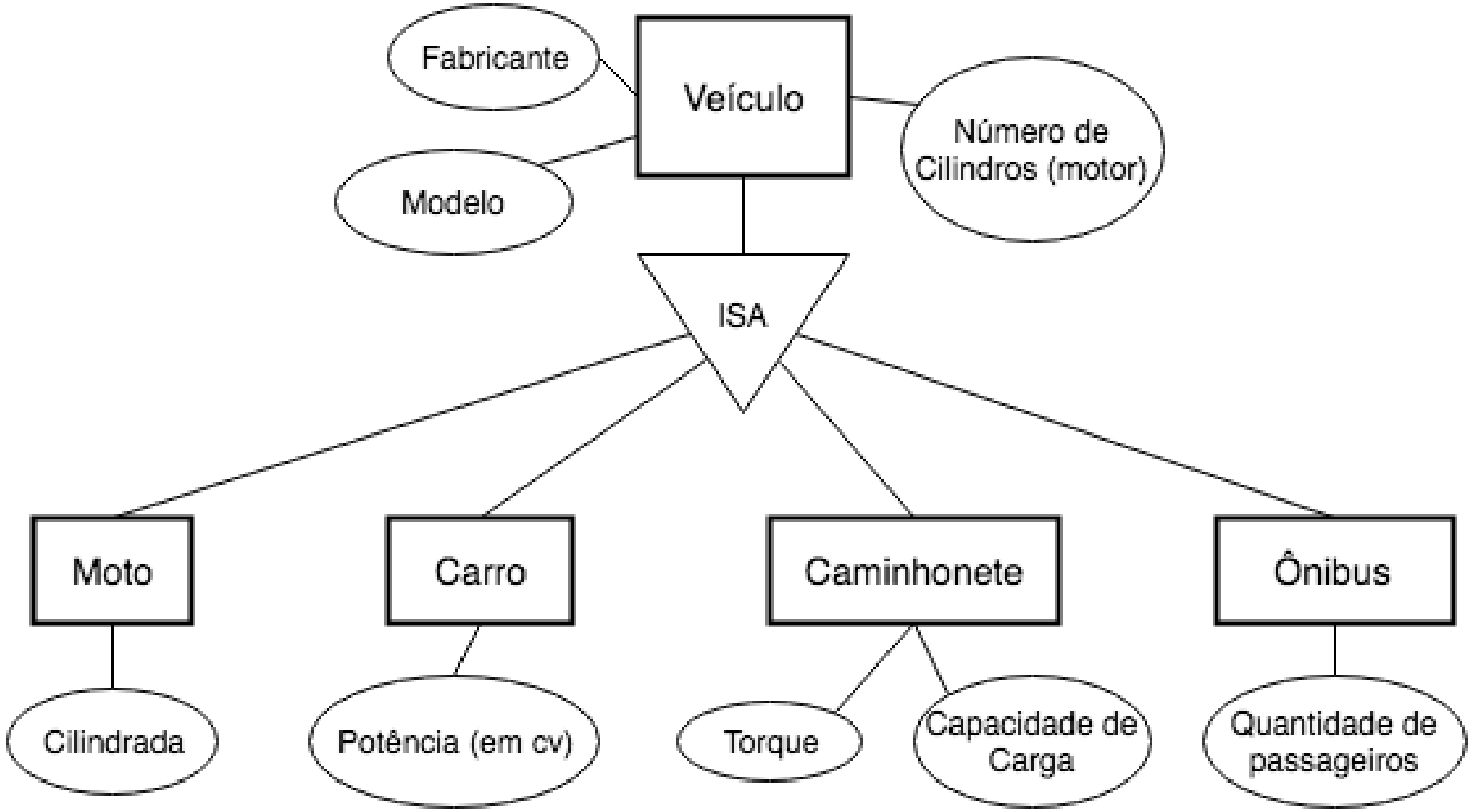
- Este modelo representa de forma abstrata a estrutura que possuirá o banco de dados da aplicação. O **modelo lógico** do banco de dados poderá conter várias outras entidades, como tabelas intermediárias, que podem só fazer sentido no contexto de bases de dados relacionais.

Diagrama ER (DER)

- A notação mais adotada para descrever o modelo ER em formato de diagramas é a diagrama entidade relacionamento (DER) seguindo a notação de Peter Chen.



DER - Exemplo


Fabricante
Veículo
Número de Cilindros (motor)
Modelo
ISA
Moto
Carro
Caminhonete
Ônibus
Cilindrada
Potência (em cv)
Torque
Capacidade de Carga
Quantidade de passageiros

MER: Entidade

Entidades podem ser classificadas como físicas ou lógicas, de acordo sua existência no mundo real.

- **Entidades físicas**: são aquelas realmente tangíveis, existentes e visíveis no mundo real, como um cliente (uma pessoa, uma empresa) ou um produto (um carro, um computador, uma roupa).
- **Entidades lógicas**: são aquelas que existem geralmente em decorrência da interação entre ou com entidades físicas, que fazem sentido dentro de um certo domínio de negócios, mas que não são objetos físicos (que ocupam lugar no espaço). Exemplos: uma venda ou uma classificação de objeto (modelo, espécie, função de um usuário do sistema).

MER: Entidade

Entidades fortes: são aquelas cuja existência independe de outras entidades, ou seja, por si só elas já possuem total sentido de existir.

Exemplo: Em um sistema de vendas, a entidade produto, por exemplo, independe de quaisquer outras para existir.



Entidades fracas: as entidades fracas são aquelas que dependem de outras entidades para existirem, pois individualmente elas não fazem sentido.

Exemplo: a entidade venda depende da entidade produto, pois uma venda sem itens não tem sentido.



Exemplo: entidade fraca

- O Modelo Entidade-Relacionamento (MER) é uma abordagem amplamente utilizada para modelar a estrutura de um banco de dados, representando as entidades do sistema e os relacionamentos entre elas. Considerando o desenvolvimento do Modelo Entidade-Relacionamento, qual das alternativas abaixo está correta?
- A) Atributos multivalorados são aqueles que podem ser derivados a partir de outros atributos e não são armazenados diretamente no banco de dados.
- B) Os atributos compostos em um MER são aqueles que podem ter múltiplos valores, como uma lista de números de telefone de um cliente.
- C) A cardinalidade de uma relação entre duas entidades define o número máximo de vezes que uma entidade pode se relacionar com outra, mas não o número mínimo.
- D) Em um Diagrama Entidade-Relacionamento, as relações entre entidades devem ser sempre do tipo "um para muitos".
- **E) Entidades fracas dependem de entidades fortes para sua existência, e sua chave primária é parcialmente derivada da chave primária da entidade forte. (gabarito)**

MER: relacionamentos

Uma vez que as entidades são identificadas, deve-se então definir como se dá o relacionamento entre elas.

- **Relacionamento 1..1 (um para um):** cada uma das duas entidades envolvidas referenciam obrigatoriamente apenas uma unidade da outra. Por exemplo, em um banco de dados de currículos, cada usuário cadastrado pode possuir apenas um currículo na base, ao mesmo tempo em que cada currículo só pertence a um único usuário cadastrado.
- **Relacionamento 1..n ou 1..* (um para muitos):** uma das entidades envolvidas pode referenciar várias unidades da outra, porém, do outro lado cada uma das várias unidades referenciadas só pode estar ligada uma unidade da outra entidade. Por exemplo, em um sistema de plano de saúde, um usuário pode ter vários dependentes, mas cada dependente só pode estar ligado a um usuário principal. Note que temos apenas duas entidades envolvidas: usuário e dependente. O que muda é a quantidade de unidades/exemplares envolvidas de cada lado.
- **Relacionamento n..m ou *..* (muitos para muitos):** neste tipo de relacionamento cada entidade, de ambos os lados, podem referenciar múltiplas unidades da outra. Por exemplo, em um sistema de biblioteca, um título pode ser escrito por vários autores, ao mesmo tempo em que um autor pode escrever vários títulos. Assim, um objeto do tipo autor pode referenciar múltiplos objetos do tipo título, e vice versa.

Source: https://www.devmedia.com.br/mer-e-der-modelagem-de-bancos-de-dados/14332

Cardinalidade

Em projetos de banco de dados, a cardinalidade define o número de ocorrências de uma entidade que pode estar associada a outra entidade. Com base nesse conceito, assinale a alternativa correta sobre cardinalidade.

A) A cardinalidade não pode ser utilizada para restringir relações entre tabelas.

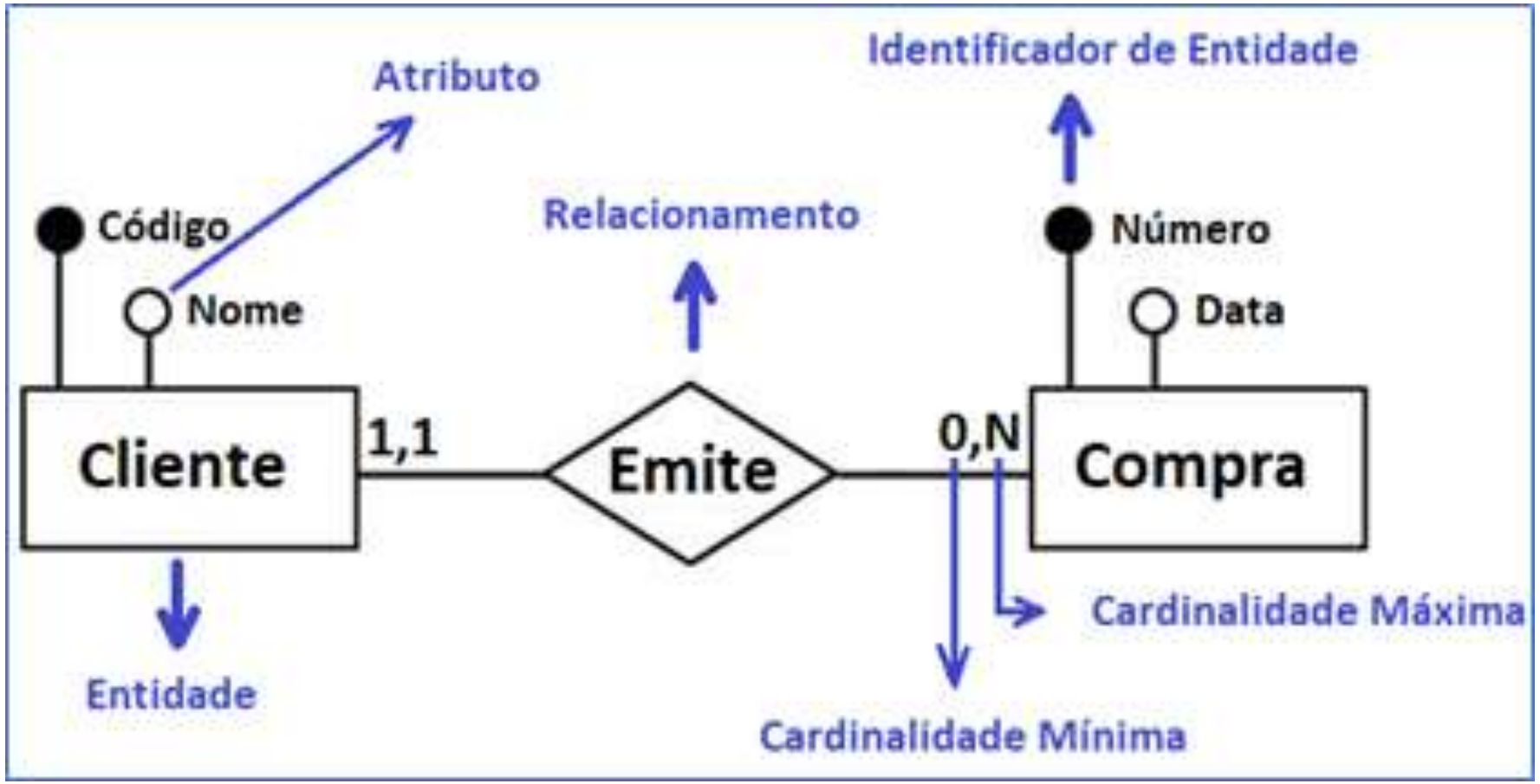
B) A cardinalidade muitos-para-muitos (N:M) é utilizada exclusivamente para normalizar tabelas.

C) A cardinalidade refere-se à quantidade máxima de colunas que uma tabela pode ter em um banco de dados.

D) Uma relação um-para-um (1:1) implica que uma entidade pode ter no máximo um atributo-chave.

E) Uma cardinalidade um-para-muitos (1:N) indica que uma instância de uma entidade pode estar associada a uma ou mais instâncias de outra entidade. (gabarito)

MER: relacionamentos



Source: https://leonardofonseca.com.br/2021/05/05/relacionamento-em-um-modelo-relacional-parte-2/

MER: Entidade associativa

- Uma tabela associativa (ou junção) mapeia duas ou mais tabelas referenciando as chaves primárias de cada tabela de dados. Normalmente necessária quando há necessidade de implementar relacionamentos n..m.



Cardinalidade

Considere as seguintes entidades em um Modelo EntidadeRelacionamento (MER):Para representar esse relacionamento no banco de dados relacional, garantindo a integridade referencial, o usuário deve:

A) adicionar uma chave estrangeira (id_cliente) na tabela Pedido, referenciando a tabela Cliente. (gabarito)

B) criar uma tabela intermediária (Cliente_Pedido) para armazenar as chaves primárias das tabelas Cliente e Pedido.

C) adicionar uma chave estrangeira (id_pedido) na tabela Cliente, referenciando a tabela Pedido.

D) usar uma chave composta (id_cliente, id_pedido) na tabela Pedido.

E) representar o relacionamento com uma view SQL que une as tabelas Cliente e Pedido.

Entidades:

Cliente (id_cliente, nome, email).
Pedido (id_pedido, data_pedido, valor_total).

Relacionamentos:

Um Cliente pode fazer vários Pedidos.
Um Pedido pertence a um único Cliente.

Regras de Negócio:

Cada Pedido deve obrigatoriamente estar associado a um Cliente.
Um Cliente pode existir sem ter feito nenhum Pedido.

MER: Atributos

Atributos são as características que descrevem cada entidade dentro do domínio. Durante a análise de requisitos, são identificados os atributos relevantes de cada entidade naquele contexto, de forma a manter o modelo o mais simples possível e consequentemente armazenar apenas as informações que serão úteis futuramente

- Exemplo: um cliente possui nome, endereço e telefone.

MER: Atributos

Atributos podem ser classificados quanto à sua **função** da seguinte forma:

- **Descritivos:** representam característica intrínsecas de uma entidade. Exemplo: nome e cor.
- **Nominativos**: além de serem também descritivos, estes têm a função de definir e identificar um objeto. Exemplo: Código e número.
- **Referenciais:** representam a ligação de uma entidade com outra em um relacionamento. Exemplo: uma venda possui o CPF do cliente, que a relaciona com a entidade cliente.

Quanto à sua **estrutura** podem ser classificados como**:**

- **Simples**: um único atributo define uma característica da entidade. Exemplos: nome, peso, etc.
- **Compostos**: para definir uma informação da entidade, são usados vários atributos. Por exemplo, o endereço pode ser composto por rua, número, bairro, etc.

Diagrama ER - Atributos

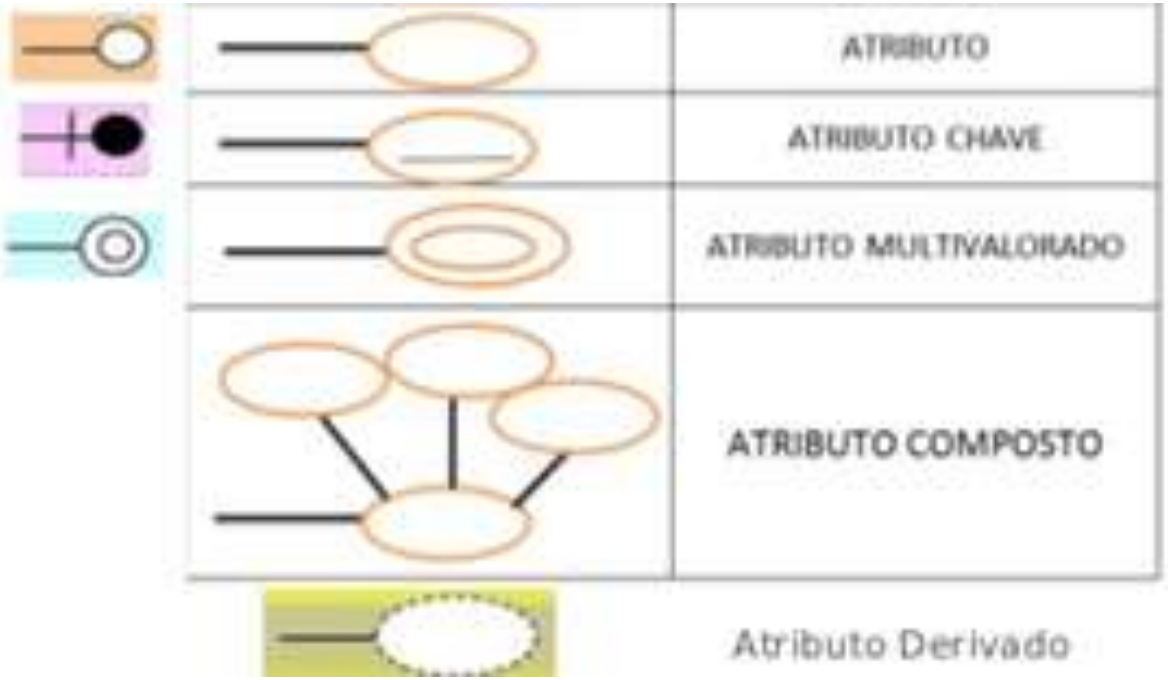
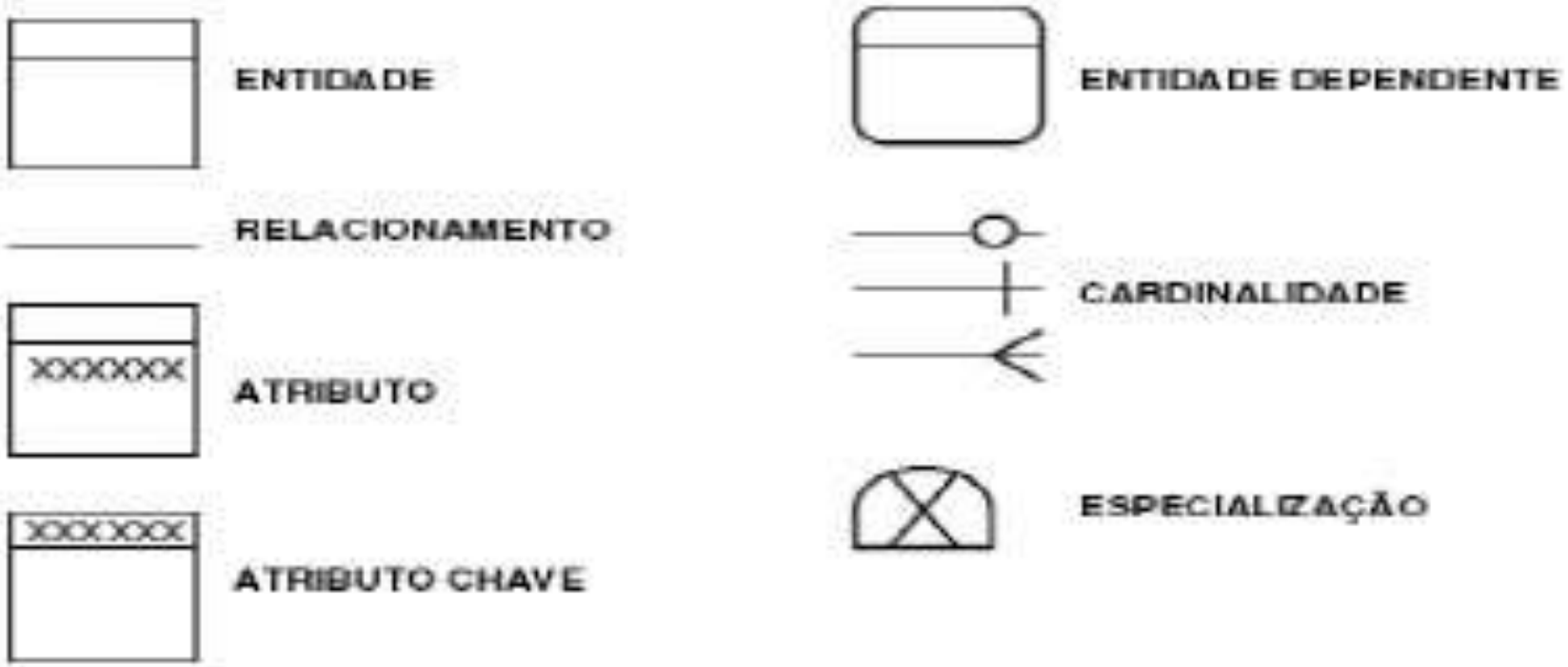
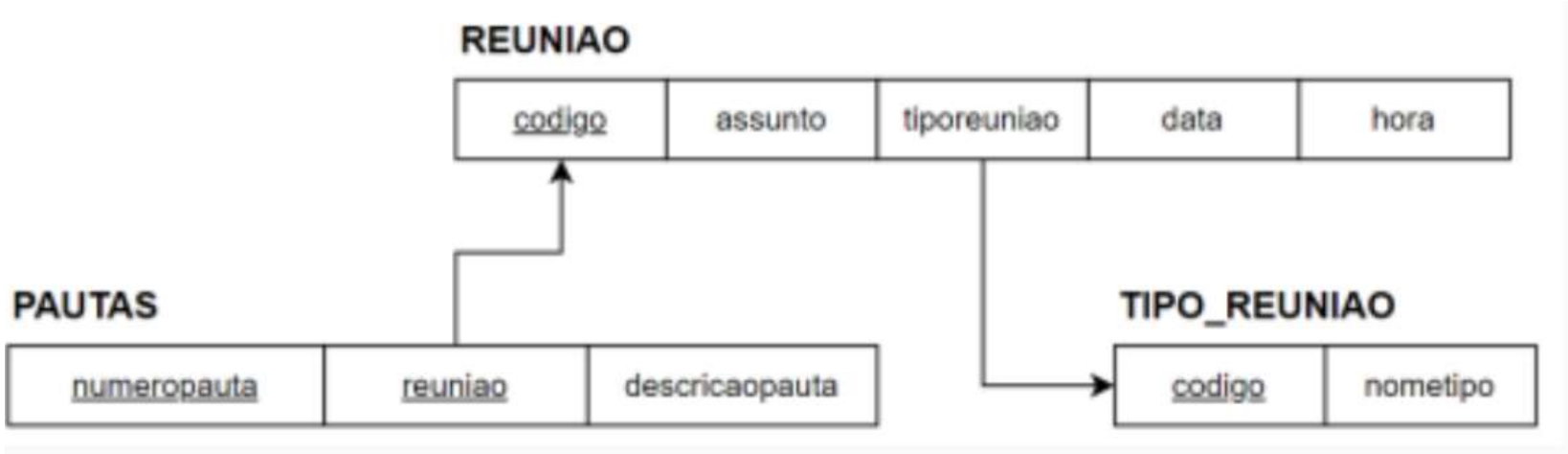


Diagrama ER – Notação James Martin



Chave primaria e estrangeira



Em relação ao esquema de banco de dados relacional, analise as seguintes afirmativas. Assina-le as corretas:

I. A chave primária da relação PAUTAS é “numeropauta”. (v)

II. O atributo “codigo” da relação TIPO_REUNIAO é chave primária e também chave estrangeira. (v)

III. O atributo “reuniao” da relação PAUTAS é uma chave estrangeira. (v)

Chave primaria

Considere que uma tabela, definida segundo o modelo relacional de dados, possui dados de pessoas físicas (chave primária: CPF).

PESSOA (CPF, nome, data-nascimento, email).

- O atributo RG (número do Registro Geral, principal documento de identificação de pessoas físicas no Brasil) precisa ser adicionado a essa tabela.

Diante disso, uma vez adicionado o atributo RG, verifica-se que:

- A) a chave primária da tabela passa a ser composta por CPF e RG.
- B) a tabela passará a ter duas chaves primárias.
- **C) o atributo CPF continuará a ser a chave primária.**
- D) o atributo RG precisa ser necessariamente definido como chave estrangeira.
- E) os domínios dos atributos CPF e RG precisam ser compatibilizados.

Entidade fraca (dependente)

Os conceitos de entidade fraca no modelo entidade relacionamento, considere como verdadeira (V) ou falsa (F) cada uma das afirmativas a seguir.

I. A existência de uma entidade fraca depende da existência de uma entidade forte.

II. A entidade que não tem chave primária é chamada de entidade fraca.

III. A chave primária de uma entidade fraca é composta pela chave primária da entidade forte e um ou mais atributos próprios. A alternativa correta é:

A) I – V, II – V, III – F.

B) B. I – V, II – F, III – V (gabarito)

C) I – F, II – V, III – F.

D) I – F, II – F, III – V.

E) I – F, II – F, III – F.

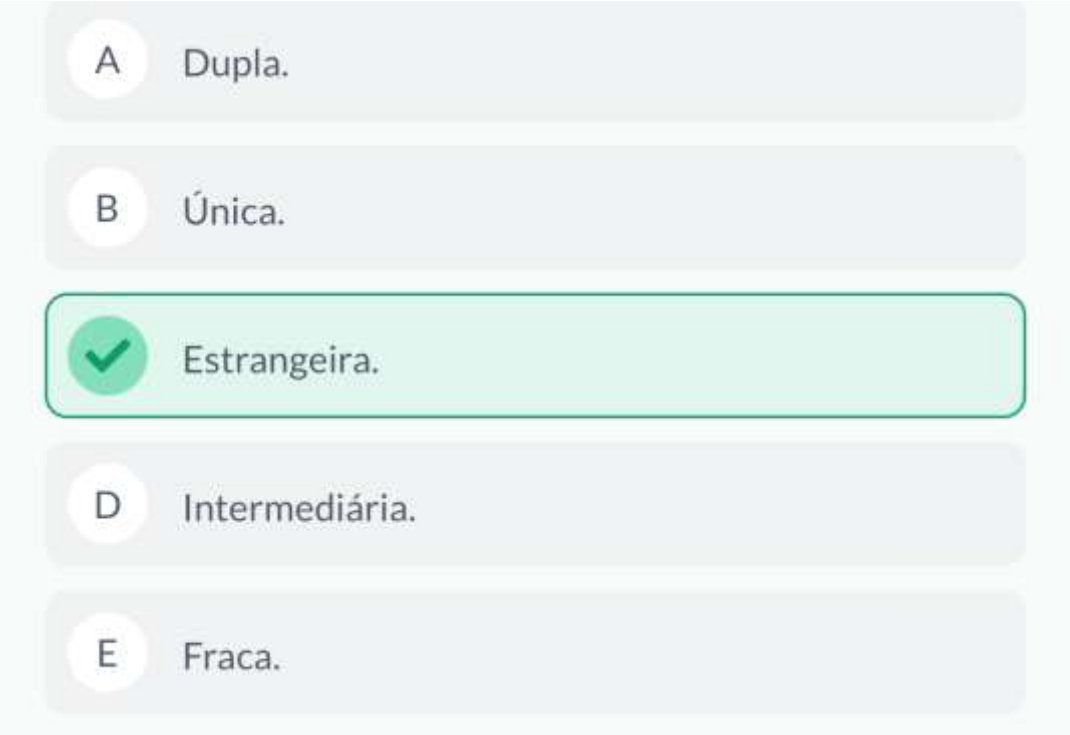
Exemplo: DER

O Diagrama de Entidades e Relacionamentos (DER) é uma ferramenta bastante conhecida e útil para a modelagem conceitual dos dados que um determinado sistema de informação deve tratar. Em particular, os relacionamentos, representados por meio de losangos no DER, são utilizados para indicar que:

- A) atributos de uma entidade têm uma relação mais forte entre si.
- B) atributos devem ser usados para identificar uma instância de uma entidade.
- C) dois, e apenas dois, relacionamentos estão relacionados entre si.
- **D) duas ou mais entidades estão relacionadas entre si.**
- E) entidades compartilham os mesmos atributos identificadores.

Exemplo – Chave estrangeira

- O tipo de chave composto por uma ou mais colunas de uma tabela, cujos valores aparecem necessariamente na chave primária de uma tabela, e que permite a implementação de relacionamentos em um banco de dados relacional, é chamado de chave:



Exemplo – Chave Estrangeira

O armazenamento de dados nos computadores modernos requer recursos de hardware sofisticados e rápidos. Considerando essa informação, julgue o item seguinte.

O controle de integridade referencial no modelo relacional garante que os dados de uma tabela nunca possam ser alterados ou deletados, independentemente de suas relações com outras tabelas.

- Certo
- **Errado (gabarito)**

Auto relacionamento

Na modelagem conceitual de dados, utilizando a abordagem Entidade-Relacionamento, são encontrados alguns padrões de relacionamentos. Trata-se do padrão de relacionamento que envolve conexões entre membros de um mesmo conjunto e que, às vezes, é chamado de relacionamento reflexivo:

A) Agregação

B) Cardinalidade

C) Generalização

D) Especialização

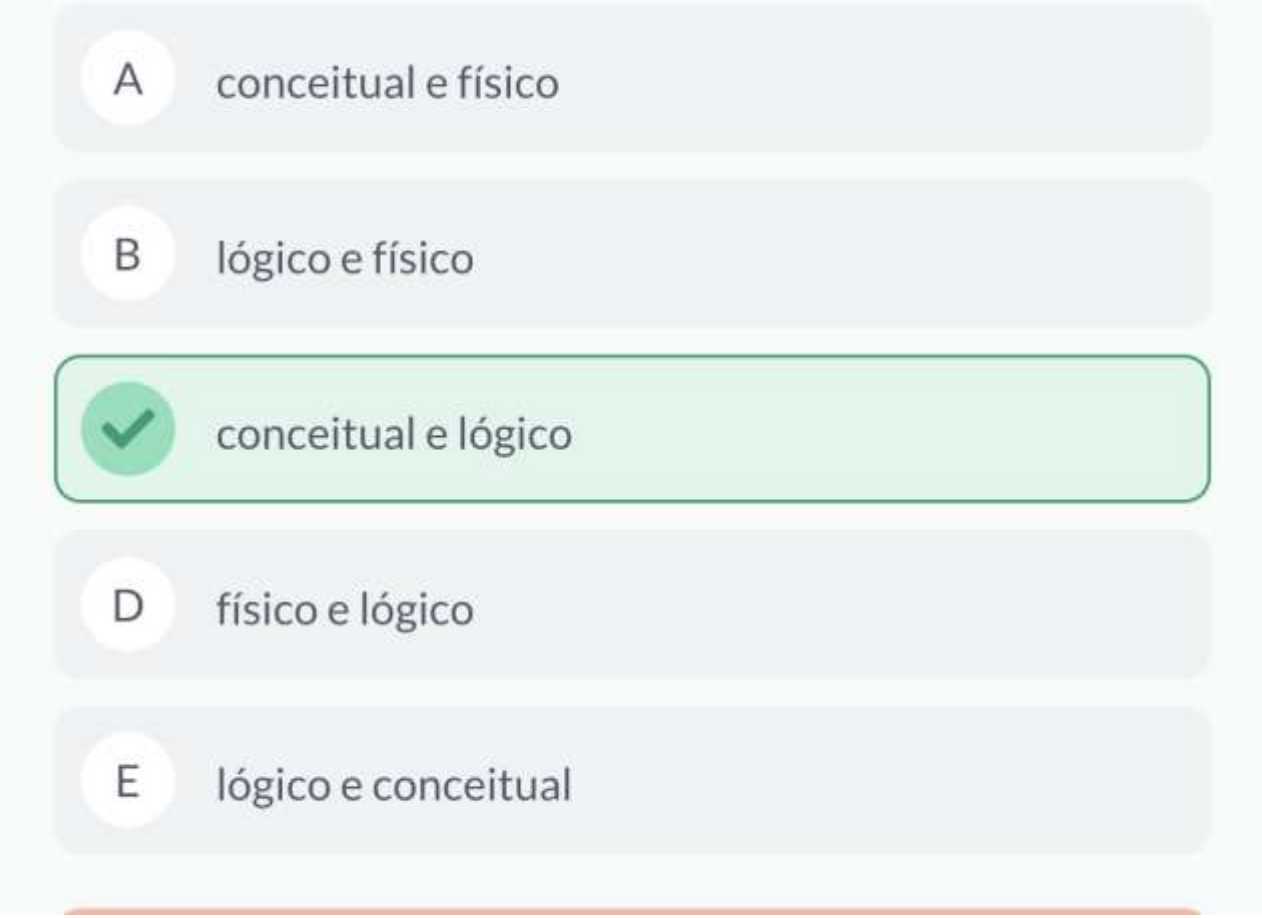
E) Auto-relacionamento (correto)

ANSI/SPARC: níveis conceitual vs lógico

Quanto aos esquemas utilizados em um projeto de banco de dados, analise as afirmações abaixo:

I. Descrição precisa dos requisitos do usuário, que utiliza o MER como modelo de dados.
II. Descreve as estruturas de representação do banco de dados.

III. Descreve as estruturas internas de armazenamento, dos esquemas, utilizando comandos DDL.



Exemplo:

Em um Modelo Entidade-Relacionamento (MER), os objetos básicos de um determinado contexto são modelados como entidades. Considere o seguinte contexto de uma empresa imobiliária:

"Um cliente aluga um imóvel. Um vendedor vende o imóvel de um cliente. A alocação do funcionário é em um setor. Um cliente tem um imóvel."

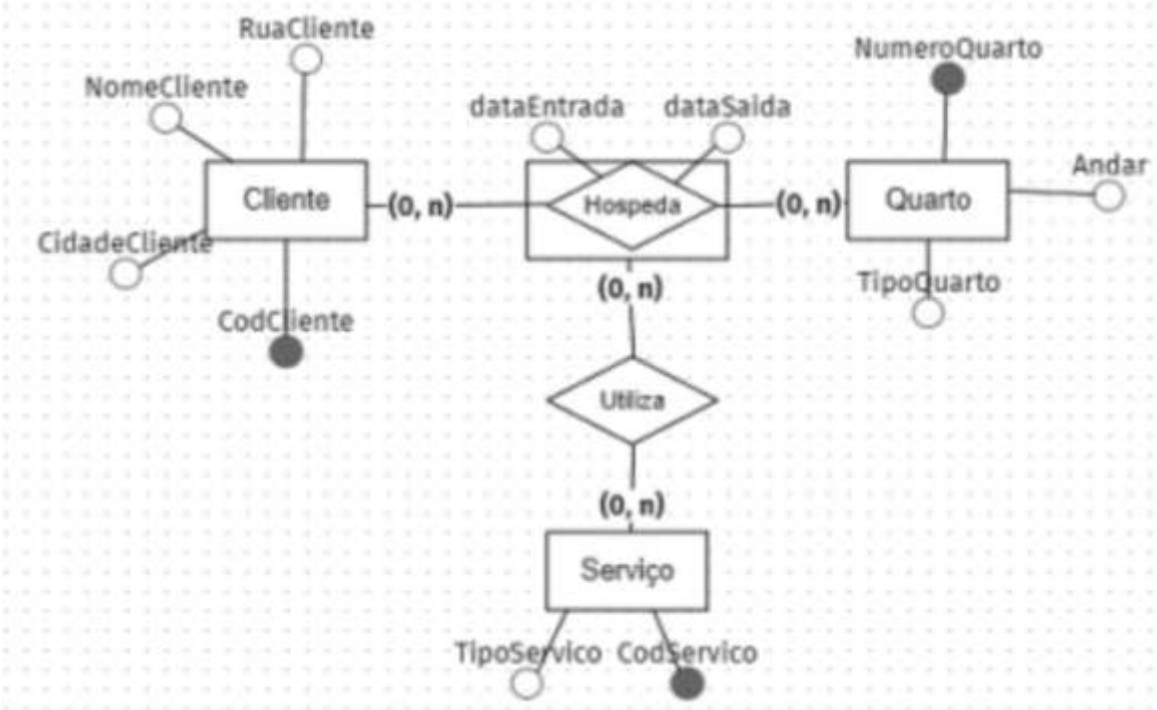
No contexto apresentado, o objeto básico que representa uma entidade é:

- A) Alocação;
- B) Cliente (certo) ;
- C) Aluga;
- D) Venda;
- E)Tem.

Ano: 2023 Banca: FGVÓrgão: Defensoria Pública do Estado do Rio Grande do Sul

Exemplo:

Analise o diagrama entidade-relacionamento de um determinado hotel:



Na transposição do Diagrama Entidade Relacionamento (DER) para o diagrama de tabelas relacionais, quais tabelas seriam criadas?

A) Cliente (CodCliente, NomeCliente, RuaCliente, CidadeCliente)Quarto (NumeroQuarto, TipoQuarto, Andar)Serviço (CodServico, TipoServico)

B) Cliente (CodCliente, NomeCliente, RuaCliente, CidadeCliente, DataEntrada, DataSaida)Quarto (NumeroQuarto, TipoQuarto, Andar, *CodCliente*)Serviço (CodServico, TipoServico, *CodCliente*)

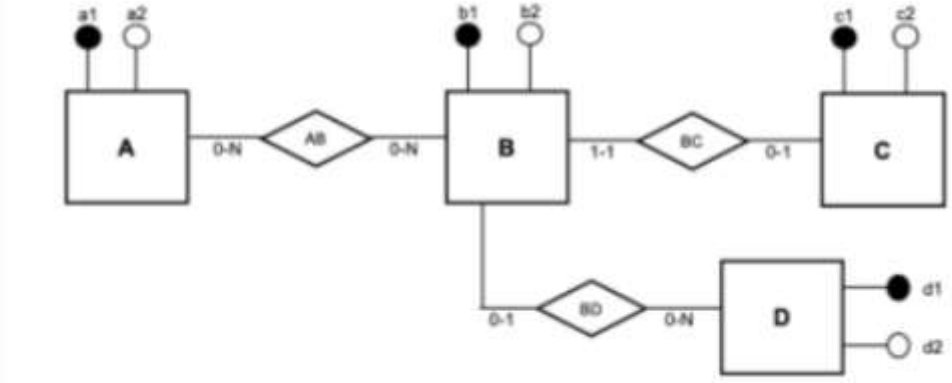
C) Cliente (CodCliente, NomeCliente, RuaCliente, CidadeCliente, DataEntrada, DataSaida)Quarto (NumeroQuarto, TipoQuarto, Andar, *CodCliente*)Utiliza (CodCliente, NumeroQuarto, CodServico)Serviço (CodServico, TipoServico)

D) Cliente (CodCliente, NomeCliente, RuaCliente, CidadeCliente)Hospeda (CodCliente, NumeroQuarto, CodHospedagem, DataEntrada, DataSaida)Quarto (NumeroQuarto, TipoQuarto, Andar)Serviço (CodServico, TipoServico, *CodHospedagem*)

E) Cliente (CodCliente, NomeCliente, RuaCliente, CidadeCliente)Hospeda (CodCliente, NumeroQuarto, DataEntrada, DataSaida)Quarto (NumeroQuarto, TipoQuarto, Andar)Utiliza (CodCliente, NumeroQuarto, CodServico)Serviço (CodServico, TipoServico)

Exemplo:

- Considere o diagrama Entidade-Relacionamento abaixo, desenhado de acordo com a notação proposta em Heuser (2009), baseada na proposta por Peter Chen com as extensões mais populares. As caixas representam as entidades, os losangos os relacionamentos, as cardinalidades dos relacionamentos são representadas por pares (min-max), e os "pirulitos" representam os atributos. Quando pintado de preto, um "pirulito" representa uma restrição de identificação.



Foi solicitado à projetista que fizesse um modelo lógico relacional correspondente a este DER. A projetista mapeou as entidades A, B, C e D nas tabelas A, B, C e D, respectivamente, cada qual com sua chave primária (atributos a1, b1, c1 e d1, respectivamente) e demais atributos (a2, b2, c2 e d2, respectivamente).Considere as seguintes assertivas sobre o mapeamento dos relacionamentos:

I. Deve ser criada uma tabela própria para representar o relacionamento AB, na qual deve haver colunas (chaves estrangeiras) referenciando as chaves primárias das tabelas A e B.
II. O relacionamento BD pode ser modelado como uma coluna adicional na tabela B (chave estrangeira referenciando a chave primária da tabela D).
III. O relacionamento BC pode ser modelado como uma coluna adicional (chave estrangeira) na tabela C ou B, sendo essas escolhas equivalentes. Se incluído na tabela C, esse atributo deve referenciar a chave primária da tabela B, e se incluído na tabela B, deve referenciar a chave primária da tabela C.

Quais estão corretas?

A) Apenas III
B) Apenas I e II
C) Apenas I e III
D) Apenas II e III
E) I, II e III

Exemplo:

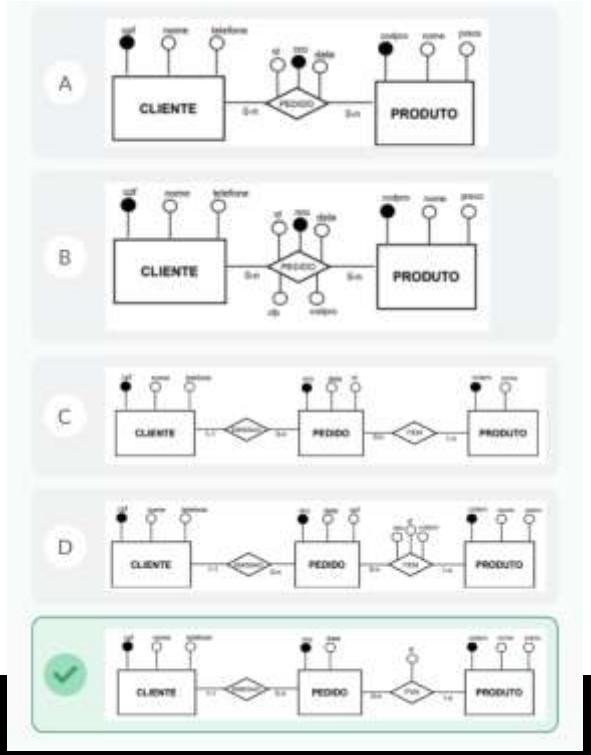
- Considere que uma empresa deseja controlar os pedidos que clientes fazem de seus produtos. Clientes são identificados por um CPF único, produtos por um código único (codpro) e pedidos por um número único (nro). Existem produtos que não fazem parte de pedidos, e clientes que não fizeram pedidos. Ao registrar um pedido de um cliente, no qual podem ser incluídos um ou mais produtos, deve ser possível gerar o recibo exemplificado abaixo.
- Foi solicitado a um estagiário fazer um projeto conceitual utilizando o modelo EntidadeRelacionamento (ER). Recebeu como recomendação que o projeto deveria conter os dados necessários para gerar este recibo, considerar o emprego correto do modelo de dados ER, e evitar redundâncias de dados que possam levar a problemas de integridade na base de dados.
- Utilizando a notação gráfica proposta por Heuser (2009), baseada na proposta por Peter Chen com as extensões mais populares, o estagiário desenhou vários diagramas ER (DER). As caixas representam as entidades, os losangos os relacionamentos, as cardinalidades dos relacionamentos são representadas por pares (min-max), e os "pirulitos" representam os atributos. Quando pintado de preto, um "pirulito" representa uma restrição de identificação.
- Qual DER melhor atende às recomendações recebidas?

Número do Pedido: 1234 **Data**: 01/02/2023

Cliente: Joao da Silva **cpf:** 123.456.789-89

Telefone: (051) 999887766

Qt	Cod Prod. - Nome	Preco Un.	Subtotal
1	123 – Achocolatado X	5,00	5,00
2	456 – Leite Y sem lactose	7,00	14,00
2	765 – Leite Z desnatado	5,00	10,00

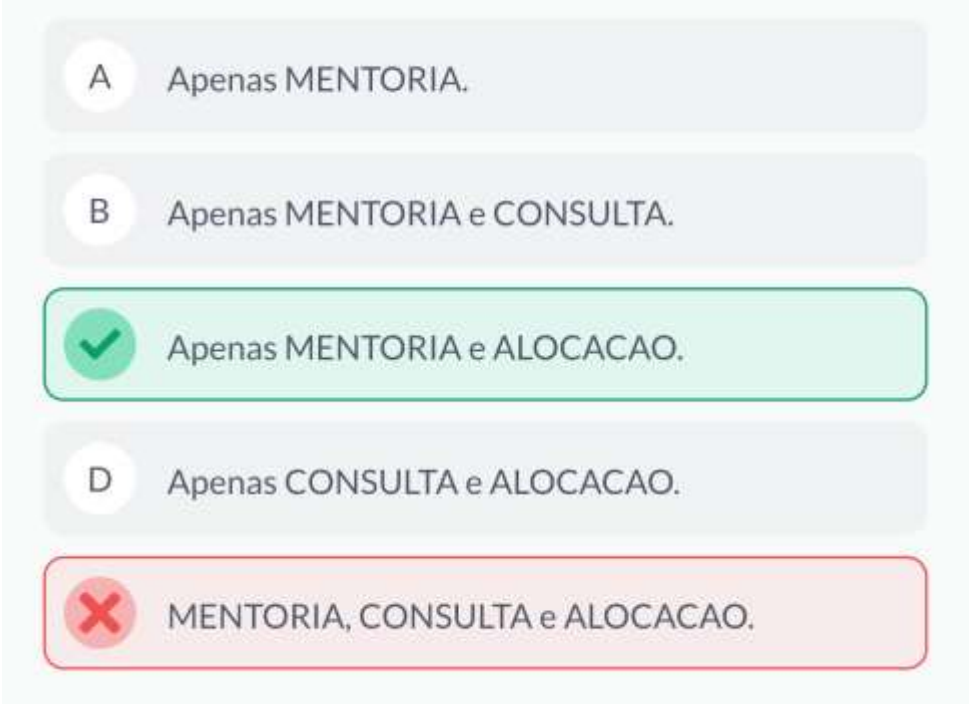


Exemplo

Considere o modelo de dados Entidade-Relacionamento (ER) e as situações descritas nos itens abaixo. Para cada situação, são fornecidos o nome de um relacionamento e de três entidades (identificados em letras maiúsculas), junto com a descrição de uma situação a ser modelada. Assuma sempre que as entidades indicadas são necessárias considerando o propósito da base de dados.

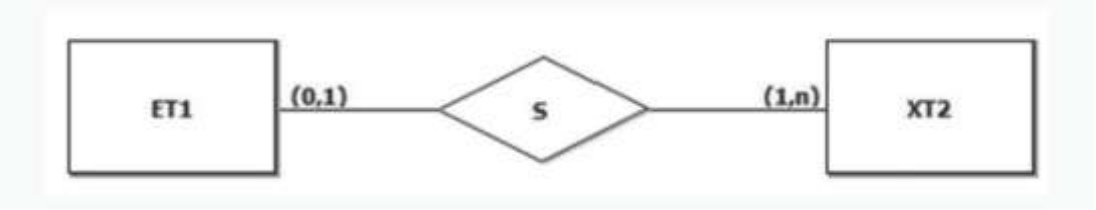
- I. Relacionamento MENTORIA, entidades ESTAGIARIO, MENTOR, e SETOR: em um programa de treinamento, um estagiário passa por vários setores da empresa, em cada um deles sendo atribuído a um mentor. Deseja-se registrar sobre cada mentoria, além do estagiário, do mentor, e do setor, a data de início e de fim da mentoria.
- II. Relacionamento CONSULTA, entidades MEDICO, PACIENTE, e EXAME: uma clínica médica oferece consultas e exames a seus pacientes. Deseja-se registrar sobre cada consulta realizada, além do médico e do paciente, a data/hora da consulta, a forma de pagamento, bem como possivelmente um ou mais exames solicitados durante a consulta.
- III. Relacionamento ALOCACAO, entidades PROJETO, FUNCIONARIO, e FUNCAO: uma empresa de TI desenvolve diferentes projetos, na qual aloca seus funcionários em uma dada função. Deseja-se registrar sobre cada alocação, além do funcionário, do projeto e da função, o número de horas alocado.

Qual dos relacionamentos acima poderia ser corretamente modelado por um relacionamento ternário entre as três entidades participantes indicadas?



Exemplo:

- A Figura abaixo exibe um diagrama E-R que contém duas entidades (ET1 e XT2) e uma relação (S).
- Admita a existência de um banco de dados relacional composto pelos conjuntos ET1 e XT2, oriundos do mapeamento das entidades ET1 e XT2. Parte do estado atual desse banco de dados é definido pelos seguintes conjuntos:
- ET1={e1,e2,e4}XT2={t1,t2,t3}
- Qual conjunto completa o banco de dados em questão, de modo a atender a todas as regras relativas à relação S?



A S={ }

B S={ (e1,t1), (e2,t2), (e2,t3) }

C S={ (e1,t1), (e2,t2), (e4,t1) }

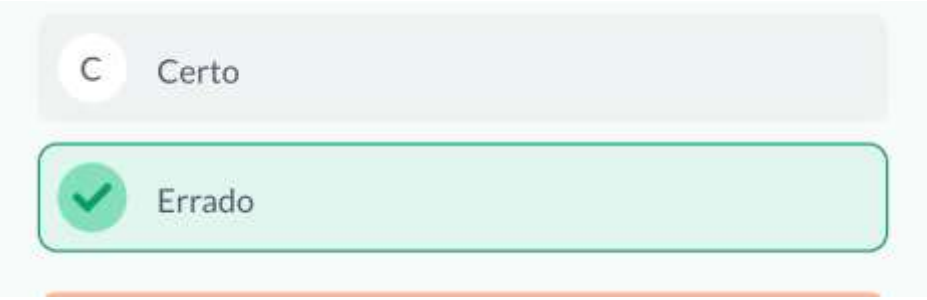
D S={ (e1,t1), (e1,t2), (e4,t3), (e4,t2) }

✓ S={ (e1,t3), (e2,t2), (e4,t1) }

Exemplo:

Julgue o item subsecutivo, que se referem a conceitos de programação e banco de dados.

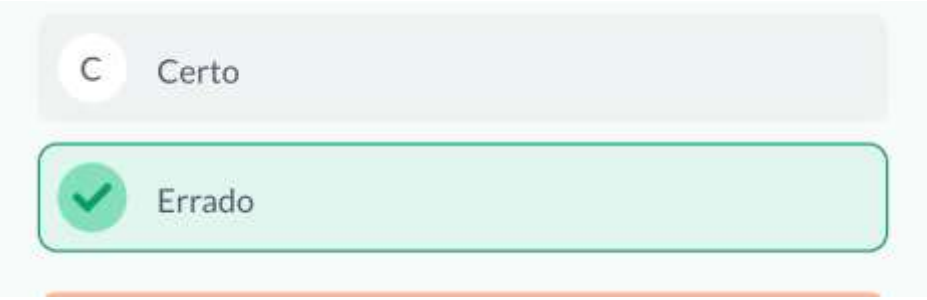
As estruturas para armazenamento dos dados e os métodos de acesso ao banco de dados fazem parte do projeto lógico de um banco de dados.



Exemplo:

Julgue o item subsecutivo, que se referem a conceitos de programação e banco de dados.

Um registro é um conjunto de itens de dados que possuem um conjunto de atributos que pertencem a determinada entidade.



Exemplo:

- Sobre a abordagem entidade-relacionamento, é correto afirmar que:

A Relacionamento é o conjunto de objetos da realidade modelagem sobre os quais se deseja manter informações no banco de dados.

B Identificador de entidade é o dado que é associado a cada ocorrência de uma entidade ou de um relacionamento.

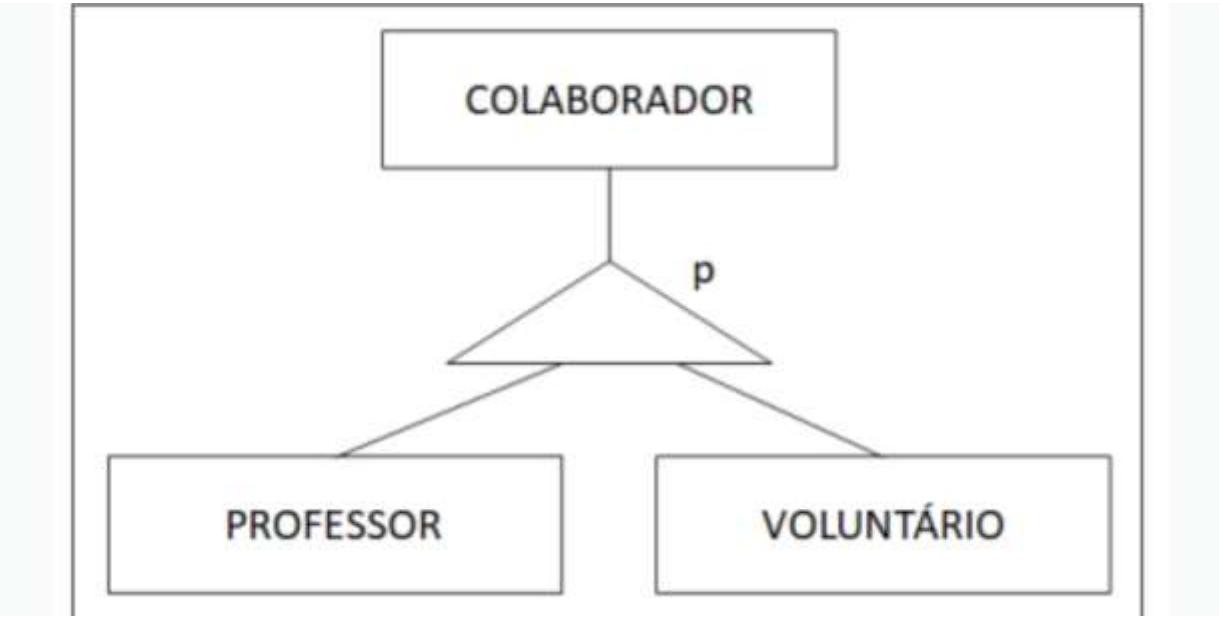
C Generalização é o conjunto de um ou mais atributos e relacionamentos cujos valores servem para distinguir uma ocorrência da entidade das demais ocorrências da mesma entidade.

D Entidade é o conjunto de associações entre ocorrências de relacionamentos.

✓ Papel de entidade em relacionamento é a função que uma instância da entidade cumpre dentro de uma instância do relacionamento.

Exemplo:

- A Figura 1 abaixo apresenta um Diagrama de Entidade Relacionamento (DER):



A Todo PROFESSOR é VOLUNTÁRIO.

B Todo COLABORADOR é PROFESSOR e VOLUNTÁRIO.

Todo COLABORADOR é PROFESSOR ou VOLUNTÁRIO.

Nem todo COLABORADOR é PROFESSOR ou VOLUNTÁRIO.

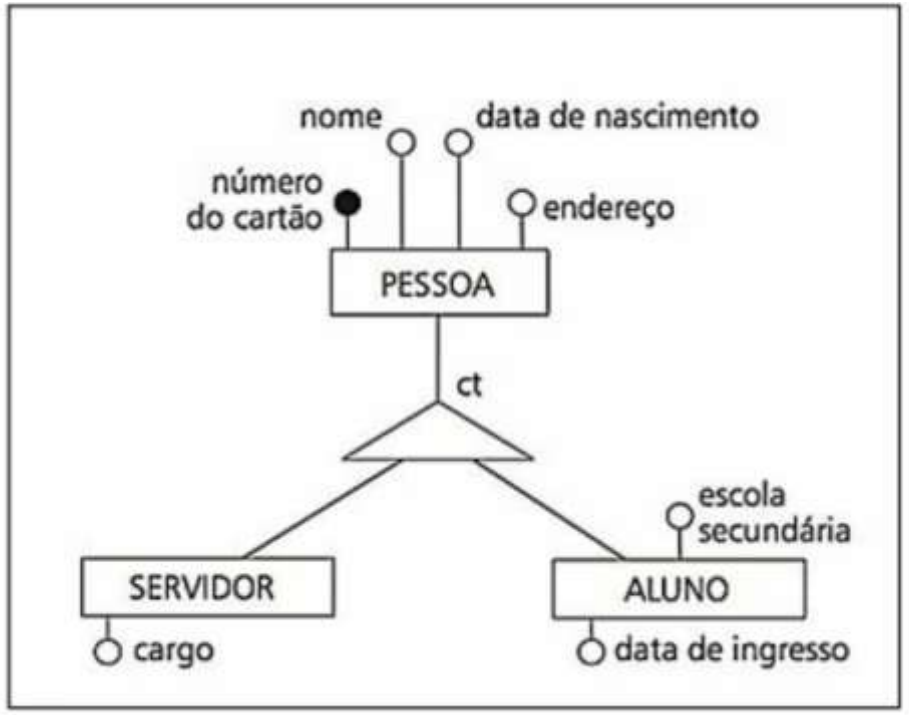
E Nem todo PROFESSOR é COLABORADOR.

Exemplo

Observe o diagrama a seguir:

(Heuser, 2011, p. 83.)

Este modelo representa uma



A entidade associativa.

✓ generalização/especialização com chave total da entidade Pessoa.

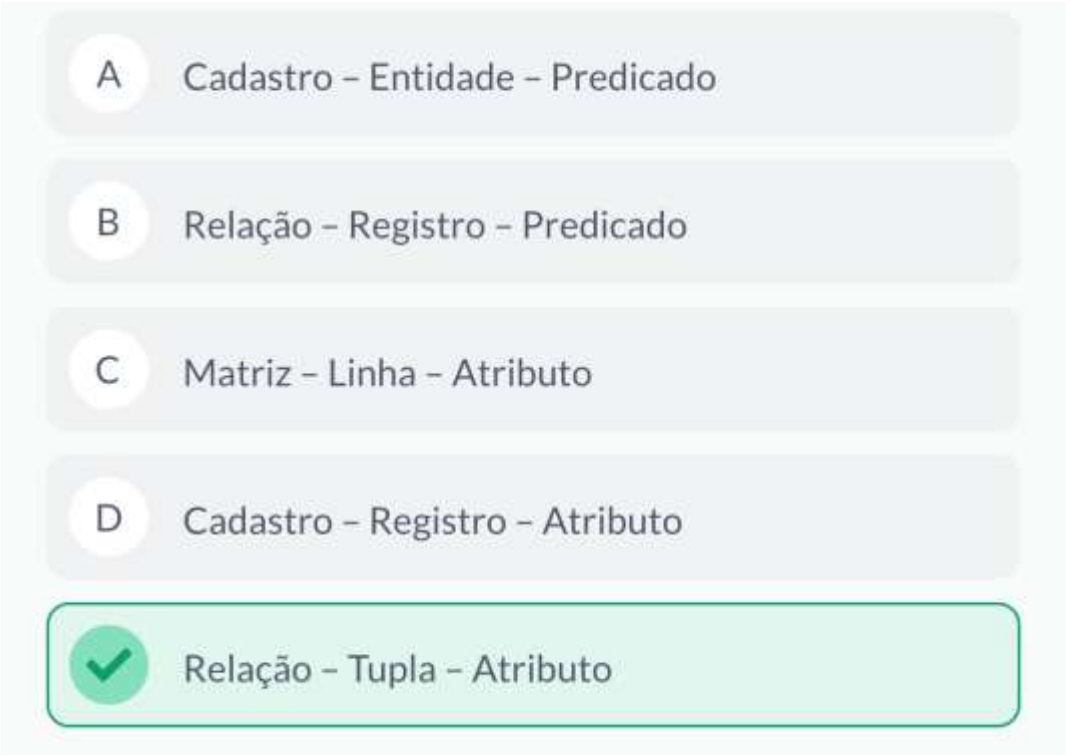
C entidade associativa compartilhada com herança de propriedades.

D generalização/especialização compartilhada e total da entidade Pessoa.

Exemplo

Considere as seguintes definições do contexto dos bancos de dados relacionais, com lacunas a preencher, e assinale a alternativa que preencha corretamente as três definições, considerando sua ordem numérica.

- 1. _____: É uma tabela que armazena dados organizados em linhas e colunas. Cada linha representa uma entrada única e cada coluna representa um atributo.
- 2. _____: É uma linha na tabela, representando uma única entrada de dados.
- 3. _____: É uma coluna na tabela, representando uma característica ou propriedade de cada entrada de dados.



Exemplo - Chave

Assinale a alternativa correta sobre o conceito de chaves primárias em bancos de dados relacionais.

- A) A chave primária pode ser uma coluna que não é obrigatória.
- B) A chave primária pode conter valores nulos.
- C) A chave primária é uma coluna ou uma combinação de colunas cujos valores distinguem uma linha das demais dentro de uma tabela. (correta)
- D) A chave primária é uma coluna que pode ter valores duplicados dentro de uma tabela.
- E) A chave primária pode ser compartilhada entre várias tabelas.

Ano: 2023 Banca: UNESPAR Órgão: Universidade Estadual do Paraná - UNESPAR

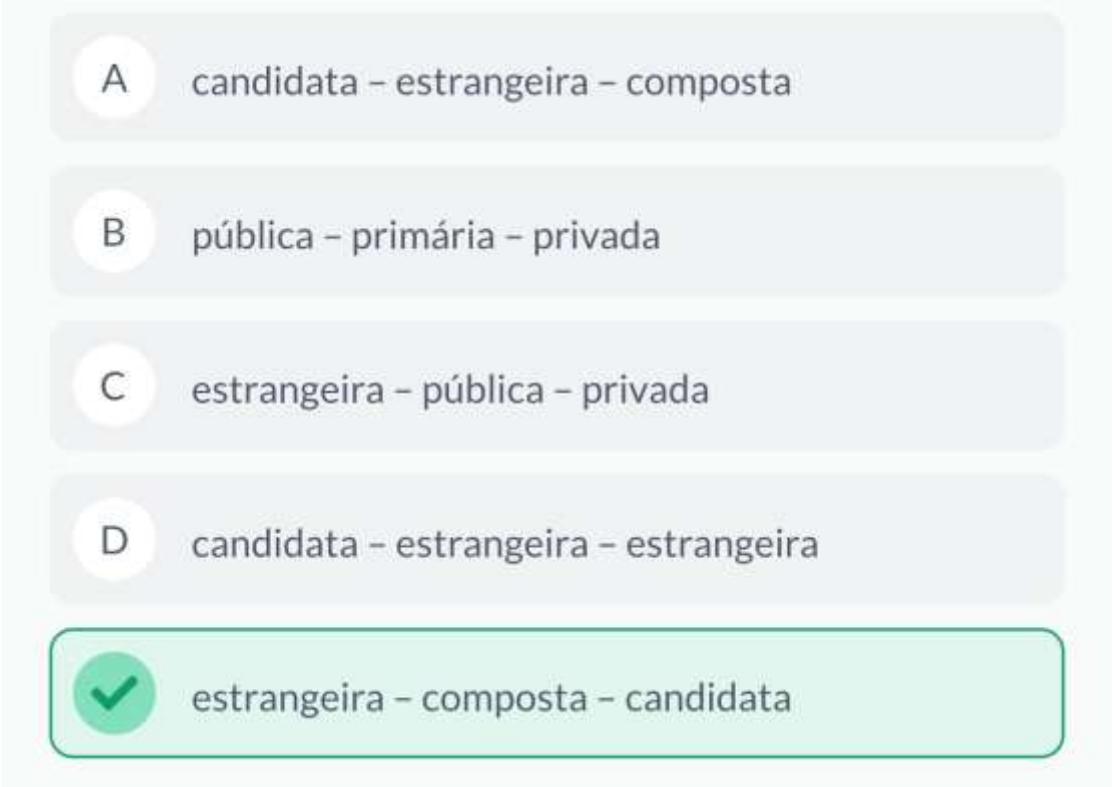
Exemplo: tuplas e relações

- Em bancos de dados relacionais, a restrição de integridade referencial é utilizada para manter a consistência entre tuplas nas relações. Assinale a alternativa que se refere ao conceito de restrição de integridade referencial.
- **A) Uma tupla em uma relação que faz referência a outra relação, deve se referir a uma tupla existente nessa outra relação.(gabarito)**
- B) Garante que o valor de um campo deve estar dentro de um range fixo predefinido.
- C) Uma relação deve envolver pelo menos 2 entidades.
- D) Uma chave primária não pode ser nula.

Exemplo

Considere as seguintes definições do contexto dos bancos de dados relacionais, com lacunas a preencher, e assinale a alternativa que preencha corretamente as três definições, considerando sua ordem.

- 1. Chave _____: Faz referência a uma chave primária em outra tabela, criando uma ligação entre as tabelas.
- 2. Chave _____: É uma chave que é composta por mais de uma coluna e que juntas formam uma chave única.
- 3. Chave _____: Pode ser utilizada como uma chave primária, mas pode não ser selecionada para tal fim devido a outros motivos, como tamanho ou performance.



Exemplo: chave estrangeira

Considere os seguintes comandos SQL foram executados em um SGBD relacional com restrição de integridade referencial, em um banco de dados vazio, para criar tabelas:

CREATE TABLE Setor (idSetor INT PRIMARY KEY, nomeSetor VARCHAR(50)); CREATE TABLE Funcionario (idFunc INT PRIMARY KEY, nome VARCHAR(255), setor INT, FOREIGN KEY (setor) REFERENCES Setor(idSetor));

O que acontece ao tentar remover uma tupla na tabela setor, caso exista uma tupla na tabela Funcionário cujo valor da coluna setor seja igual ao atributo idSetor, que foi removido?

A) A tupla também será removida na tabela funcionário.

B) O campo setor será preenchido como nulo.

C) O campo setor será atualizado para o valor default.

D) Não é possível remover a tupla, uma vez que ela é referenciada por outra tabela. (gabarito)

Atividade

Faça o diagrama ER que represente as entidades e relacionamentos da seguinte situação:

- Unidade (cidade: varchar(40))
- Produto (descrição: varchar(50), preço: float)
- Vendedor (nome: varchar(60), matricula: int)
- Venda (unidade: Unidade; produto: Produto, quantidade: float; vendedor: Vendedor)
- Vinculo (vendedor: Vendedor, unidade: Unidade, data_inicio: date, data_fim: date)

Aula 3

Algebra relacional e SQL

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Algebra relacional

Como realizar operações entre entidades e seus relacionamentos?

Álgebra relacional

A Álgebra Relacional é uma linguagem de consulta formal, porém procedimental, ou seja, o usuário dá as instruções ao sistema para que o mesmo realize uma sequência de operações na base de dados para calcular o resultado desejado.

Símbolo	*Operação*	*Sintaxe*
σ	Seleção / Restrição	$\sigma_{\text{condição}}$ (Relação)
π	Projeção	$\pi_{\text{expressões}}$ (Relação)
∪	União	Relação1 ∪ Relação2
∩	Intersecção	Relação1 ∩ Relação2
-	Diferença de conjuntos	Relação1 - Relação2
x	Produto cartesiano	Relação1 x Relação2
|x|	Junção	Relação1 |x| Relação2
÷	Divisão	Relação1 ÷ Relação2
ρ	Renomeação	ρ_{nome} (Relação)
←	Atribuição	variável ← Relação

Terminologia:

- Relação = Tabela (entidade)
- Tupla = Linha da tabela (registro)
- Pi = projeção π
- Sigma = seleção (filtro) σ

Source: https://dhg1h5j42swfq.cloudfront.net/2017/02/03143346/Algebra-relacional.gif

Álgebra relacional

Quadro Resumo de Operações/Operadores em Álgebra Relacional:

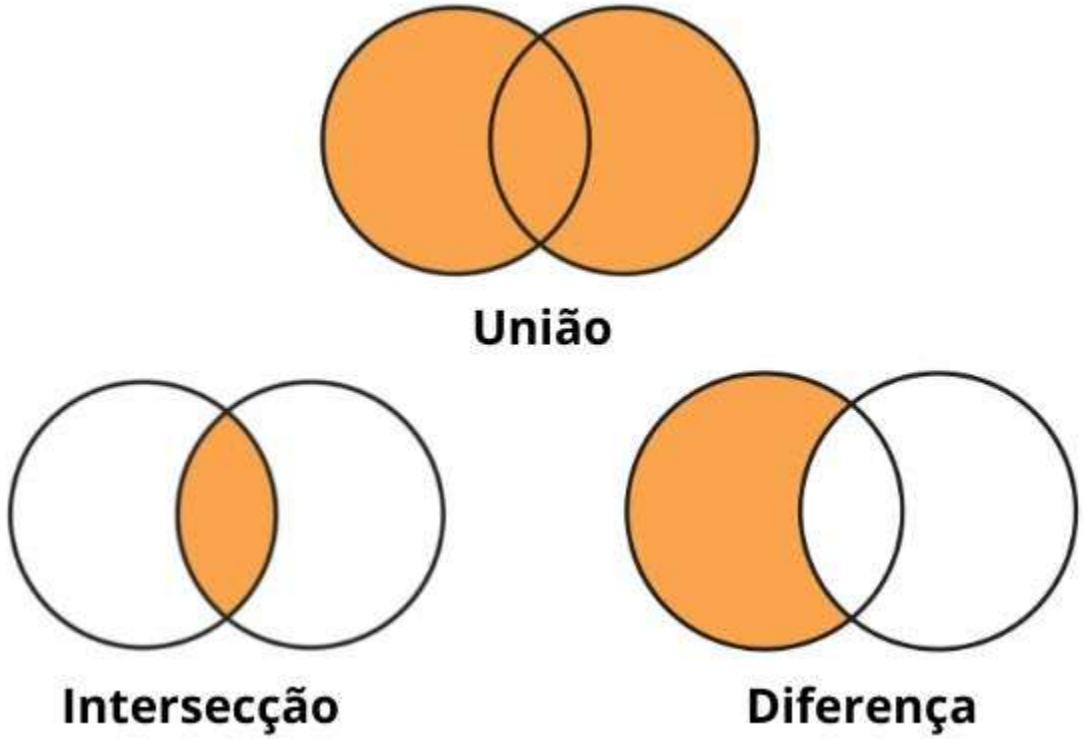
OPERAÇÃO	SÍMBOLO	SINTAXE
Projeção	π ("pi")	π <lista de campos> (Tabela)
Seleção/ Restrição	σ ("sigma")	σ <condição de seleção> (Tabela)
União	∪	(Tabela 1) ∪ (Tabela 2)
Interseção	∩	(Tabela 1) ∩ (Tabela 2)
Diferença	–	(Tabela 1) – (Tabela 2)
Produto Cartesiano	X	(Tabela 1) X (Tabela 2)
Junção	|X|	(Tabela 1) |X| <condição de junção> (Tabela 2)
Divisão	÷	(Tabela 1) ÷ (Tabela 2)
Renomeação	ρ ("rho")	ρ Nome(Tabela)
Atribuição	←	Variável ← Tabela

Source: https://pedrogalvaojunior.wordpress.com/2016/05/26/dica-do-mes-conhecendo-a-algebra-relacional-e-seus-operado

Álgebra relacional

Aplica-se a teoria dos conjuntos:

União	∪	(Tabela 1) ∪ (Tabela 2)
Interseção	∩	(Tabela 1) ∩ (Tabela 2)
Diferença	–	(Tabela 1) – (Tabela 2)



Source: https://pedrogalvaojunior.wordpress.com/2016/05/26/dica-do-mes-conhecendo-a-algebra-relacional-e-seus-operado

Álgebra relacional

Álgebra	SQL
(Pacientes)	Select * From Pacientes
$\sigma_{\text{idade}> 18}$ (Pacientes)	Select * From Pacientes Where idade > 18
$\pi_{\text{CPF, nome}}$ (Pacientes)	Select CPF, nome From Pacientes
$\pi_{\text{CPF, nome}}$ ($\sigma_{\text{idade}> 18}$ (Pacientes))	Select CPF, nome From Pacientes Where idade > 18

Algebra relacional - Exemplo

Considere o comando SQL a seguir.

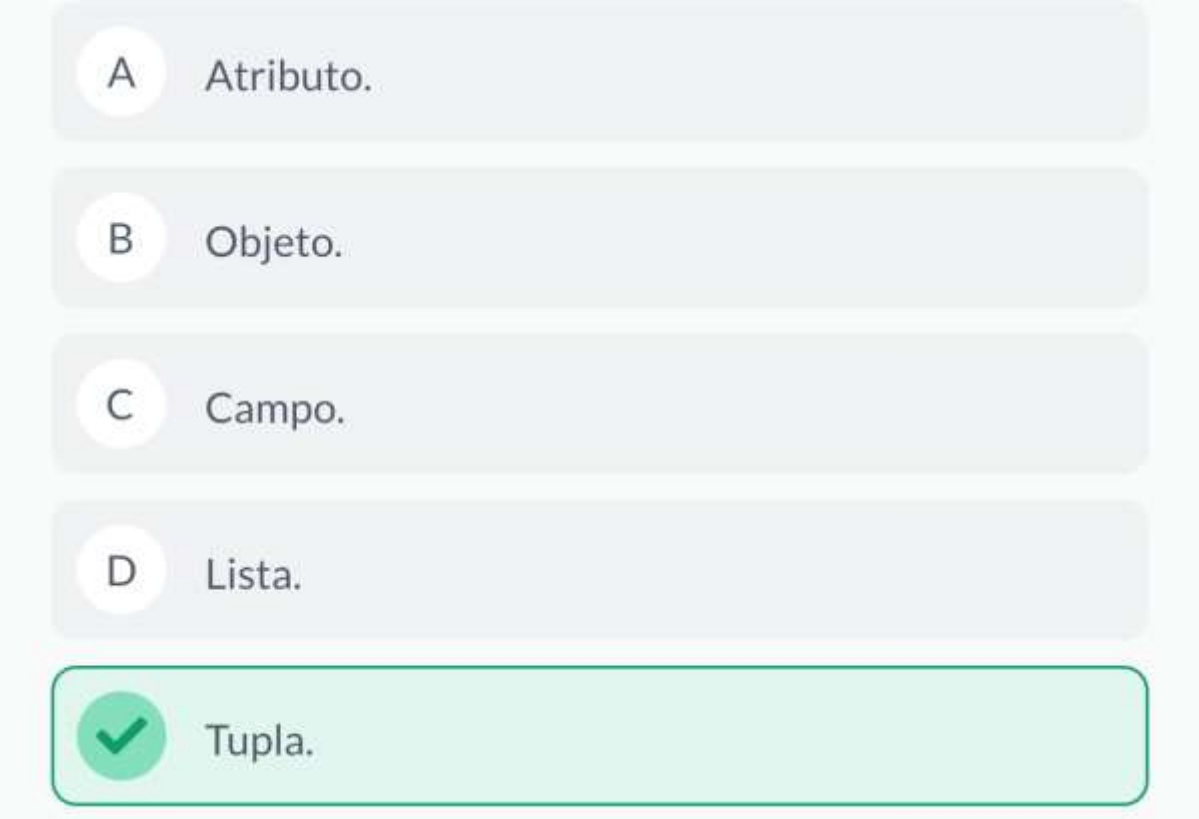
- SELECT a.X, b.Y FROM T1 a, T2 b WHERE a.R = b.S

Dado que essa consulta pode ser expressa usando as operações primitivas da Álgebra Relacional, a lista que contém as operações necessárias e suficientes para essa conversão é, em ordem alfabética:



Algebra relacional - Vocabulário

Em um banco de dados relacional, uma tabela é um conjunto não ordenado de linhas. Cada uma dessas linhas também é conhecida como:



Algebra Relacional - consulta simples

SELECT * FROM Produtos WHERE Preco > 0.00;

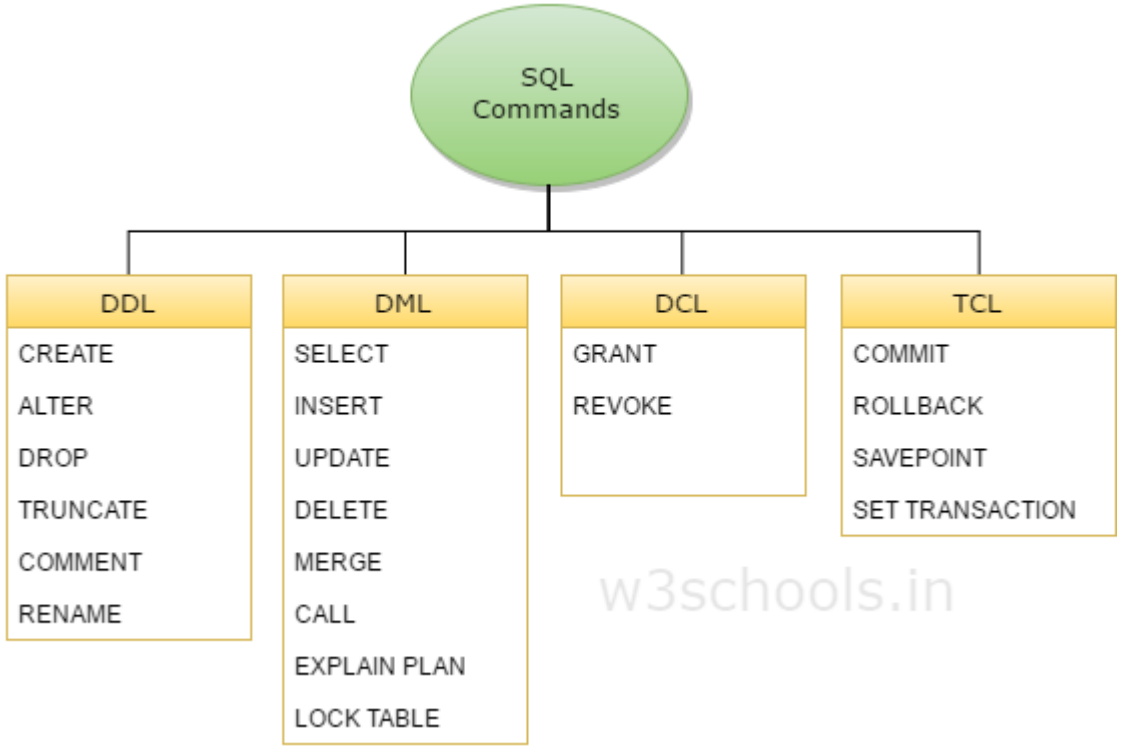
Considerando trecho de código SQL precedente, julgue o seguinte item.

O retorno do comando apresentado será necessariamente **uma lista de todos** os itens da tabela Produtos, ordenados pelo campo Preco, do menor valor para o maior valor.

- Certo
- **Errado (gabarito)**

Visão Geral SQL

- DDL: Data Definition Language
- DML: Data Manipulation Language
- DCL: Data Control Language
- TCL: Transaction Control Language



Source: https://dfilitto.com.br/banco-de-dados/o-que-sao-ddl-dml-dcl-e-tcl/

SQL: subdivisões da linguagem

Sobre os comandos SQL, analise as assertivas abaixo:

I. Comandos DML são utilizados para manipular dados no banco de dados.

II. Comandos DDL são utilizados para definir a estrutura do banco de dados.

III. Comandos DCL são utilizados para controlar o acesso aos dados.

IV. Comandos DTL são utilizados para **definir** as transações no banco de dados.

V. Comandos DQL são utilizados para consultar dados no banco de dados.

Quais estão **INCORRETAS**?

A) Apenas I.

B) Apenas II.

C) Apenas IV. (gabarito)

D) Apenas I e III.

E) Apenas III e V.

O que conseguimos fazer com SQL (DML):

Pesquisa (DQL):

- Filtro
- Projeção
- Junção (cruzamento de dados)
- Agrupamento (categorização)
- Agregação (somatórios, média, mediana, desvio padrão, etc.)
- Ordenação

Escrita: inserção, deleção, atualização.

DER x SQL

No modelo relacional, a transformação do Diagrama Entidade-Relacionamento (DER) para o esquema relacional envolve a derivação de ____ para tabelas, ____ para colunas e ____ para restrições de integridade.

Assinale a alternativa que preenche, correta e respectivamente, as lacunas do trecho acima.

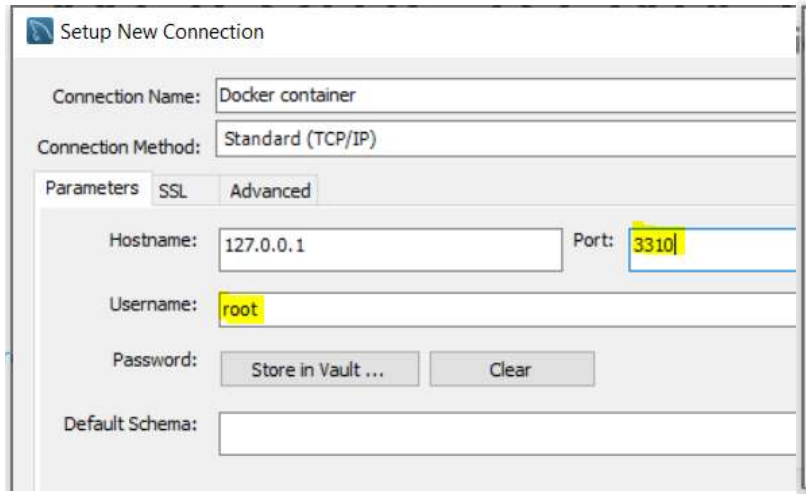
- **A) entidades – atributos – relacionamentos (gabarito)**
- B) relacionamentos – entidades – atributos
- C) entidades – relacionamentos – atributos
- D) atributos – entidades – relacionamentos
- E) relacionamentos – atributos – entidades

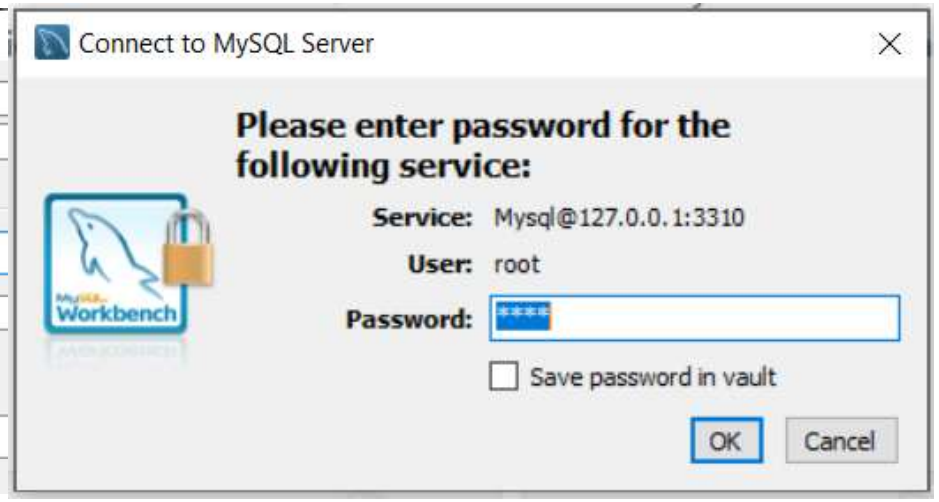
Docker run: MySQL example

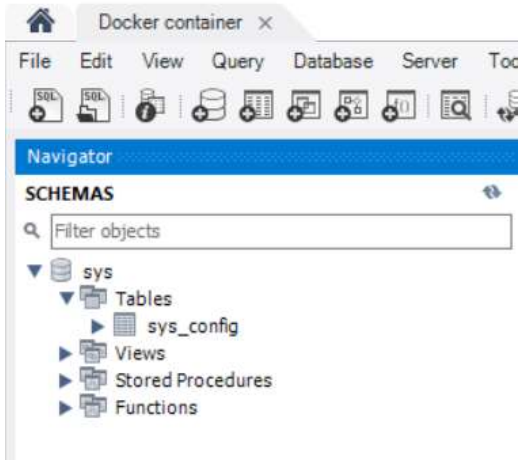
```
docker run --name mysql-serv1 -e MYSQL_ROOT_PASSWORD=root -d -p 3310:3306 -v mysql_data:/var/lib/mysql mysql
```

-e: variáveis de ambiente
-d: detached
-p: mapeamento de portas host:container

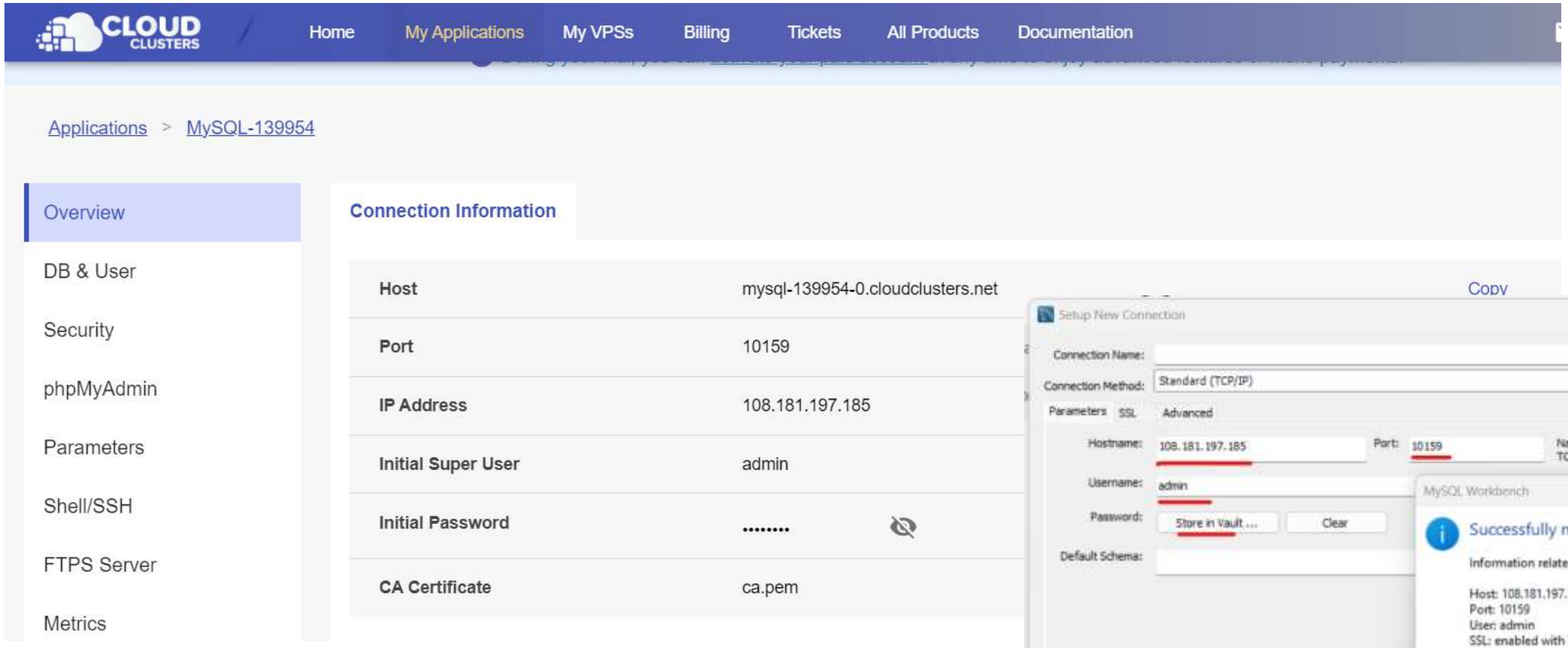
```
C:\Users\rafae>docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                NAMES
7738d2fa2410   mysql     "docker-entrypoint.s…"   10 seconds ago  Up 8 seconds   33060/tcp, 0.0.0.0:3310->3306/tcp    mysql-tacs
```

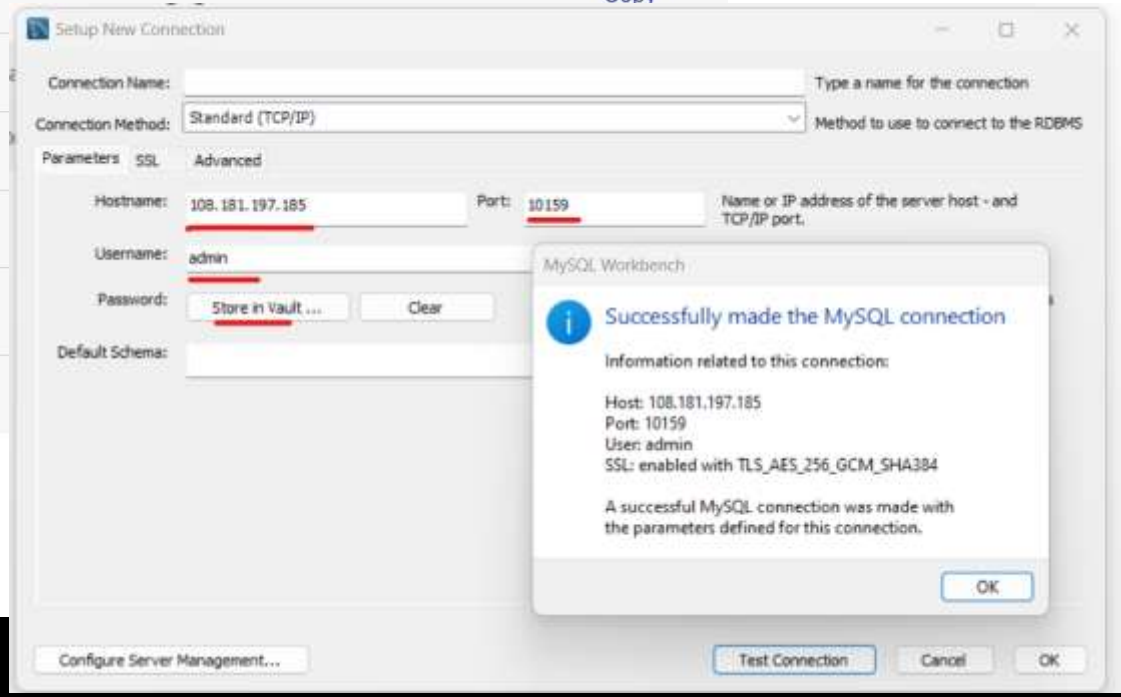






MySQL - Cloud


CLOUD CLUSTERS
Home
My Applications
My VPSs
Billing
Tickets
All Products
Documentation
Applications > MySQL-139954
Overview
DB & User
Security
phpMyAdmin
Parameters
Shell/SSH
FTPS Server
Metrics
Connection Information
Host
mysql-139954-0.cloudclusters.net
Copy
Port
10159
IP Address
108.181.197.185
Initial Super User
admin
Initial Password
.....
CA Certificate
ca.pem


Setup New Connection
Connection Name:
Type a name for the connection
Connection Method:
Standard (TCP/IP)
Method to use to connect to the RDBMS
Parameters
SSL
Advanced
Hostname:
108.181.197.185
Port:
10159
Name or IP address of the server host - and TCP/IP port.
Username:
admin
Password:
Store in Vault ...
Clear
Default Schema:
MySQL Workbench
Successfully made the MySQL connection
Information related to this connection:
Host: 108.181.197.185
Port: 10159
User: admin
SSL: enabled with TLS_AES_256_GCM_SHA384
A successful MySQL connection was made with the parameters defined for this connection.
OK
Configure Server Management...
Test Connection
Cancel
OK

Atividade – Configurando SGBD MySQL

- Reconhecendo servidor e cliente
- Acesso ao SGBD – IP/Porta/Login/Senha

Versão online para testes básicos:
https://sqliteonline.com/

SQL: DDL

```
create database espaco_virtual;

use espaco_virtual;

create table localidades_clientes(
    chave INT auto_increment primary key,
    nome_localidade varchar(50),
    cep varchar(9),
    endereco varchar(50)
);
```

Como comentar código em SQL?

- Considerando o sistema gerenciador de banco de dados MySQL 9.2, assinale a alternativa que apresenta, corretamente, uma maneira de se declarar um comentário nesse gerenciador.
- A) % Comentário
- **B) # Comentário (gabarito)**
- C) Comentário
- D) ! Comentário
- E) -* Comentário

No SQL, pode fazer comentários de três maneiras:

- -- (dois hífens) para comentários de uma linha.
- # para comentários de uma linha.
- /* */ para comentários de várias linhas.

A sintaxe -- encerra o comentário automaticamente no final da linha, enquanto /* */ deve ter um */ explícito para fechar o bloco de comentário.

Entendendo as restrições de integridade:

Relacione a Coluna 1 à Coluna 2, associando as categorias de restrições de integridade da abordagem relacional às suas definições.

Coluna 1

- 1. Integridade de domínio.
- 2. Integridade de vazio.
- 3. Integridade de chave.
- 4. Integridade referencial.

Coluna 2

( ) Especificam se o valor de um campo deve obedecer a definição de valores admitidos para a coluna.

( ) Especificam se os campos de uma coluna são opcionais ou obrigatórios.

( ) Definem que os valores devem ser únicos.

( ) Definem se os valores dos campos que aparecem em uma chave estrangeira devem aparecer na chave primária da tabela referenciada.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

A 2 – 3 – 4 – 1.

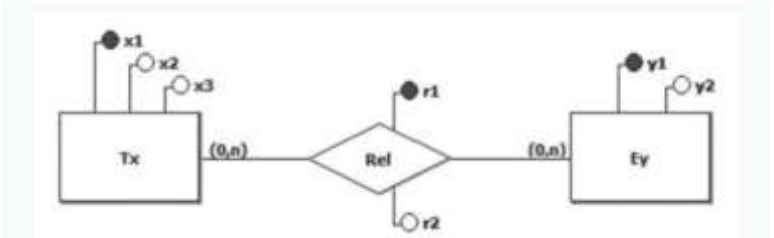
✓ 1 – 2 – 3 – 4.

C 3 – 2 – 1 – 4.

D 4 – 1 – 2 – 3.

E 1 – 4 – 3 – 2.

ER para SQL



Considere o seguinte diagrama E-R:

Foi criado um conjunto de tabelas relacionais, a partir do modelo E-R acima. Uma vez que as regras de transformações de entidades e relações para tabelas relacionais independem dos tipos de dados dos atributos, todos os atributos do modelo E-R acima foram tratados como itens de dados do tipo cadeia de caracteres (TEXT).

As tabelas resultantes são as seguintes:

```
CREATE TABLE TX (
      X1          TEXT          NOT NULL,
      X2          TEXT          NOT NULL,
      X3          TEXT          NOT NULL,
  PRIMARY KEY (X1));
CREATE TABLE EY (
      Y1          TEXT      NOT NULL,
      Y2          TEXT      NOT NULL,
PRIMARY KEY (Y1));
```

Qual transformação da relação Rel irá preservar a semântica do diagrama E-R apresentado?

A

```
CREATE TABLE REL (
X1      TEXT      NOT NULL,
Y1      TEXT      NOT NULL,
R1      TEXT      NOT NULL,
R2      TEXT      NOT NULL,
PRIMARY KEY (X1, Y1),
FOREIGN KEY (X1)
      REFERENCES TX (X1),
FOREIGN KEY (Y1)
      REFERENCES EY (Y1));
```

B

```
CREATE TABLE REL (
X1      TEXT      NOT NULL,
Y1      TEXT      NOT NULL,
R1      TEXT      NOT NULL,
R2      TEXT      NOT NULL,
PRIMARY KEY (X1, R1),
FOREIGN KEY (X1)
      REFERENCES TX (X1),
FOREIGN KEY (Y1)
      REFERENCES EY (Y1));
```

C

```
CREATE TABLE REL (
X1      TEXT      NOT NULL,
Y1      TEXT      NOT NULL,
R1      TEXT      NOT NULL,
R2      TEXT      NOT NULL,
PRIMARY KEY (Y1, R1),
FOREIGN KEY (X1)
      REFERENCES TX (X1),
FOREIGN KEY (Y1)
      REFERENCES EY (Y1));
```

✓

```
CREATE TABLE REL (
X1      TEXT      NOT NULL,
Y1      TEXT      NOT NULL,
R1      TEXT      NOT NULL,
R2      TEXT      NOT NULL,
PRIMARY KEY (X1, Y1, R1),
FOREIGN KEY (X1)
      REFERENCES TX (X1),
FOREIGN KEY (Y1)
      REFERENCES EY (Y1));
```

DDL - DROP

Luiz é o DBA do TJRN e atendeu um chamado da equipe de desenvolvimento que pedia para criar um banco de dados com 5 Gigabytes, mas, na hora de criar, Luiz digitou 50 Gigabytes.

Para **apagar o banco de dados** criado equivocadamente, Luiz deve utilizar o comando:



Exemplo – tipos de dados

Ana está desenvolvendo um programa em PL/SQL para capturar e armazenar um histórico de mensagens que contém **3 GB de tamanho**.
Para manipular essa informação em uma variável, no Oracle PL/SQL, Ana deve declarar uma **variável do tipo**:

A) CHAR;

b) CLOB (certo) ;

C) NCHAR;

D) VARCHAR2;

E) NVARCHAR2.

MySQL Server support CLOB data type with column type of LONGTEXT that hold a character value of up to 4,294,967,295 or 4GB (2**32 - 1) characters.
https://www.herongyang.com/JDBC/MySQL-CLOB-Large-Object.html

LOB stands for **Large OBject**

Ano: 2023 Banca: FGV Órgão: Defensoria Pública do Estado do Rio Grande do Sul

DDL e metadados

Em um sistema de gerenciamento de banco de dados, qual conceito descreve informações sobre a estrutura de tabelas, como nomes de colunas, tipos de dados e restrições, sendo fundamental para a definição e organização dos dados?

A) Metadados. (gabarito)

B) Transações.

C) Tuplas.

D) Índices.

E) Esquema.

SQL – DDL – Chave estrangeira

```
CREATE TABLE CLIENTE
(
id INT primary key auto_increment,
nome VARCHAR(45) not NULL,
chave_localidade INT,
FOREIGN KEY (chave_localidade) REFERENCES localidades_clientes(chave)
);

insert into cliente (nome, chave_localidade) values
("Hospital dos bugs",1),
("Oficina do refactoring",1),
("Espaço da lógica",2),
("Zero ou Um Negócios",2),
("Sistema órfão", NULL),
("Big Data - Inteligência Digital",4)
;
```

SQL – DDL – Chave estrangeira

Em 1970, Codd propôs o modelo de dados relacional. Na época, os bancos de dados usavam modelos hierárquicos e de rede.

O modelo relacional revolucionou a área de banco de dados deixando os dois outros modelos praticamente esquecidos.

Como o nome do modelo já diz, um banco de dados relacional é uma coleção de relações com nomes distintos entre entidades. Um banco de dados é considerado bom quando o SGBD garante que as informações sendo armazenadas estão corretas e, para isso, é utilizado o conceito de **restrições de integridade**.

Assim, para garantir a integridade dos dados, SGBDs como o Microsoft SQL Server implementam o conceito de chaves primária e estrangeiras.

Em relação a chaves primárias e estrangeiras no Microsoft SQL Server, assinale V para a afirmativa verdadeira e F para afirmativa falsa.
( ) Se, ao criar uma chave primária em uma tabela, não for especificado NOCLUSTERED ou CLUSTERED e se essa tabela não possuir um índice CLUSTERED, por definição a chave primária é criada junto a um índice CLUSTERED.
( ) Os dados das colunas usadas para compor a chave primária devem possui valores exclusivos, de modo que **o valor NULL só pode aparecer uma vez**.
( ) Uma chave estrangeira de uma tabela pode fazer referência a uma coluna de outra tabela de outro banco de dados.
( ) Uma chave estrangeira de uma tabela não precisa estar vinculado a uma chave primária de outra tabela, desde que a coluna referenciada seja UNIQUE.

As afirmativas são, respectivamente,

(A) F, F, V e V.
(B) V, F, V e F.
(C) V, F, F e V.
(D) F, V, V e F.
(E) V, F, F e F.

SQL: DDL – Alter table

```
ALTER TABLE <table_name> ADD COLUMN <column_name> <column_datatype>
```

```
ALTER TABLE city ADD COLUMN create_date DATETIME
```

Com chave estrangeira:

```
alter table payment add column company_id INT;
alter table payment ADD FOREIGN KEY (company_id) REFERENCES companies(company_id);
```

Com múltiplas colunas:

```
alter table contract add column contract_start_date DATE, add column contract_end_date DATE;
```

SQL: DML - Insert

```
insert into localidades_clientes
(nome_localidade, cep, endereco)
values
("Florianópolis","0000-000","");

insert into localidades_clientes
(nome_localidade, cep, endereco)
values
("Itajaí","0000-000",""),("Balneário Camburiu","0000-000","");
```

SQL – DML - SELECT

```
SELECT * FROM
espaco_virtual.localidades_clientes;

SELECT chave, nome_localidade FROM
espaco_virtual.localidades_clientes;

SELECT nome_localidade FROM
localidades_clientes where chave = 1;

SELECT nome_localidade, cep FROM
localidades_clientes where chave > 0 and
nome_localidade != ""
```

Exemplo SQL - SELECT

Assinale a alternativa que corresponde, na linguagem SQL, a uma **sintaxe correta** de uma instrução SELECT.

A) SELECT <nome(s)_colunas>WHERE <condição>GROUP BY <nome(s)_colunas>FROM <nome_tabela>

B) SELECT <nome(s)_colunas>FROM <nome_tabela>GROUP BY <nome(s)_colunas>ORDER BY <nome(s)_colunas>WHERE <condição>

✓ SELECT <nome(s)_colunas>FROM <nome_tabela>WHERE <condição>GROUP BY <nome(s)_colunas>ORDER BY <nome(s)_colunas>

D) SELECT <nome(s)_colunas>ORDER BY <nome(s)_colunas>FROM <nome_tabela>WHERE <condição>

E) SELECT <nome(s)_colunas>WHERE <condição>FROM <nome_tabela>ORDER BY <nome(s)_colunas>GROUP BY <nome(s)_colunas>

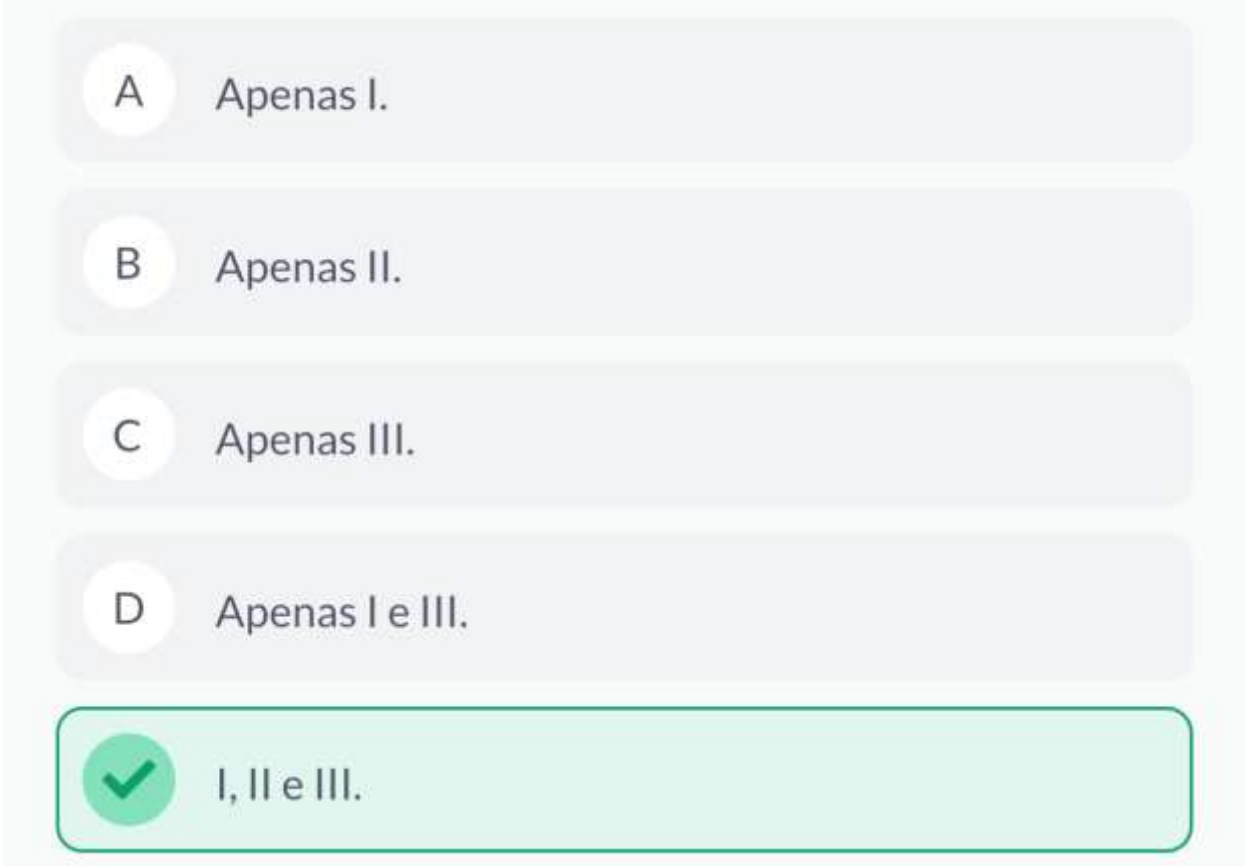
Compreendendo o operador LIKE

Considere a consulta abaixo escrita na linguagem SQL:

1. SELECT nome_pessoa as NOME
2. FROM PESSOA_FISICA
3. WHERE nome_pessoa **LIKE** 'J%';

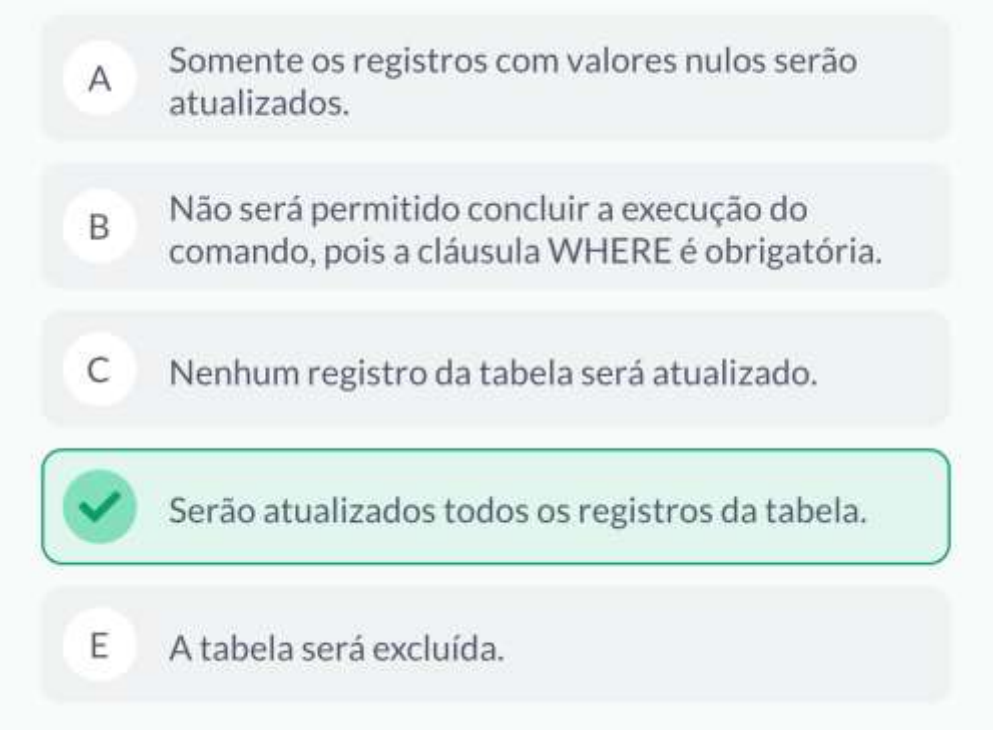
Analise as assertivas abaixo referentes ao caracter especial % na linha 3 da consulta SQL apresentada acima:

- I. No final do padrão pesquisado, indica que os registros do resultado deverão começar com o padrão do elemento indicado, conforme a linha 3 desse exemplo.
- II. No início do padrão pesquisado, por exemplo, alterando a linha 3 para **WHERE nome_pessoa LIKE '%J'**;, indica que os registros do resultado deverão terminar com o padrão indicado.
- III. No início e no final do padrão pesquisado, por exemplo, alterando a linha 3 para **WHERE nome_pessoa LIKE '%J%'**;, indica que o padrão deverá constar na coluna.



SQL – UPDATE - Importância do "where"

Na linguagem SQL (Structured Query Language), ao ser executado um comando UPDATE, qual será o resultado se for omitida a cláusula WHERE?



Filtro de dados

Clausula WHERE, conectores e operadores lógicos

- And
- Or
- =, !=, <, >
- Like (%%)

SQL – DML - SELECT

```sql
SELECT * FROM espaco_virtual.localidades_clientes
-- where nome_localidade = "São José"
-- where nome_localidade != "São José"
-- where chave > 2
-- where nome_localidade like 'Ita%'
-- where nome_localidade like '%ça'
-- where nome_localidade like '%o%'
-- where chave > 2 and nome_localidade like 'Itajai'
where chave > 3 or nome_localidade like 'São%'
```

SQL

CREATE e ALTER são exemplos de comandos de qual grupo de comandos da linguagem SQL?

A. DQL
B. DCL
C. DDL
D. DBL
E. DLL

SQL

Baseando-se em conhecimentos acerca de SQL. Qual das alternativas a seguir NÃO é um comando DML?

A. SELECT
B. UPDATE
C. DELETE
D. DROP
E. INSERT

SQL -DDL

- Determinado analista de TI recebeu uma solicitação: criar um código em SQL com o objetivo de implementar, em um banco de dados relacional, uma tabela com o nome de SIGA, contendo os seguintes campos: cod_siga, descricao, data_acesso e id_pessoa. Considerando a situação hipotética, assinale o código SQL que deverá ser apresentado pelo referido analista.

A
```
create table siga
(
cod_siga,
descricao,
data_acesso,
id_pessoa,
primary key (cod_siga)
);
```

B
```
insert table siga
(
cod_siga,
descricao,
data_acesso,
id_pessoa,
primary key (cod_siga)
);
```

C
```
create table siga
(
cod_siga:
descricao:
data_acesso:
id_pessoa:
primary key (cod_siga)
);
```

D
```
create table siga
(
cod_siga,
descricao,
data_acesso,
id_pessoa,
foreign key (cod_siga)
);
```

UNION VS UNION ALL

```
SELECT 1 UNION SELECT 1;
SELECT 1 UNION ALL SELECT 1;
```

O tratamento para linhas duplicadas tem comportamento diferenciado.

Por que UNION demanda mais processamento do que o UNION ALL?

```
SELECT * FROM sakila.customer;
SELECT first_name FROM sakila.customer;
SELECT first_name FROM sakila.customer;
SELECT first_name FROM sakila.customer union  SELECT first_name FROM sakila.customer;
SELECT first_name FROM sakila.customer union all  SELECT first_name FROM sakila.customer;
```

✓	10	11:53:17	SELECT first_name FROM sakila.customer union SELECT first_name FROM sakila.customer	591 row(s) returned
✓	11	11:53:22	SELECT first_name FROM sakila.customer union all SELECT first_name FROM sakila.customer	1198 row(s) returned

“EXCEPT” – NOT IN

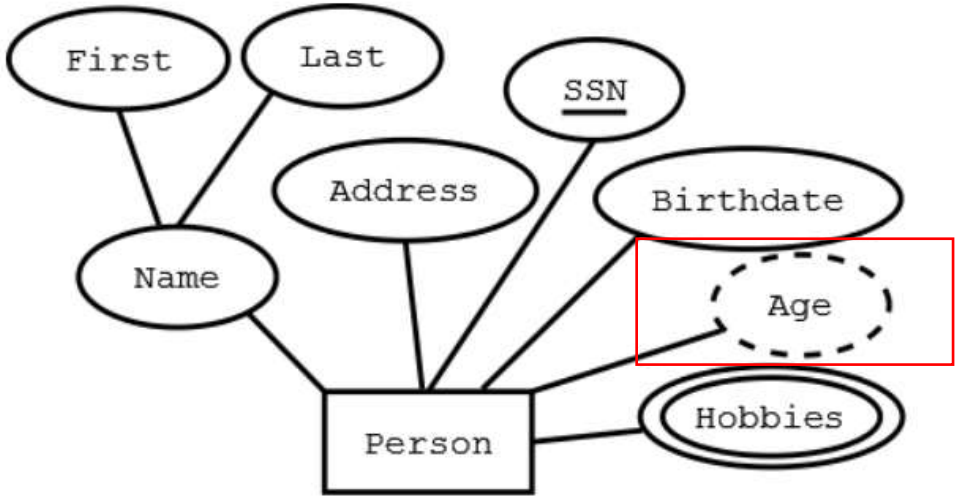
- SELECT first_name FROM sakila.actor where first_name NOT IN (SELECT first_name FROM sakila.customer);

SQL CASE Expression

```
SELECT film_id, title, rental_rate,
    CASE
    WHEN rental_rate < 1 THEN "low"
    WHEN rental_rate < 4 THEN "standard"
    WHEN rental_rate > 4 THEN "high"
    ELSE "-"
    END AS rental_level,
rental_duration
FROM sakila.film;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

film_id	title	rental_rate	rental_level	rental_duration
1	ACADEMY DINOSAUR	0.99	low	6
2	ACE GOLDFINGER	4.99	high	3
3	ADAPTATION HOLES	2.99	standard	7
4	AFFAIR PREJUDICE	2.99	standard	5
5	AFRICAN EGG	2.99	standard	6
6	AGENT TRUMAN	2.99	standard	3
7	AIRPLANE SIERRA	4.99	high	6
8	AIRPORT POLLOCK	4.99	high	6
9	ALABAMA DEVIL	2.99	standard	3
10	ALADDIN CALENDAR	4.99	high	6
11	ALAMO VIDEOTAPE	0.99	low	6
12	ALASKA PHANTOM	0.99	low	6



SQL CASE

A linguagem SQL oferece uma variedade de comandos e funcionalidades para manipulação e consulta de dados em bancos de dados relacionais. Com base nas práticas recomendadas e comandos do padrão ANSI SQL, analise as assertivas abaixo:

- I. A cláusula JOIN é utilizada para combinar registros de duas ou mais tabelas com base em uma condição comum entre elas.
- II. A cláusula GROUP BY deve ser sempre acompanhada da cláusula HAVING, pois esta última é responsável por agrupar os dados.
- III. O comando CASE só pode ser utilizado em instruções SQL do tipo SELECT, **não** sendo permitido em outras operações como UPDATE ou DELETE.

Quais estão corretas?

- **A. Apenas I (gabarito)**
- B. Apenas III
- C) Apenas I e II.
- D) Apenas II e III.
- E) I, II e III.

```
update produto2 set valor =
case
 when categoria = 'A' then valor * (valor * 5 / 100)
 when categoria = 'B' then valor * (valor * 10 / 100)
 when categoria = 'C' then valor * (valor * 15 / 100)
end;
```

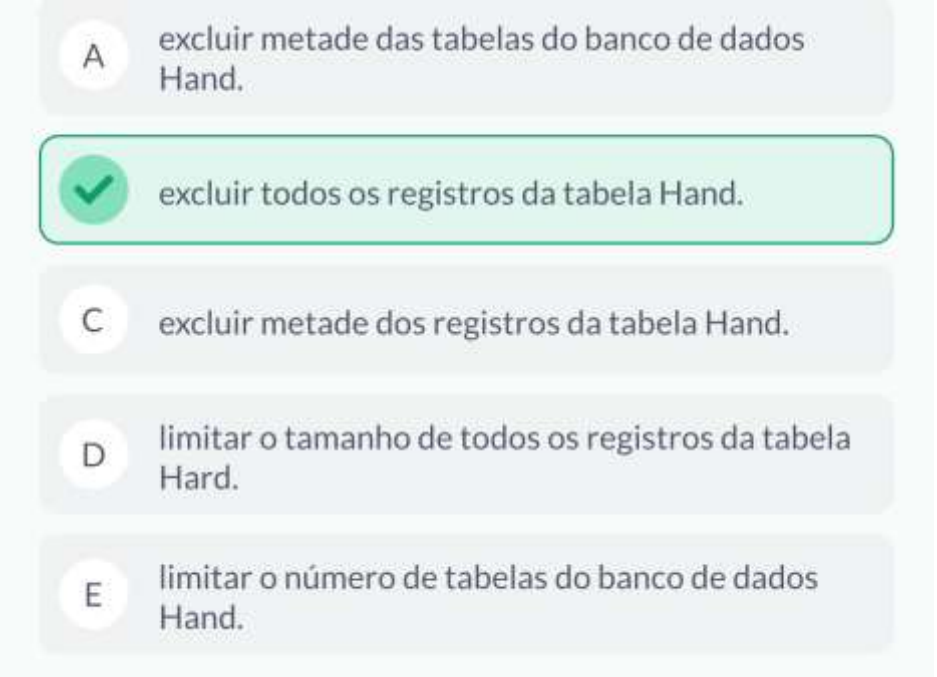
drop table vs truncate vs delete all

- DROP TABLE <table>: Remove a tabela (estrutura) e todos os seus dados. (DDL)
- TRUNCATE <table>: remove todos os dados de uma tabela. (DDL)
- DELETE FROM <table> where: remove apenas os dados que atendam o filtro (WHERE) (DML)

Truncate differs from DELETE in the following ways: Truncate operations drop and re-create the table, which is much faster than deleting rows one by one, particularly for large tables.

Exemplo

- Considere o seguinte comando do sistema gerenciador de bancos de dados Microsoft SQL Server:TRUNCATE TABLE Hand;
- O resultado da execução desse comando é



Exemplo:

Seja a tabela TBL1 a seguir, pertencente ao banco de dados BDD1:

MATRÍCULA	NOME	SALÁRIO
X100A	José	3612
X100B	Maria	4129
X100C	Fernanda	3000
X100D	Carlos	2345

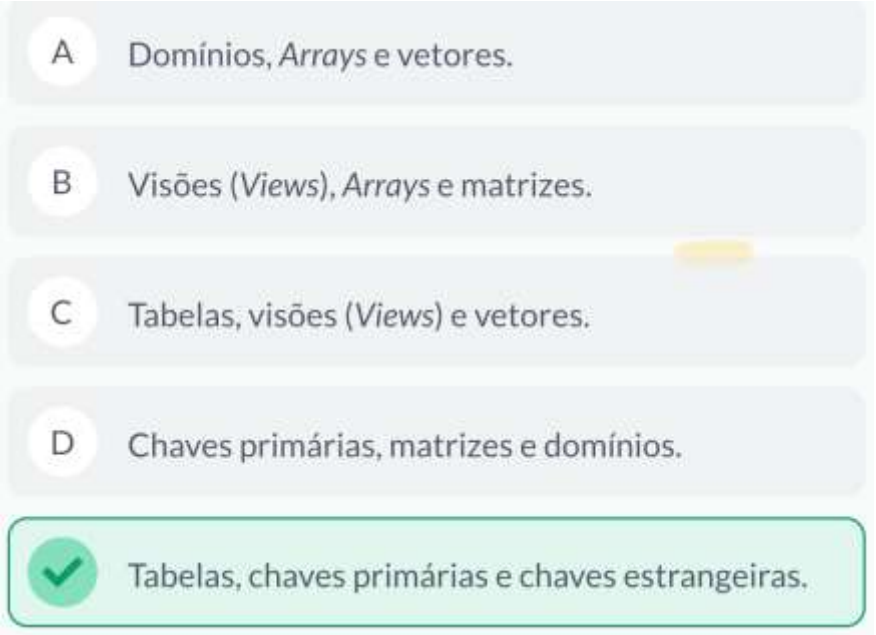
A execução da consulta SQL:

```
SELECT NOME FROM TBL1 WHERE (SALARIO > 2000) AND (SALARIO < 3500)
```

Vai gerar como resposta a tabela:

Exemplo:

- A implementação de Banco de Dados requer a criação de diversos objetos, seja pelo Administrador do Banco de Dados (DBA) ou de um outro usuário que detenha privilégios para realizar tal procedimento. São exemplos de objetos criados por ocasião da implementação do banco de dados:



Exemplo:

- Analise as afirmativas abaixo:
- 1. Uma restrição de integridade de entidade é aquela que postula que nenhuma chave primária pode ser nula em uma relação.
- 2. Uma restrição de integridade referencial é aquela que é especificada entre duas relações, necessariamente, de modo a manter a consistência entre os atributos nas duas relações.
- 3. Todas as restrições de integridade devem ser especificadas no esquema relacional do banco de dados, como parte da sua definição, via DDL.

Exemplo:

- No contexto das restrições em bancos de dados relacionais, no âmbito do modelo de dados relacionais, identifique restrições baseadas em esquema (schema-based) válidas:
- 1. Restrições de chave.
- 2. Restrições de domínio.
- 3. Restrições sobre valores nulos.
- 4. Restrições de gatilhos.

A São corretas apenas as afirmativas 1, 2 e 3.

B São corretas apenas as afirmativas 1, 2 e 4.

C São corretas apenas as afirmativas 1, 3 e 4

D São corretas apenas as afirmativas 2, 3 e 4.

E São corretas as afirmativas 1, 2, 3 e 4.

Exemplo:

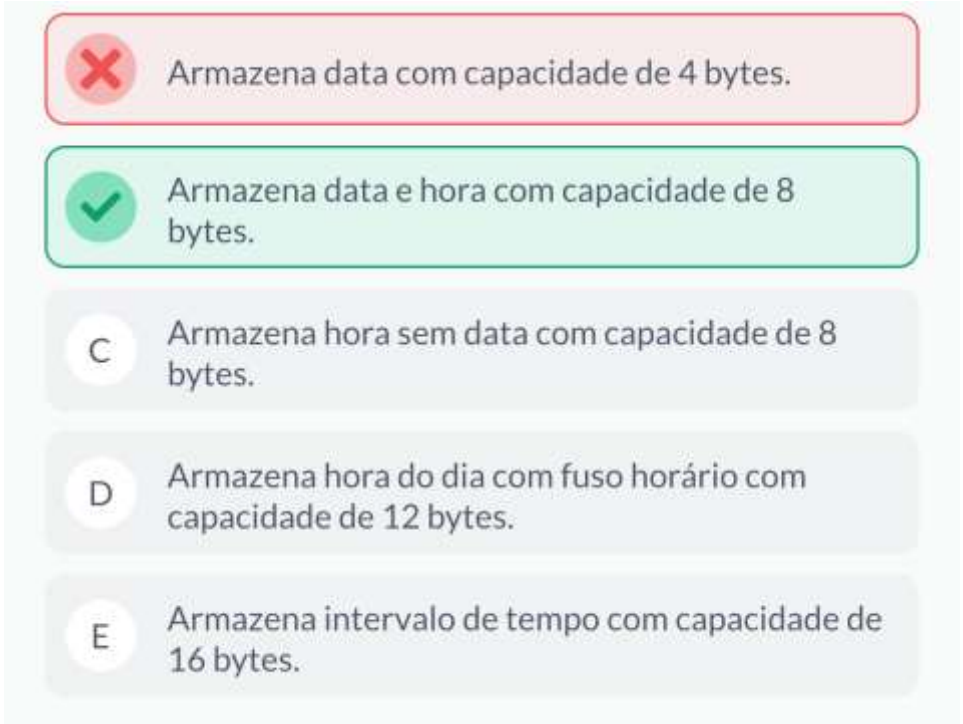
Analise as afirmativas abaixo sobre sistemas gerenciadores de bancos de dados relacionais , no que diz respeito ao modelo de dados relacional.

- 1. No modelo formal, um domínio é um conjunto de valores atômicos; isto é, cada valor no domínio é indivisível no que diz respeito ao modelo relacional formal.
- 2. O grau de uma relação é o número de atributos n do seu esquema de relação.
- 3. Um esquema de relação R, denotado por R(A1,A2,...,An), é composto por uma relação R e uma lista de atributos A1,A2,...,An; cada qual pertencente a um domínio D.

Assinale a alternativa que indica todas as afirmativas corretas.

Exemplo:

- O sistema de gerenciamento de banco de dados PostgreSQL fornece várias opções para armazenar datas, o tipo timestamp armazena:



Exemplo:

- Em um banco de dados Microsoft SQL Server 2016, existe uma tabela denominada “estoque”, que possui, entre outros, um campo de nome “preco” que armazena o preço de venda de vários produtos. Os preços dos produtos são R$ 270,00, R$ 280,00, R$ 295,00 e R$ 380,00. A tabela deve ser atualizada de forma que, no campo “preco”, nenhum produto deverá possuir preço menor que R$ 295,00, e não pode haver redução de preço nos produtos. Assinale a alternativa que possui o comando correto para executar essa atualização.



Exemplo:

- Observe os comandos SQL abaixo:

```
❶ CREATE TABLE teachers (
    ❷ id bigserial,
    ❸ first_name varchar(25),
      last_name varchar(50),
      school varchar(50),
    ❹ hire_date date,
    ❺ salary numeric
❻ );
```

A) na linha 1 está sendo criada a tabela "teachers".

B) na linha 2 o campo "id" é do tipo de dados "bigserial", um tipo inteiro especial que decrementa automaticamente toda vez que você adiciona uma linha à tabela.

C) na linha 3, está assumindo que ninguém no banco de dados terá um campo "first_name" menor do que 25 caracteres obrigatoriamente.

D) na linha 4, o campo "hire_date", está definido para a data do tipo de dados no formato "mês/ano".

E) na linha 5, o campo "salary" é do tipo numérico inteiro.

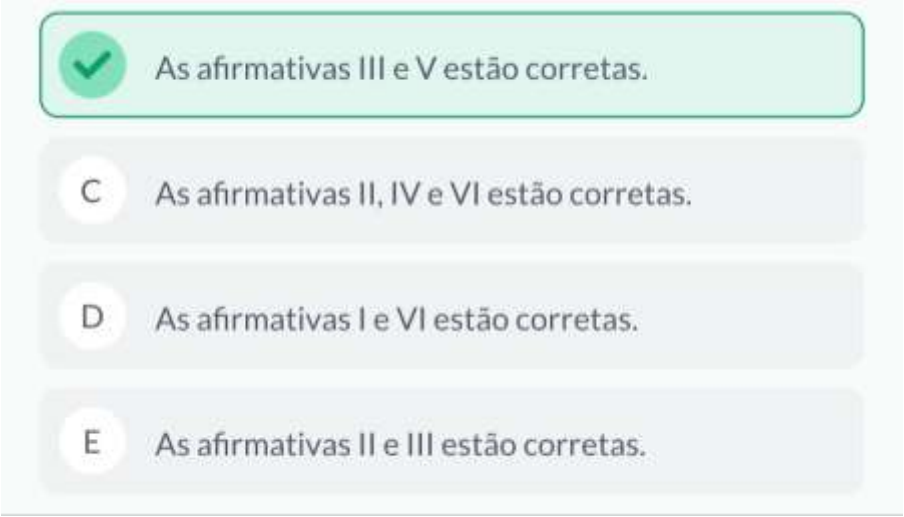
resposta A

Exemplo DDL

Considere o seguinte código SQL, do contexto dos bancos de dados relacionais, e as afirmativas a seguir a seu respeito. Em seguida, assinale a alternativa que contém apenas afirmações corretas.

- 1. CREATE DATABASE id01;
- 2. USE id01;
- 3. CREATE TABLE id02 (
- 4. id03 VARCHAR(255),
- 5. id04 VARCHAR(255),
- 6. id05 VARCHAR(255)
- 7.);
- 8. ALTER TABLE id06 ADD COLUMN id07 VARCHAR(20);

I. Na linha 1, é criado um novo banco de dados.
II. Na linha 1, é criada uma nova relação no banco de dados id01.
III. Na linha 2, é selecionada uma tabela já existente do banco de dados.
IV. Na linha 2, é selecionada ou criada (se ainda não existir) a tabela id01 do banco de dados.
V. O comando que se estende da linha 3 até a linha 7 cria uma relação id02 no banco de dados id01 e inclui na relação criada três atributos, id03, id04 e id05.
VI. O comando da linha 8 cria uma relação id06 no banco de dados id01 e inclui na relação criada o atributo id07.



Exemplo:

- insert into ESPECIALIDADES values (1,'cardiologia’);
- insert into ESPECIALIDADES values (2,'oftalmologia’);
- insert into ESPECIALIDADES values (3,'pediatria’);
- insert into MEDICOS values (1, 'joao', 1, 'ufrgs’);
- insert into MEDICOS values (2, 'maria', 1, 'pucrs’);
- insert into MEDICOS values (3, 'pedro', 2, 'ufsm');

Considere a sequência de comandos SQL abaixo, em que cada comando deve ser considerado uma transação separada:

- I. delete from ESPECIALIDADES where nomee = 'pediatria’;
- II. update ESPECIALIDADES set code = 4 where nomee = 'oftalmologia’;
- III. delete from ESPECIALIDADES where nomee = 'cardiologia';

Após a execução das transações I, II e III, é possível afirmar que:

- A tabela ESPECIALIDADES tem _____ tupla(s);
- Na tabela MEDICOS, no registro em que MEDICOS.codm = 1, o valor do atributo MEDICOS.code é _____;
- Na tabela MEDICOS, no registro em que MEDICOS.codm = 3, o valor do atributo MEDICOS.code é _____.

Assinale a alternativa que preenche, correta e respectivamente, as lacunas do trecho acima.

create table ESPECIALIDADES (code int not null primary key, nomee varchar(50) not null);	create table MEDICOS (codm int not null primary key, nomem varchar(50) not null, code int, formacao varchar(100) not null, foreign key(code) references ESPECIALIDADES (code) on delete set null);

A ~~1 – 1 – 2~~

B ~~2 – NULL – 4~~

C 1 – NULL – 4

D ~~2 – 1 – 2~~

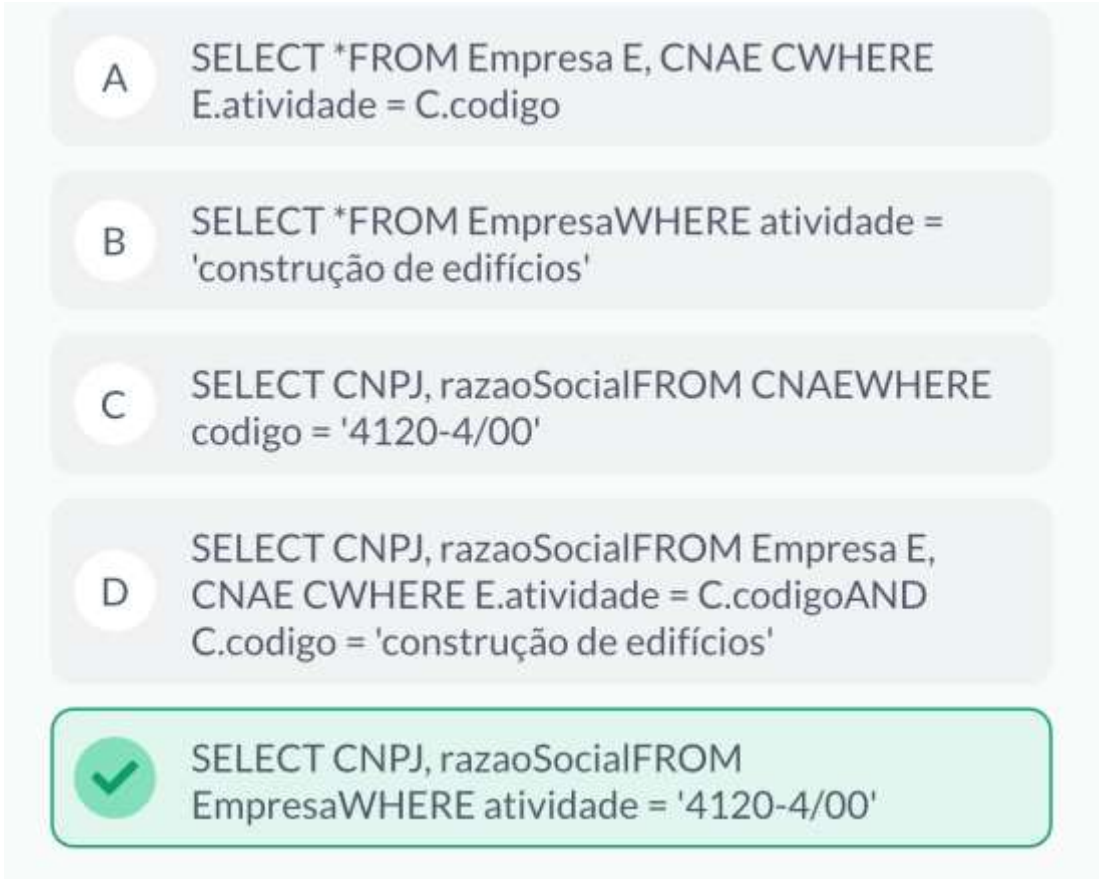
✓ 1 – NULL – 2

Exemplo:

- A CNAE (Classificação Nacional de Atividades Econômicas), de responsabilidade do IBGE, possui códigos que são utilizados para caracterizar as atividades econômicas das empresas no Brasil. Por exemplo: empresas da área de construção de edifícios utilizam o CNAE de código 4120-4/00 para caracterizar a sua atividade econômica principal. Considere que existe um banco de dados em uma empresa, que concede empréstimos a outras empresas, com as seguintes tabelas:

- Empresa (CNPJ, razaoSocial, endereco, atividade)CNAE (codigo, descricao)

- A coluna "atividade" da tabela Empresa é uma chave estrangeira que referencia a coluna "codigo" da tabela CNAE. Que comando SELECT do SQL retorna apenas o CNPJ e a razão social das empresas cuja atividade econômica principal é a construção de edifícios (código 4120-4/00)?



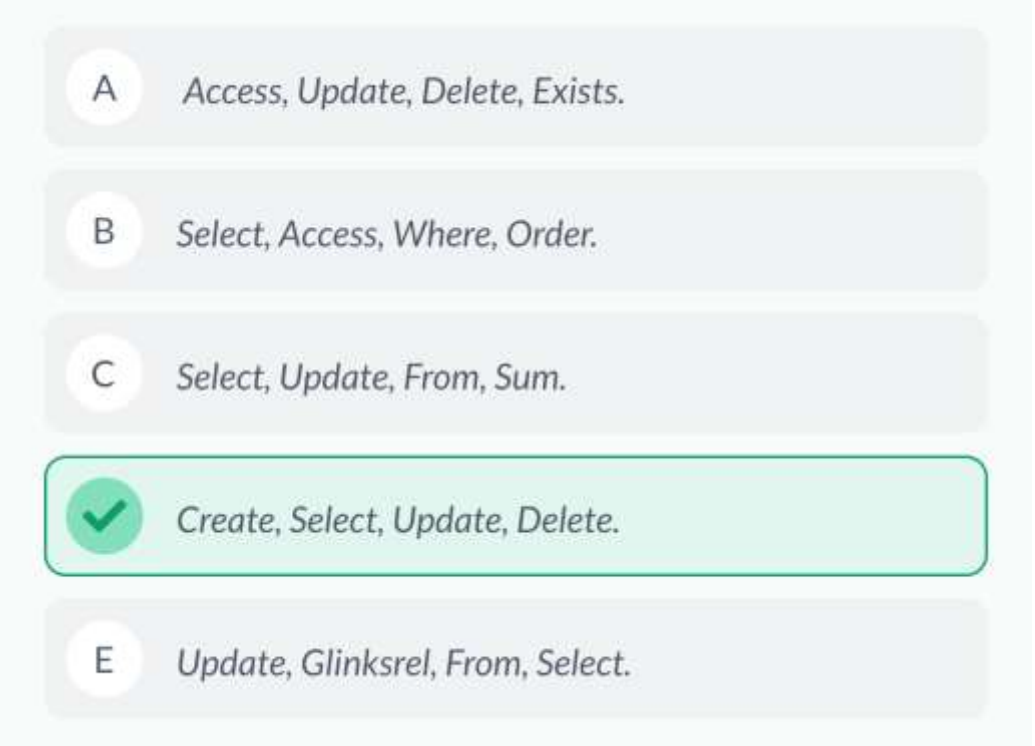
Exemplo

- Um técnico quer utilizar uma query contendo um comando SELECT com uma cláusula WHERE para uma recuperação de registros. Na SQL esse processo é realizado por meio da:



Exemplo

- O termo CRUD descreve as quatro operações básicas para gerenciar elementos de dados persistentes, especialmente em bancos de dados relacionais e na Structured Query Language (SQL) para a manipulação de dados. Sobre os comandos em questão, qual das alternativas a seguir está CORRETA?



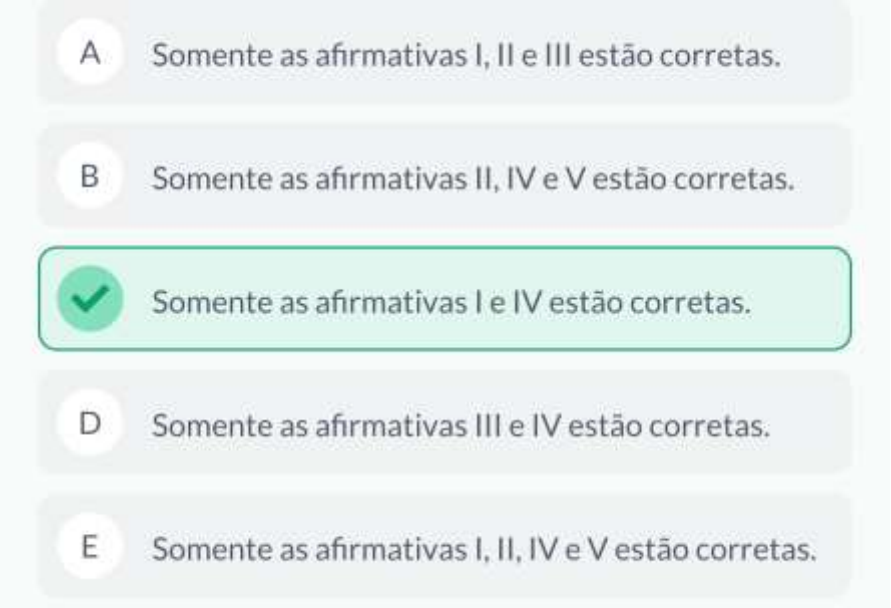
Exemplo

- Considere a tabela abaixo, contida em um hipotético banco de dados relacionais, e as afirmações a seguir a seu respeito. Em seguida, assinale a alternativa correta.

Alunos

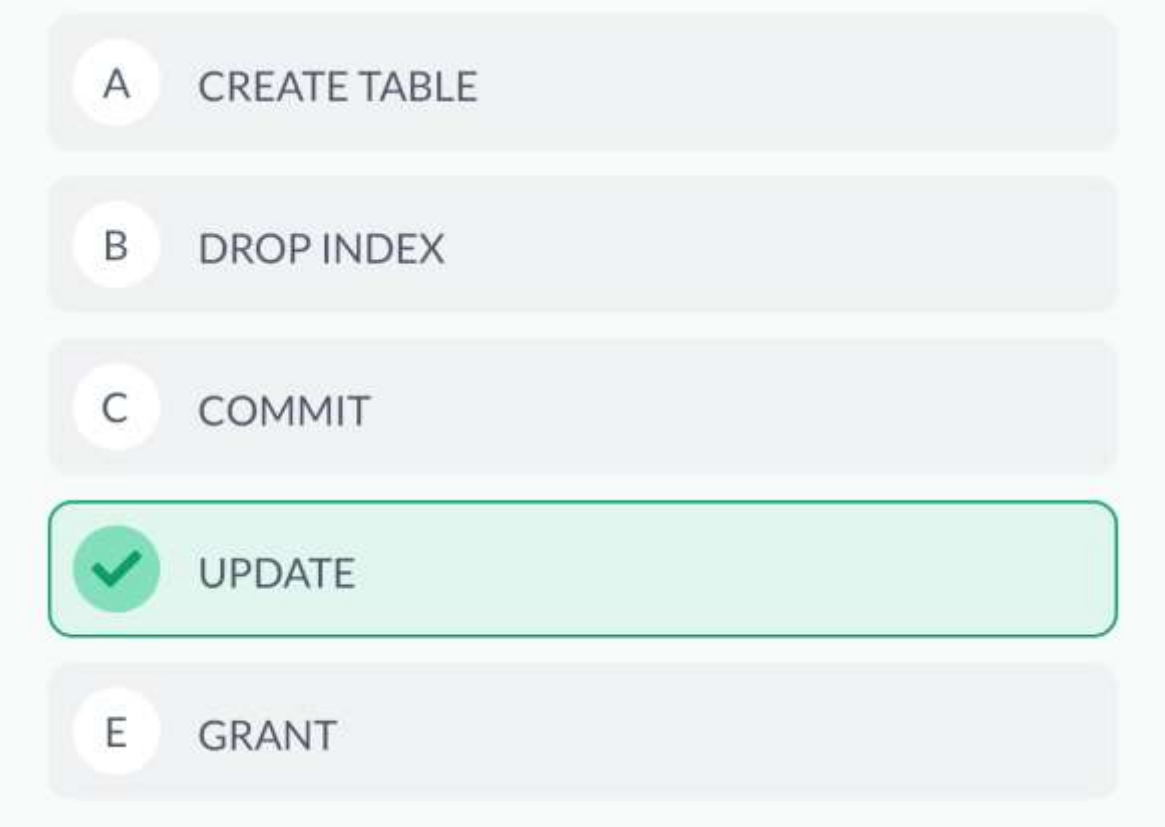
- nome nota indice
- Jose 5 1
- Manuel 8 2
- Francisco 10 4
- Alice 7 3

- I. O comando SELECT nome FROM Alunos WHERE nota > 7; retornaria os valores “Manuel” e “Francisco”.
- II. O comando SELECT nome FROM Alunos WHERE nota > 8 AND indice >= 2; retornaria os valores “Manuel” e “Francisco”.
- III. O comando SELECT indice FROM Alunos WHERE nota >= 7 AND indice <= 2; retornaria os valores “1” e “2”.
- IV. O comando SELECT nome FROM Alunos WHERE nota >= 8 AND indice >= 2; retornaria os valores “Manuel” e “Francisco”.
- V. O comando SELECT indice FROM Alunos WHERE nota <= 7 AND indice <= 2; retornaria os valores “1” e “2”.



Exemplo

- Qual das seguintes opções representa corretamente um comando DML (Data Manipulation Language) utilizado em SGBD relacionais?



Exemplo - Like

A programadora Fabiana elaborou um relatório com o nome do autor dos processos utilizando o seguinte comando SQL:

- SELECT Autor FROM tab_processo

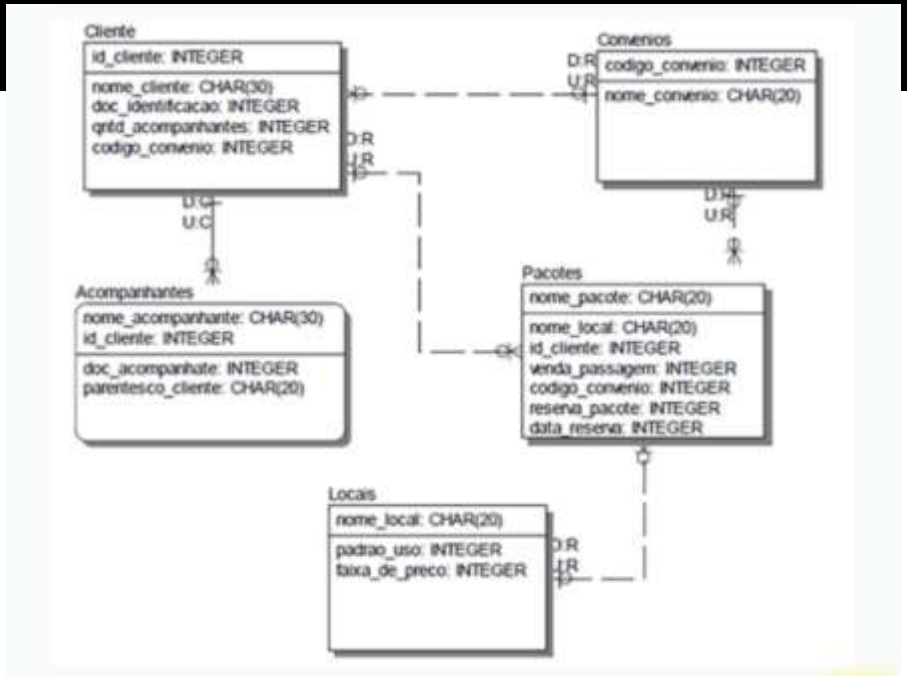
Ao ver o relatório gerado, o chefe da Fabiana solicitou um relatório contendo apenas os autores que possuem o primeiro nome Elizabeth. Contudo, Fabiana observou que havia diferentes grafias, como: Elisabeth, Elizabete etc. No MySQL, para garantir que qualquer Autor cujo nome comece pelas letras “eli” sejam recuperados, Fabiana deve complementar o comando SQL com a cláusula WHERE Autor LIKE:

- a)'eli%' (correto)
- B'%eli%'
- C'eli_'
- D'_eli'
- E'_eli%'

Ano: 2023 Banca: FGV Órgão: Defensoria Pública do Estado do Rio Grande do Sul

Exemplo

A linguagem SQL (Structured Query Language) possui um grupo de comandos denominados DDL (Data Definition Language). Analise o modelo a seguir:



Considerando que o modelo acima seja implementado na versão 3.0.10 do banco de dados Firebird SQL, um desenvolvedor faz as seguintes afirmações:

- I. O código SQL, para criar a tabela convenios, é: create table convenios(cod_convenio integer not null foreign key, nome_convenio varchar(20)).
- II. O código SQL, para adicionar uma chave estrangeira no campo id_cliente na tabela acompanhante em referência a tabela cliente, é: alter table acompanhante add primary key(id_cliente) references cliente(id_cliente).
- III. O código SQL, para inserir os valores “002” e “unimed” aos campos codigo_convenio e nome_convenio da tabela convenios, é: insert into convenios(codigo_convenio, nome_convenio) values (002,'unimed').

Assinale a alternativa CORRETA.

A Apenas a afirmação I está correta.

B As afirmações I e II estão corretas.

C Apenas a afirmação II está correta.

D As afirmações II e III estão corretas.

✓ Nenhuma das afirmações está correta.

Exemplo – MySQL CMD - Backup

- No contexto da administração de um Sistema de Gerenciador de Banco de Dados MySQL, analise o comando a seguir.

  mysqldump -u root -p exemploDB > saida.sql

- O efeito causado pela execução desse comando é:Assinale a alternativa CORRETA:
  - A) A criação de políticas de segurança do tipo “dump” na base de dados especificada.
  - **B) A criação de um backup da base de dados especificada.**
  - C) A criação de um relatório de avaliação do desempenho da base de dados especificada.
  - D) A criação de um relatório de transações que falharam na base de dados especificada.
  - E) A criação de uma política de duplicação do tipo “dump” na base de dados especificada.

Exemplo Backup - PostgreSQL

Em relação a APIs, serviços de autenticação, webhook, PostgreSQL e microsserviços, julgue o item a seguir.

O PostgreSQL possui a ferramenta PITR (point-in-time recovery), que permite restaurar o banco de dados para um momento específico no tempo.

Certo (gabarito)

Errado

Backup - Oracle

Julgue o próximo item, relativo à normalização de dados, à linguagem de definição de dados (DDL), a sistema gerenciador de banco de dados (SGBD) Oracle 21C e a data lake.

O Oracle 21C exige que todas as operações de backup sejam realizadas exclusivamente no Oracle Cloud, removendo a opção de backups locais.

- Certo
- **Errado (gabarito)**

Atividade

- Crie as tabelas definidas na aula sobre modelo ER e faça a inserção de ao menos 10 registros em cada tabela.
- OBS: criação das tabelas deve ser feita com instruções SQL (DDL).

Aula 4

Normalização: Formas Normais

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Tipos de Dependências entre Atributos

- Dependência Funcional
- Dependência Funcional Total
- Dependência Funcional Parcial
- Dependência Funcional Transitiva
- Dependência Multivalorada

Source: http://www.bosontreinamentos.com.br/modelagem-de-dados/modelagem-de-dados-dependencias/

Dependência Funcional

Seja E uma entidade, e X e Y dois atributos quaisquer de E. Dizemos que Y é funcionalmente dependente de X se e somente se cada valor de X tiver associado a ele exatamente um valor de Y.
Simbolicamente:

$$X \rightarrow Y$$

Que lemos como “X determina funcionalmente Y”.

Exemplo

Ex.: Pensando em uma tabela de pedidos; O prazo de entrega de um pedido depende do número do pedido considerado:

Dependência Funcional Total

Em uma relação com uma PK composta, um atributo não-chave que dependa dessa PK como um todo, e não somente de parte dela, é dito como possuindo Dependência Funcional Total.

Exemplo

Veja o exemplo a seguir:

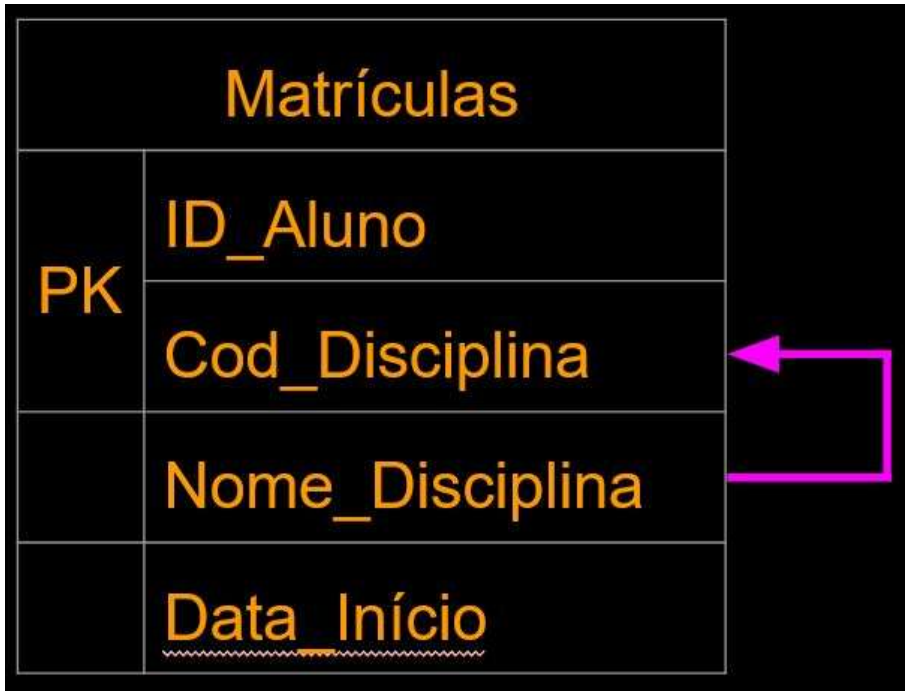
Aqui, Quant_produto depende tanto de Num_Pedido quanto de Cod_Produto, ao mesmo tempo.

Obs.: A dependência funcional total só pode ocorrer em tabela com chave primária composta



Dependência Funcional Parcial

Uma dependência funcional é parcial quando os atributos não-chave não dependem funcionalmente de toda a PK quando esta for composta.
Ou seja, existe uma dependência funcional, mas somente de uma parte da chave primária.
Exemplo
Campo Nome_Disciplina é dependente de Cod_Disciplina, mas não do ID_Aluno.



Dica: Segunda forma normal

Dependência Funcional Transitiva

Este tipo de dependência ocorre quando um campo não-chave não depende diretamente da chave primária da tabela (nem mesmo parcialmente), mas depende de um outro campo não-chave na tabela.

Exemplo



No exemplo, o atributo Nome_Vendedor depende funcionalmente do Cód_Vendedor, que não é **chave primária** na tabela. Já o campo Prazo_Entrega depende da PK, Num_Pedido

Dica: Terceira forma normal

Dependência Multivalorada

Ocorre quando, para cada valor de um atributo A, existe um conjunto de valores para outros atributos B e C que estão associados a ele, mas que são independentes entre si.
Representamos a dependência multivalorada assim:

A —>> B

Onde B é a coluna que depende da coluna A.

Exemplo
Neste exemplo, os atributos Ano e Cor são independentes entre si e dependem do modelo do carro. Essas duas colunas são *dependentes multivaloradas* do atributo Modelo.

Modelo	Ano	Cor
Gol	2016	Prata
Uno	2016	Preto
Uno	2015	Prata
Fox	2016	Vermelho
Fox	2014	Branco

Dica: Quarta forma normal

Formas normais

Normalização de banco de dados é um conjunto de regras que visa, principalmente, a organização de um projeto de banco de dados para reduzir a redundância de dados, aumentar a integridade de dados e o “desempenho”(?).

Formas normais

No contexto de bancos de dados, as técnicas de organização e validação dos dados, bem como as características de diferentes modelos de armazenamento, são essenciais para a eficiência e a integridade das informações. Isso posto, analise as afirmativas a seguir e registre V, para verdadeiras, e F, para falsas:

(__) A normalização de dados tem como principal objetivo converter dados não estruturados em dados estruturados, organizando-os em tabelas.

(__)Dados semiestruturados podem ser validados por meio de esquemas, como DTD (Document Type Definition) para XML.

(__) Todos os bancos de dados NoSQL são projetados para armazenar exclusivamente dados não estruturados.

Assinale a alternativa que apresenta a sequência correta:

A) F – F – V.

B) F – V – F. (gabarito)

C) V – F – V.

D) F – V – V.

E) V – V – F.

Normalização é a reorganização dos dados com o intuito de **reduzir as redundâncias**.

Primeira Forma Normal (1FN)

- Nesta forma os atributos precisam ser **atômicos**, o que significa que as tabelas não podem ter valores repetidos e nem atributos possuindo mais de um valor. Exemplo: CLIENTE = {ID + ENDEREÇO + **TELEFONES**}. Porém, uma pessoa poderá ter mais de um número de telefone, sendo assim o atributo "TELEFONES" é multivalorado. Para normalizar, é necessário:
  - Identificar a chave primária e também a coluna que possui dados repetidos (nesse exemplo "TELEFONES") e removê-los;
  - Construir uma outra tabela com o atributo em questão, no caso "TELEFONES". Mas não se esquecendo de fazer uma relação entre as duas tabelas: CLIENTE = {ID + ENDEREÇO} e TELEFONE (nova tabela) = {CLIENTE_ID (chave estrangeira) + TELEFONE}.

Exemplo: 1FN

Durante o desenvolvimento de um sistema de gestão acadêmica em uma universidade, a equipe de Tecnologia da Informação está modelando o banco de dados para armazenar informações sobre os alunos e os cursos em que estão matriculados. As principais entidades identificadas são **Aluno, Curso e Matrícula**. O objetivo é garantir que o sistema permita consultas eficientes sobre os alunos e suas respectivas matrículas em cursos. Considerando o modelo Entidade-Relacionamento (MER) e a linguagem SQL, a maneira CORRETA de modelar a situação descrita e realizar consultas para obter os dados desejados é:

A) A entidade Aluno deve incluir um atributo que armazena diretamente a lista de cursos em que o aluno está matriculado, simplificando a consulta SQL.

B) A entidade Aluno deve ter uma chave estrangeira para Matrícula, para permitir a vinculação direta entre aluno e matrícula em consultas SQL.

C) A entidade Curso deve ser modelada como um atributo multivalorado na entidade Aluno para facilitar a vinculação dos dados na consulta SQL.

D) A entidade Matrícula deve conter as chaves estrangeiras que referenciam as entidades Aluno e Curso, permitindo consultas SQL com JOIN para vincular essas informações.(gabarito)

E) A entidade Matrícula deve ser modelada como uma entidade fraca, dependente de Aluno, e sua chave primária deve combinar atributos de Aluno e Curso.

Segunda Forma Normal (2FN)

Primeiramente, para estar na 2FN é preciso estar também na 1FN.

2FN define que os atributos normais, ou seja, os não chave, **devem depender unicamente da chave primária da tabela**. Assim como as colunas da tabela que não são dependentes dessa chave devem ser removidas da tabela principal e cria-se uma nova tabela utilizando esses dados. Exemplo:

PROFESSOR_CURSO = {ID_PROF + ID_CURSO + SALARIO + DESCRICAO_CURSO}.

Como podemos observar, o atributo "DESCRICAO_CURSO" não depende unicamente da chave primária ("ID_PROF", "ID_CURSO“) mas sim somente da chave "ID_CURSO". Para normalizar, é necessário:

- Identificar os dados não dependentes da chave primária (nesse exemplo "DESCRICAO_CURSO") e removê-los;
- Construir uma nova tabela com os dados em questão: PROFESSOR_CURSO = {ID_PROF + ID_CURSO + SALARIO} e CURSOS (nova tabela) = {ID_CURSO + DESCRICAO_CURSO}.

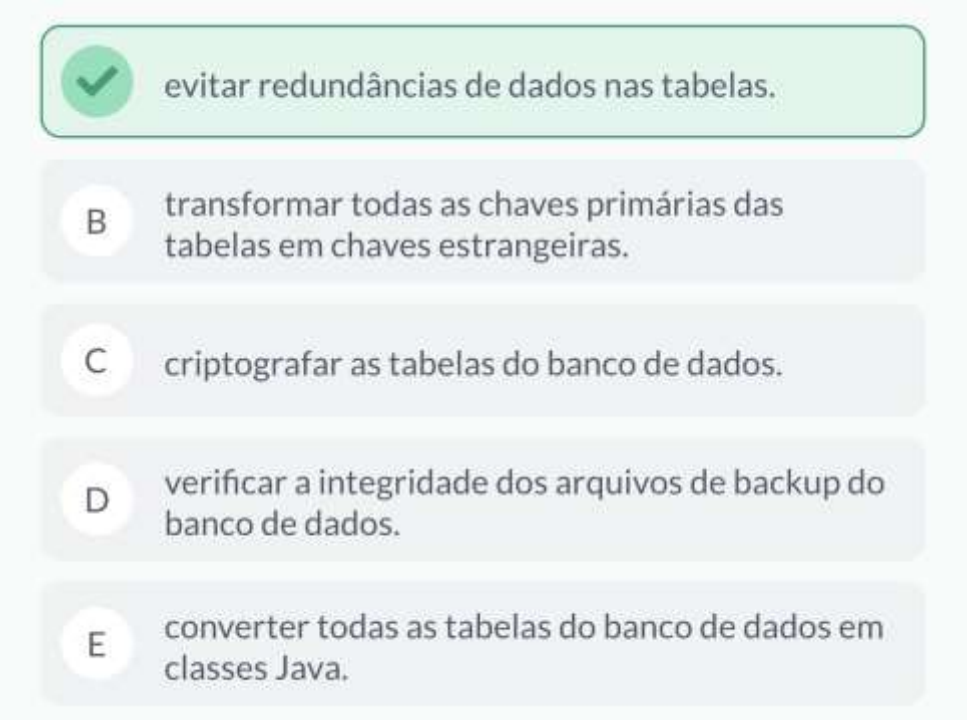
Terceira Forma Normal (3FN)

Assim como para estar na 2FN é preciso estar na 1FN, para estar na 3FN é preciso estar também na 2FN.

- 3FN define que todos os atributos dessa tabela devem ser funcionalmente independentes uns dos outros, ao mesmo tempo que devem ser dependentes exclusivamente da chave primária da tabela.
- 3FN foi projetada para melhorar o desempenho de processamento dos banco de dados e minimizar os custos de armazenamento.
- Exemplo: FUNCIONARIO = {ID + NOME + VALOR_SALARIO + VALOR_FGTS}.
- O valor do FGTS é proporcional ao salário, logo o atributo "VALOR_FGTS" é dependente do também atributo "VALOR_SALARIO". Para normalizar, é necessário:
  - Identificar os dados dependentes de outros (nesse exemplo "VALOR_FGTS");
  - Removê-los da tabela. Esses atributos poderiam ser definitivamente excluídos -- e deixando para a camada de negócio a responsabilidade pelo seu cálculo -- ou até ser movidos para uma nova tabela e referenciar a principal ("FUNCIONARIO").

Objetivos da normalização

- Uma das boas práticas recomendadas para um projeto lógico de um banco de dados relacional, é verificar o nível de normalização das tabelas do banco de dados. A função da normalização para um banco de dados, é

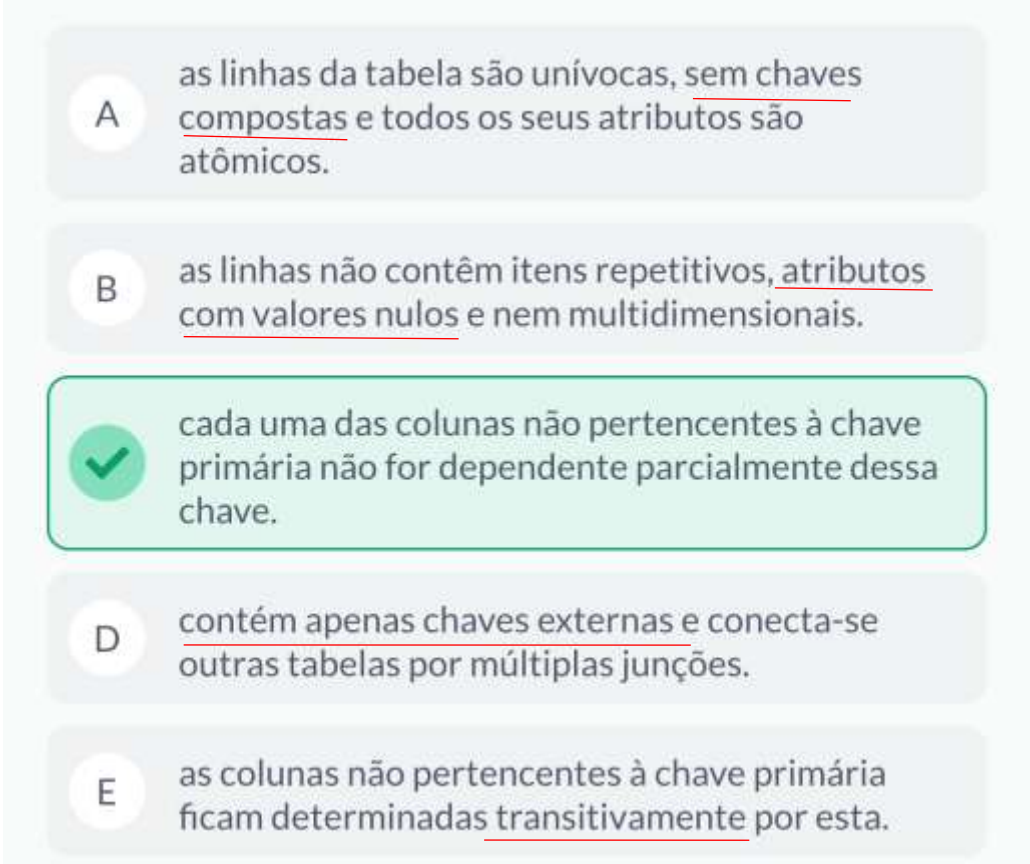


2FN - exercício

- A normalização das estruturas de dados é uma etapa importante do processo de modelagem relacional para eliminar distorções ou anomalias no modelo.

•

Diz-se que uma tabela está na segunda forma normal, de acordo com as regras da normalização, se ela está na primeira forma normal e ....



3FN - Exercício

Julgue o próximo item, relativo à normalização de dados, à linguagem de definição de dados (DDL), a sistema gerenciador de banco de dados (SGBD) Oracle 21C e a data lake.

Para que uma relação esteja na terceira forma normal (3FN), ela deve estar na segunda forma normal (2FN) e todos os seus atributos não chave devem depender diretamente da chave primária, sem dependências transitivas.

- **Certo (gabarito)**
- Errado

Atividade: formas normais

1. Normalize a tabela abaixo para a 3FN, onde idPedido e CodProduto formama chave da tabela:

Pedido

idPedido	dataPeddo	codProduto	nomeProduto	qtde	valorUnitario	valorTotal
1	01/07/09	1234	HD 250GB	2	R$ 100	R$ 200
2	01/07/09	1235	HD 180GB	1	R$ 80	R$ 80
3	03/07/09	1235	HD 180GB	4	R$ 80	R$ 320
4	05/07/09	1234	HD 250GB	6	R$ 100	R$ 600

Dependências funcionais:

idPedido → dataPedido

idPedido, codProduto → qtde, valorTotal

codProduto → nomeProduto, valorUnitário

3FN exercício

Em um banco de dados em que são armazenados dados previdenciários, a tabela contribuintes a seguir, que possui os seguintes atributos, não está na terceira forma normal, pois apresenta dependência funcional transitiva entre atributos.

- cpf (chave primária)
- nome
- data_nascimento
- cod_municipio
- aliquota_contribuicao
- renda_mensal
- contribuicao mensal (calculada como: renda mensal × aliquota contribuicao)

- **Certo (gabarito)**
- Errado

Atividade: formas normais

Normalize a tabela aluno descrita abaixo:

aluno (nro_aluno, cod_depto, nome_depto, sigla_depto, cod_orient, nome_orient, fone_orient, cod_curso)

As seguintes dependências funcionais devem ser garantidas na normalização:

- cod_depto → nome_depto, sigla_depto
- cod_orient → nome_orient, fone_orient
- nro_aluno → cod_depto, cod_orient, cod_curso

Observações adicionais:

- um aluno somente pode estar associado a um departamento
- um aluno cursa apenas um único curso
- um aluno somente pode ser orientado por um único orientador

Atividade: formas normais

Considere a tabela a seguir:- Verifique se a tabela está na 1º Forma Normal, caso não esteja normalize.

Código_cliente	Nome	Telefone	Endereço
C001	José	9563-6352 9847-2501	Rua Seis, 85 Morumbi 12536-965
C002	Maria	3265-8596	Rua Onze, 64 Moema 65985-963
C003	Janio	8545-8956 9598-6301	Praça ramos Liberdade 68858-633

Observações adicionais: O endereço e o telefone pode conter vários valores. É necessário que o endereço seja detalhado, ou seja, o endereço deve ter; Rua, número, bairro e CEP.

Atividade: formas normais

Observe a seguinte ficha de inscrição de um determinado aluno para o curso de informática:

ESCOLA XXX LTDA

MATRICULA: 00010 **NOMEALUNO**: MARIA DE OLIVEIRA

IDENTIDADE: 1-111.111 **CPF**: 222.222.222-22 **TITULODEELEITOR**: 3333 3333 3333

CELULAR: (99)9999-9999 **EMAIL**: MARIA@GMAIL.COM

CODIGOCURSO: 01 **NOMECURSO**: INFORMÁTICA **CARGAHORARIACURSO**: 80

CODIGOTURNO: 02 **NOMETURNO**: NOTURNO

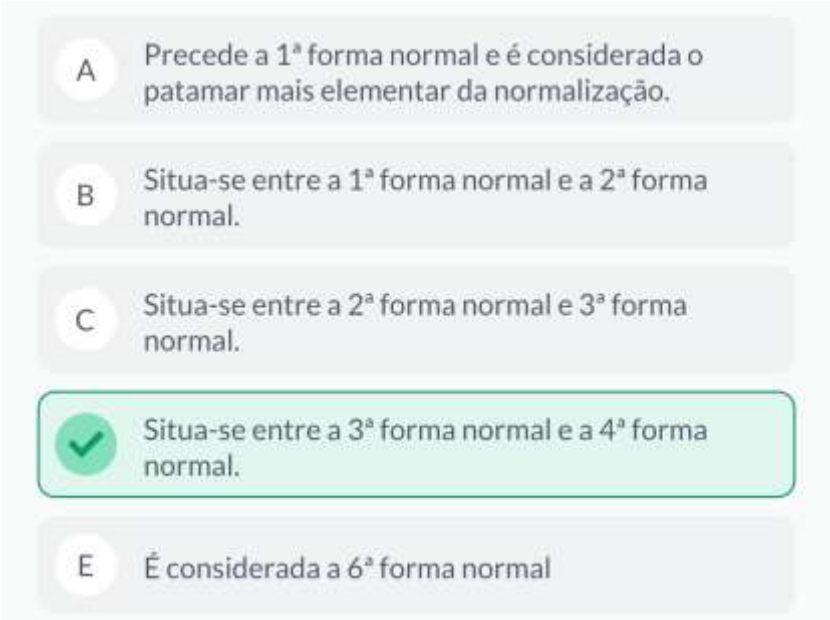
Alternativas:

A) Não normalizada
B) Primeira forma normal
C) Segunda forma normal
D) Terceira forma normal

A ficha representa uma tabela com todos os atributos (em negrito) e a chave desta tabela é MATRICULA. (Matricula, Nomealuno, Identidade, CPF, TitulodeEleitor, Celular, Email, CodigoCurso, NomeCurso, CargaHorariaCurso, CodigoTurno, NomeTurno). Podemos afirmar que tabela encontra-se em que forma normal?

Exemplo:

- Com relação às formas normais em bancos de dados relacionais, assinale a alternativa correta no que diz respeito à forma normal de Boyce/Codd, com relação à sua posição na hierarquia das formas normais.



Exemplo: dicionário de dados

O **dicionário de dados** é uma ferramenta fundamental na administração de bancos de dados, responsável por documentar e armazenar informações sobre os elementos de dados que compõem o sistema. Ele facilita a padronização, integração e controle dos dados, fornecendo uma visão clara sobre suas características, definições e restrições. Com base nisso, complete as lacunas a seguir.

O dicionário de dados contém informações sobre os atributos dos dados, como seu nome, tipo de dado e tamanho. Ele também armazena as ___(1), que são as regras e condições que garantem a integridade dos dados no sistema, e as ___(2), que identificam as ligações e dependências entre os diferentes dados no modelo de banco de dados.

A. (1) restrições de integridade / (2) relações de dependência. (gabarito)

B. (1) especificações de segurança / (2) entidades relacionais.

C. (1) chaves estrangeiras / (2) cardinalidades.

D. (1) definições de índices / (2) associações de integridade.

E. (1) chaves primárias / (2) referências cruzadas.

Parte II



Aula 7

Índices – Otimizando o desempenho de consultas em bancos de dados e DTL

Prof. Dr. Rafael Queiroz Gonçalves, PMP

INDEX

- Os índices (INDEX) são importantes estruturas, facultativas, que podem otimizar o tempo necessário para o banco de dados realizar determinada consulta.
- Isto pois índices armazenam os dados de determinada tabela em de forma pré agrupada e ordenada de acordo com um conjunto de campos de uma tabela.
- Deste modo quando uma consulta é realizada, antes de realizar o fullscan (busca sequencial) em uma tabela, é verificado se algum índice atende aos critérios de busca de determinada consulta.

INDEX

- Indexes are used to find rows with specific column values quickly.
- Without an index, MySQL must begin with the first row and then read through the entire table to find the relevant rows.
- The larger the table, the more this costs.
- If the table has an index for the columns in question, MySQL can quickly determine the position to seek to in the middle of the data file without having to look at all the data. This is much faster than reading every row sequentially.

MySQL :: MySQL 8.0 Reference Manual :: 8.3.1 How MySQL Uses Indexes

INDEX

- O tradeoff (efeito colateral) do uso de índices é que os mesmos aumentam o espaço em disco ocupado pelo banco de dados (os dados ficam armazenados em redundância) e as operações de escrita ficam mais lentas: quando atualiza os dados de uma tabela, todos os índices precisam ser atualizados também.

INDEX

- Porém o uso de índices costuma ser benéfico considerando que tipicamente **são realizadas mais operações de leitura** do que de escrita sob uma tabela.
- Uma boa estratégia é criar índices que atendam as **consultas tipicamente mais utilizadas** – e principalmente as consultas mais “caras”.

INDEX

```
CREATE INDEX id_index ON t1 (id)
```

```
CREATE INDEX part_of_name ON customer (name(10))
```

Se determinada consulta está utilizando índices ou realizando fullscan, você pode observar utilizando o comando explain:

```
Explain SELECT * FROM sakila.actor where last_update > '2006-01-01';
```



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	actor	NULL	ALL	NULL	NULL	NULL	NULL	201	3 Using where

EXPLAIN vs EXPLAIN ANALYZE

`EXPLAIN ANALYZE`:

For a more in-depth analysis, `EXPLAIN ANALYZE` executes the query and provides runtime statistics, including actual execution times and row counts for each operation, complementing the estimated plan from `EXPLAIN`.

https://dev.mysql.com/blog-archive/mysql-explain-analyze/

```
EXPLAIN ANALYZE
SELECT first_name, last_name, SUM(amount) AS total
FROM staff INNER JOIN payment
 ON staff.staff_id = payment.staff_id
   AND
   payment_date LIKE '2005-08%'
GROUP BY first_name, last_name;
```

```
-> Table scan on <temporary>  (actual time=0.001..0.001 rows=2 loops=1)
  -> Aggregate using temporary table  (actual time=58.104..58.104 rows=2 loops=1)
    -> Nested loop inner join  (cost=1757.30 rows=1787) (actual time=0.816..46.135 rows=5687 loops=1)
      -> Table scan on staff  (cost=3.20 rows=2) (actual time=0.047..0.051 rows=2 loops=1)
      -> Filter: (payment.payment_date like '2005-08%')  (cost=117.43 rows=894) (actual time=0.464..22.767 rows=2844 loops=2)
        -> Index lookup on payment using idx_fk_staff_id (staff_id=staff.staff_id)  (cost=117.43 rows=8043) (actual time=0.
```

INDEX

Exibir os índices de determinada tabela:

SHOW INDEXES FROM *table*;

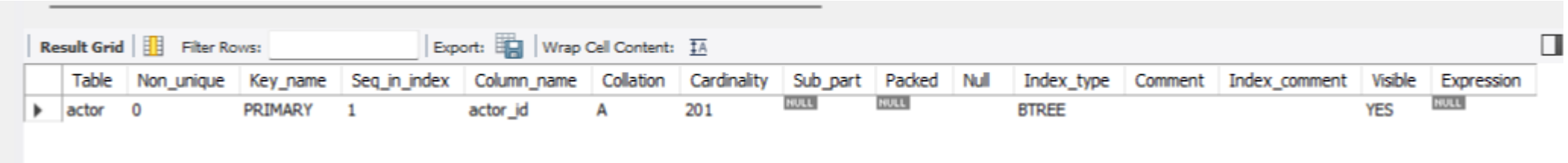

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
actor	0	PRIMARY	1	actor_id	A	201	NULL	NULL		BTREE			YES	NULL

INDEX

É possível ter uma estrutura de índices baseadas em múltiplos campos.

Estas são úteis quando os critérios de filtro (WHERE) são baseados em mais de um campo.

```
CREATE TABLE test ( id INT NOT NULL, last_name CHAR(30) NOT NULL, first_name CHAR(30) NOT NULL,
PRIMARY KEY (id), INDEX name (last_name,first_name) );
```

```
SELECT * FROM tbl_name WHERE col1=val1 AND col2=val2;
```

MySQL :: MySQL 8.0 Reference Manual :: 8.3.6 Multiple-Column Indexes

Index – Plano de execução

Os sistemas gerenciadores de banco de dados (SGBD) relacionais possuem diversos componentes.

Um componente que todos SGBD têm é o otimizador, que tem por objetivo:

A) agrupar gatilhos que possuem condições semelhantes de ativação.

B) ajustar o conjunto ótimo de restrições de integridade.

C) criar um plano para que a execução de uma consulta seja rápida.

D) melhorar o desempenho de novas definições de tabelas.

E) tornar uma atualização de dados a mais rápida possível.

SQL – DDL - INDEX

```
explain select * from sakila.customer;
explain select * from sakila.customer c where active = 1;
explain select customer_id from sakila.customer c where active = 1;
create index active_customer on sakila.customer (active,customer_id);
drop index active_customer on sakila.customer;
-- type ALL: fullscan
```

Exemplo:

Adding the index {a:1} can potentially decrease the speed of which of the following operations? Check all that apply.

- A) db.collection.update({b:456>, {$inc: {a:1} }) (correct)
- B) db.collection.find({a : 232})
- C) db.collection.insert({ a:341}) (correct)

INDEX - HASH

Em um sistema de e-commerce, foi criado um índice do tipo hash sobre a coluna preço da tabela produtos. Esse índice está sendo utilizado em consultas com filtros como:

SELECT * FROM produtos WHERE preco BETWEEN 100 AND 300;

I. Índices hash são ineficientes para consultas com operadores de intervalo, como BETWEEN, porque sua estrutura não mantém ordenação dos valores indexados.

II. A atualização de índices hash ocorre na reconstrução completa do índice sempre que um novo registro é inserido na tabela produtos em sistemas de banco de dados como PostgreSQL e MySQL.

III. A limitação dos índices hash em consultas de intervalo reside em sua restrição a campos numéricos, o que os tornam inutilizáveis para campos textuais ou data. É correto o que se afirma em:

A) I, apenas. (gabarito)

B) I e II, apenas.

C) II e III, apenas.

D) III, apenas.

E) I, II e III.

Exemplo:

Considere a tabela EMPREGADOS definida abaixo em SQL.

```
Create table EMPREGADOS
(CODEMP INT PRIMARY KEY,
NOMEEMP VARCHAR(300) NOT NULL UNIQUE,
FUNCAO INT CHECK(FUNCAO BETWEEN 1 AND 5),
SALARIO FLOAT NOT NULL,
DEPTO INT NOT NULL);
```

- Sobre esta tabela, foi definido um índice **primário (codemp – chave primária)**, e dois índices secundários, um sobre **nomeemp**, e outro sobre **funcao**.
- Uma pessoa do desenvolvimento reclamou à DBA que algumas de suas consultas sobre essa tabela estavam muito demoradas, e pediu apoio para melhoria do desempenho. A DBA examinou o plano de execução das consultas e, em vez de uma solução sobre o esquema da base de dados, sugeriu a reescrita das consultas.

	ORIGINAL	REESCRITA
I.	SELECT NOMEEMP, FUNCAO, DEPTO FROM EMPREGADOS WHERE SUBSTR(NOMEEMP, 1, 5) = 'MARIA';	SELECT NOMEEMP, FUNCAO, DEPTO FROM EMPREGADOS WHERE NOMEEMP LIKE 'MARIA%';
II.	SELECT NOMEEMP FROM EMPREGADOS WHERE FUNCAO <> 5;	SELECT NOMEEMP FROM EMPREGADOS WHERE FUNCAO BETWEEN 1 and 4;
III.	SELECT DISTINCT NOMEEMP, SALARIO, DEPTO FROM EMPREGADOS WHERE FUNCAO = 1;	SELECT NOMEEMP, SALARIO, DEPTO FROM EMPREGADOS WHERE FUNCAO = 1;

Saída de função não consegue mapear campos de índice.

Operador de <> não consegue fazer acesso indexado, salvo comparação, no caso a caso.

a) Apenas I
b) Apenas II
c) Apenas III
d) Apenas I e II
e) I, II, e II

Resposta: D

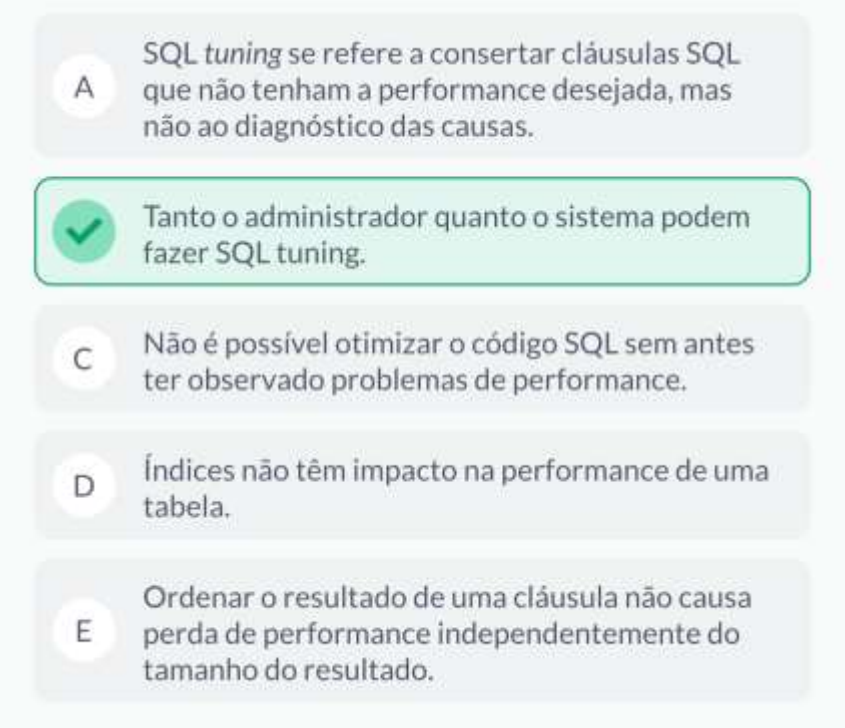
Exemplo

- Um banco de dados (BD) persiste dados de forma organizada e controlada. Em adição, um BD deve prover recursos para permitir que **consultas** que necessitem de velocidade (baixo tempo de resposta) no acesso aos dados possam ter um **bom desempenho**.
- Um dos recursos que um profissional de tecnologia da informação tem à disposição para configurar um BD, de modo a melhorar o desempenho de consultas selecionadas, é a criação de



Exemplo

- Assinale a alternativa correta sobre otimização de código SQL (tuning).



Criação de indexes relevantes é uma prática de *tuning* de banco de dados.

Exemplo

- Jeremias é o administrador do banco de dados BD_MAIS gerenciado pelo MySQL. Jeremias tem recebido reclamações de que operações de manipulações de dados estão com tempos de respostas altos.
  Para **diagnosticar o motivo dos tempos de respostas** do BD_MAIS, Jeremias deve:



INDEX

Em um banco de dados relacional, o índice:

A) garante a consistência dos dados, evitando conflitos e duplicidades.

B) permite a criação de relacionamentos entre tabelas, garantindo a integridade referencial.

C) é responsável por controlar o acesso e a autenticação dos usuários, garantindo a segurança dos dados.

D) permite a organização física dos registros, melhorando o desempenho das consultas. (correta)

E) facilita a modelagem conceitual, definindo os atributos e as entidades do sistema.

Exemplo: indexes

Ao analisar as consultas mais frequentes em um banco de dados relacional, com baixa frequência de alterações de dados, um DBA decidiu tomar uma ação visando melhorar a performance, reduzindo o tempo de resposta dessas consultas. Assinale a alternativa que seria efetiva no cenário descrito.

- A) Normalizar até 4NF as tabelas que envolvem consultas com muitos JOINs.
- **B) Criar índices em colunas utilizadas com frequência em operações de busca (gabarito).**
- C) Utilizar comandos NoSQL para remover o overhead das consultas.
- D) Alterar a estrutura de dados do armazenamento das tabelas por arquivos JSON.

Index

A respeito dos sistemas gerenciadores de banco de dados (SGBDs), julgue o item a seguir.

Em um SGBD, o uso de índices pode melhorar o desempenho das consultas, mas pode diminuir a performance nas operações de inserção, atualização e exclusão, uma vez que **o índice precisa ser atualizado sempre** que os dados da tabela forem modificados.

Certo (gabarito)

Errado

INDEX - atividades

a) Crie um índice com base em uma coluna do tipo number e experimente seu uso em uma consulta.

b) Crie um índice com base em coluna do tipo date, e experimente o seu uso em uma consulta.

c) Crie um índice composto por duas colunas, uma number e outra date e experimente seu uso.

DTL/TCL – Data Transaction Language

```
START TRANSACTION; /*Marca o começo de uma transação no banco da dados  que pode ser completada ou não.*/
-- instruções SQL
COMMIT; /*Envia todos os dados da transação permanentemente para o banco de dados.*/

ROLLBACK; /*Desfaz as alterações feitas na transação realizada./*
```

Rollback só desfaz transações ainda não “comitadas”

DTL :: Savepoint

- A save point is a logical rollback point within a transaction. When you set a save point, whenever an error occurs past a save point, you can undo the events you have done up to the save point using the rollback.

SAVEPOINT *identifier*
ROLLBACK [WORK] TO [SAVEPOINT] *identifier*
RELEASE SAVEPOINT *identifier*

- The `SAVEPOINT` statement sets a named transaction savepoint with a name of ***`identifier`***. If the current transaction has a savepoint with the same name, the old savepoint is deleted and a new one is set.
- The `ROLLBACK TO SAVEPOINT` statement rolls back a transaction to the named savepoint without terminating the transaction.
- The `RELEASE SAVEPOINT` statement removes the named savepoint from the set of savepoints of the current transaction. No commit or rollback occurs. It is an error if the savepoint does not exist

https://dev.mysql.com/doc/refman/8.0/en/savepoint.html

Nível de Isolamento

Transações ACID em bancos de dados relacionais garantem propriedades fundamentais para manter a integridade dos dados em operações concorrentes. Um dos parâmetros mais importantes é o nível de isolamento (Isolation Level), que define como e quando os efeitos das alterações realizadas por uma transação são visíveis para outras. Considere uma aplicação bancária de grande porte, utilizando um sistema de gerenciamento de banco de dados (SGBD) com suporte ao nível de isolamento SERIALIZABLE. Nessa configuração, assinale a alternativa que descreve corretamente o comportamento pela escolha do nível de isolamento SERIALIZABLE:

A) Transações podem visualizar dados modificados por outras transações antes do commit (dirty reads).

B) Transações não utilizam bloqueios para controlar o acesso concorrente.

C) A atomicidade da transação é garantida, não a consistência.

D) Nenhuma transação concorrente pode causar leituras fantasmas (phantom reads). (gabarito)

E) Esse nível prioriza o desempenho ao invés da consistência dos dados.

Concorrência

Em uma aplicação bancária que utiliza um sistema de banco de dados relacional compatível com o padrão ANSI SQL, um desenvolvedor implementa a seguinte lógica para garantir a consistência ao realizar transferências entre contas:

```
BEGIN;
SELECT saldo FROM contas WHERE id = 101 FOR UPDATE;
UPDATE contas SET saldo = saldo - 500 WHERE id = 101;
SELECT saldo FROM contas WHERE id = 202 FOR UPDATE;
UPDATE contas SET saldo = saldo + 500 WHERE id = 202;
COMMIT;
```

Durante testes de carga, outra transação concorrente tenta acessar a conta de id 101 logo após o início da transação acima. Com base nesse contexto, avalie as afirmações a seguir:

I. A cláusula FOR UPDATE utilizada no SELECT bloqueia a linha lida até o fim da transação, impedindo que outras transações atualizem ou façam SELECT FOR UPDATE na mesma linha enquanto o bloqueio estiver ativo.

II. O uso de FOR UPDATE é eficaz para evitar condições de corrida (race conditions) ao modificar linhas críticas em um modelo de leitura-modificação-escrita.

III. A cláusula FOR UPDATE garante isolamento no nível SERIALIZABLE por si só, impedindo leitura fantasma e garantindo que nenhuma transação concorrente acesse qualquer dado relacionado.

É correto o que se afirma em:

A) I, II e III.

B) I e III, apenas.

C) I e II, apenas. (gabarito)

D) II, apenas.

E) III, apenas

Problemas da concorrência não tratada

Em um sistema de gerenciamento de banco de dados relacional que implementa os níveis de isolamento definidos pelo padrão ANSI SQL, cada nível oferece diferentes garantias contra anomalias de concorrência, como:

- Leitura suja (dirty read): leitura de dados ainda não confirmados.
- Leitura não repetível (non-repeatable read): dados lidos são alterados por outra transação entre duas leituras.
- Leitura fantasma (phantom read): novas linhas são inseridas por outras transações entre duas leituras.

Com base nesse contexto, avalie as afirmações a seguir:

I. O nível Read Uncommitted permite todas as três anomalias: leituras sujas, leituras não repetíveis e leituras fantasmas.

II. O nível Repeatable Read previne leituras sujas e leituras não repetíveis, mas não previne leituras fantasmas.

III. O nível Serializable evita todos os três fenômenos de concorrência, oferecendo o mais alto grau de isolamento conforme o padrão ANSI SQL.

É correto o que se afirma

A) I, II e III. (gabarito)

B) I e II, apenas.

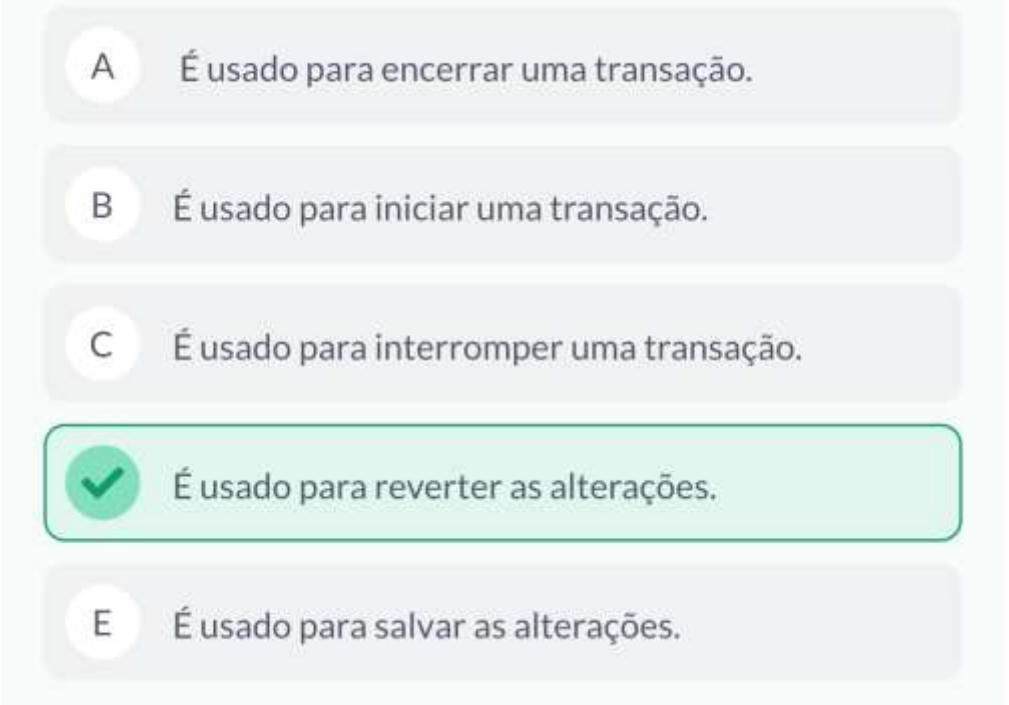
C) II e III, apenas.

D) III, apenas.

E) I, apenas

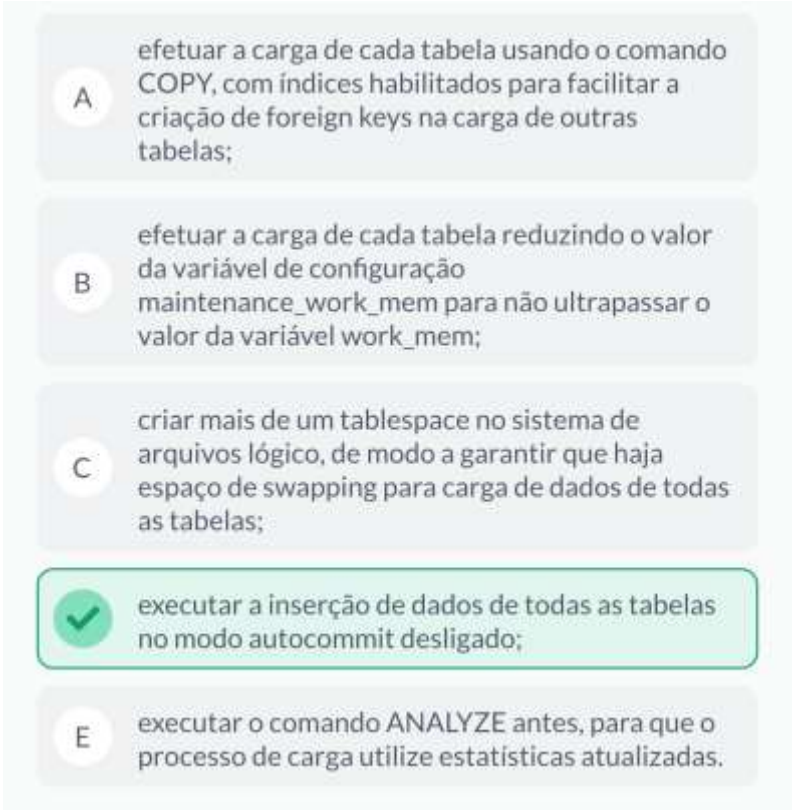
Exemplo:

- A transação é executada em um banco de dados para realizar uma ou mais operações. Por exemplo, para atualizar um registro ou excluir um, o usuário realizará uma transação na tabela. Desta forma, tendo como referência o PostgreSQL, indique a opção correta sobre o comando ROLLBACK:



Exemplo

- Juliana necessita preparar o banco de dados BD_APOIO criado no PostgreSQL para o primeiro uso. De modo a executar a tarefa de popular o BD_APOIO com celeridade, Juliana deve:



Exemplo - Transações

- Relacione cada tipo de transação disponível no SQL Server às suas características correspondentes.
- I) Transações explícitas
- II) Transações implícitas
- III) Transações de escopo de lote
- a) Cada transação é iniciada com a instrução BEGIN TRANSACTION e finalizada somente com uma instrução COMMIT ou ROLLBACK.
- b) Aplicável apenas a MARS. A transação que não é confirmada ou revertida quando um lote é concluído, será revertida automaticamente pelo SQL Server.
- c) Uma transação nova é iniciada quando a transação anterior for concluída, mas cada transação é somente concluída com uma instrução COMMIT ou ROLLBACK.
- A) I - a; II - b; III - c.
- **B) I- a; II - c; III - b. (gabarito)**
- C) I - c; II - b; III - a.
- D) I - c; II - a; III - b.
- E) I - b; II - a; III - c

Transações

Considerando os serviços de reparos e o funcionamento de Sistemas de Gerenciamento de Banco de Dados (SGBD), analise as afirmativas a seguir.

I. Em um SGBD, os serviços de reparos são essenciais para garantir a integridade dos dados, especialmente após falhas no sistema, como quedas de energia ou corrupção de arquivos.

II. Um SGBD oferece mecanismos de recuperação automática, como logs de transações e pontos de restauração (checkpoints), que permitem restaurar o banco de dados para um estado consistente após uma falha.

III. Os serviços de reparos em um SGBD envolvem a correção de dados e a recuperação de transações para garantir a consistência do banco de dados.

Estão corretas as afirmativas

A) I e II, apenas.

B) I e III, apenas.

C) II e III, apenas.

D) I, II e III. (gabarito)

Exemplo:

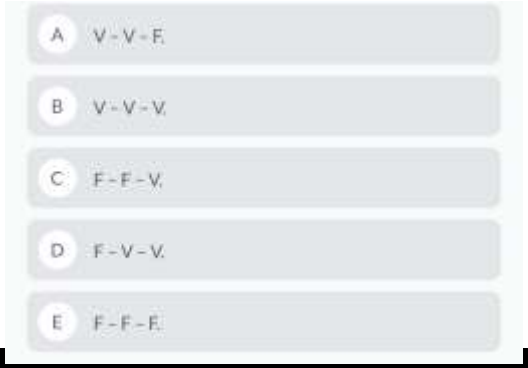
- Suponha que um programador tenha especificado a seguinte sequência de comandos SQL:

```
•set transaction isolation level repeatable read;
•Select max(salario) from funcionarios;
•Update funcionariosset salario = salario*10 where salario < (select avg(salario) from funcionarios);
•commit;
```

Ao definir esse nível de isolamento para sua transação, o programador está sujeito à ocorrência de problemas de acesso concorrente com outras transações. Analise cada assertiva abaixo, assinalando V, se verdadeira, ou F, se falsa.

- ( ) Leitura suja.
- ( ) Leitura não repetitiva.
- ( ) Registro fantasma.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:



Repeatable Read Isolation Level.
The Repeatable Read isolation level only sees data committed before the transaction began; it never sees either uncommitted data or changes committed during transaction execution by concurrent transactions.

Transações: Técnicas de controle de concorrênica

Em sistemas que utilizam múltiplas transações simultâneas, mecanismos de controle de concorrência são essenciais para evitar inconsistências e conflitos. Sobre tais mecanismos, assinale a alternativa INCORRETA.

A) Locks exclusivos impedem que duas transações modifiquem o mesmo dado ao mesmo tempo.

B) Timestamps podem ser usados para serializar transações e evitar conflitos lógicos.

C) Multiversion concurrency control (MVCC) permite leituras consistentes sem bloquear operações de escrita.

D) O log de transações substitui mecanismos de concorrência, sendo suficiente para impedir conflitos. (gabarito)

E) Locks compartilhados permitem leitura simultânea por várias transações

DTL - Atividade

```
START TRANSACTION; /*Marca o começo de uma transação no banco da dados  que pode ser completada ou não.*/
insert into actor (first_name, last_name) values ("Rafael","Gonçalves");
-- COMMIT; /*Envia todos os dados da transação permanentemente para o banco de dados.*/
ROLLBACK; /*Desfaz as alterações feitas na transação realizada./*
```

1. Iniciar uma transação – e executar operações de escrita. Não realizar commit.
2. Selecionar os dados nas tabelas escritas: observar o resultado.
3. Aplicar rollback e repetir o passo 2.
4. Repetir o procedimento do início, porém após o passo 1 realizar operação de commit.

Aula 8

Check e subqueries

Prof. Dr. Rafael Queiroz Gonçalves, PMP

CHECK

- CHECK constraint pode limitar o interval de valores uma certa coluna pode assumir, em uma escala ordinal.
- Pode ser permitido somente valores específicos dentro de uma escala nominal.
- Pode ser utilizado para alguma regra de permissionamento baseado nos valores de outras colunas da mesma linha.

CHECK – create table

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CHECK (Age>=18)
);
```

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255),
    CONSTRAINT CHK_Person CHECK (Age>=18 AND
City='Sandnes')
);
```

CHECK – alter table

```
ALTER TABLE Persons
ADD CHECK (Age>=18);

ALTER TABLE Persons
ADD CONSTRAINT CHK_PersonAge CHECK (Age>=18 AND City='Sandnes');

ALTER TABLE Persons
DROP CHECK CHK_PersonAge;
```

```
ADD CONSTRAINT CHK_LOGDATE tl.task_log_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
```

Check

Em um banco de dados Oracle aberto e em condições ideais, o comando utilizado para adicionar uma coluna custas do tipo NUMBER (10,2) à tabela processo, estabelecendo uma restrição que impede que valores de custas menores ou iguais a 10000 sejam inseridos, é

A) ADD COLUMN custas NUMBER(10,2) TO processo WHERE custas > 10000;

B) ALTER TABLE processo ADD custas NUMBER (10,2) CONSTRAINT processo_custas_ck CHECK (custas > 10000);

C) ADD COLUMN custas NUMBER(10,2) TO processo WITH CONSTRAINT processo_custas_ck CHECK (custas 10000);

D) ALTER TABLE processo ADD custas NUMBER (10,2) ADD CONSTRAINT processo_custas_ck CHECK (custas >= 10000); (gabarito)

E) ALTER TABLE processo ADD custas NUMBER (10,2) CONSTRAINT processo_custas_ck WHERE (custas > 10000);

Exemplo

- Considere a tabela ALUNOS criada usando o comando SQL (padrão SQL99 ou superior) abaixo.

Create table Alunos(pk int not null primary key,nomea varchar(100),idade int,nascimento date not null);

- Assinale qual, dentre as instruções ALTER TABLE listadas abaixo, representa um comando SQL INVÁLIDO, que causa um erro de execução.




```
create index idx_name on table table_name (column_name)
```

Exemplo - Check

No momento da criação de uma tabela em um Sistema Gerenciador de Banco de Dados (SGBD), por meio do comando CREATE TABLE, definem-se regras para o controle de integridade e para a garantia de qualidade dos dados dessa tabela.

Considere que W e Y são duas tabelas. A é um atributo de W, e B é um atributo de Y. Foi criada corretamente uma cláusula CHECK no atributo A.

Essa cláusula permite verificar se o atributo A:

A) tem valor único em todas as linhas da tabela W.

B) tem valor maior ou igual a zero. (gabarito)

C) tem valor existente em uma ou mais linhas da tabela Y na coluna B.

D) inviabiliza o acesso a alguma linha da tabela Y.

E) está preenchido com algum valor ou não. => (NOT NULL)

Atividade: check

A) Considerando uma entidade Incidentes (descrição: varchar (255); prioridade (ALTA | MÉDIA | BAIXA: varchar (10); criado_em:datetime), faça:

- A) Não possibilitar inserção de registros com valor de prioridade diferente dos 3 previstos no enunciado.
- B) Não possibilitar descrição de incidente com menos de 30 caracteres.

B) Considerando uma entidade Resultado (nome_candidato: varchar(60); nome: float; prova: varchar(60), data_prova: date): faça:

- C) A nota não pode assumir valores inferiores a 0 ou superiores a 100.
- D) O nome do candidato deve possuir ao menos 2 palavras.
- E) A data da prova não pode ser nula.

SQL – SELECT – “sub queries”

IN operator

```
select c.city_id, c.city from sakila.city as c where c.city_id in (SELECT a.city_id
FROM sakila.address a)

select c.city_id, c.city from sakila.city as c where c.city_id not in (SELECT a.city_id
FROM sakila.address a)
```

SQL – SELECT – “sub queries”

```
SELECT * FROM sakila.film;
SELECT * FROM sakila.film_actor;

SELECT * FROM sakila.film f where exists (SELECT * FROM sakila.film_actor fa where
f.film_id=fa.film_id and fa.actor_id=5);

SELECT * FROM sakila.customer;
SELECT * FROM sakila.payment;

SELECT * FROM sakila.customer c where 10 < any (select amount from sakila.payment p
where c.customer_id = p.customer_id);

SELECT * FROM sakila.customer c where 2 < all (select amount from sakila.payment p
where c.customer_id = p.customer_id);
```

Subquery – plano de execução

Analise as seguintes afirmações a respeito de subconsultas (nested queries) em SQL e registre V, para verdadeiras, e F, para falsas:

(__) Subconsultas são consultas aninhadas dentro de outras consultas.

(__) Subconsultas podem aparecer em diferentes cláusulas de uma consulta, como WHERE, FROM e HAVING.

(__) Subconsultas são executadas apenas uma vez, independentemente da consulta externa.

Assinale a alternativa que apresenta a sequência correta:

A) V – V – F. (gabarito)

B) F – F – F.

C) F – V – V.

D) F– F – V.

E) V – V – V.

SQL – SELECT – “sub queries”: EXISTS vs IN

Afirmação: Ao ser executado, o comando SQL a seguir mostrará o CPF e o nome de todas as pessoas que cometeram pelo menos um crime do tipo hediondo.

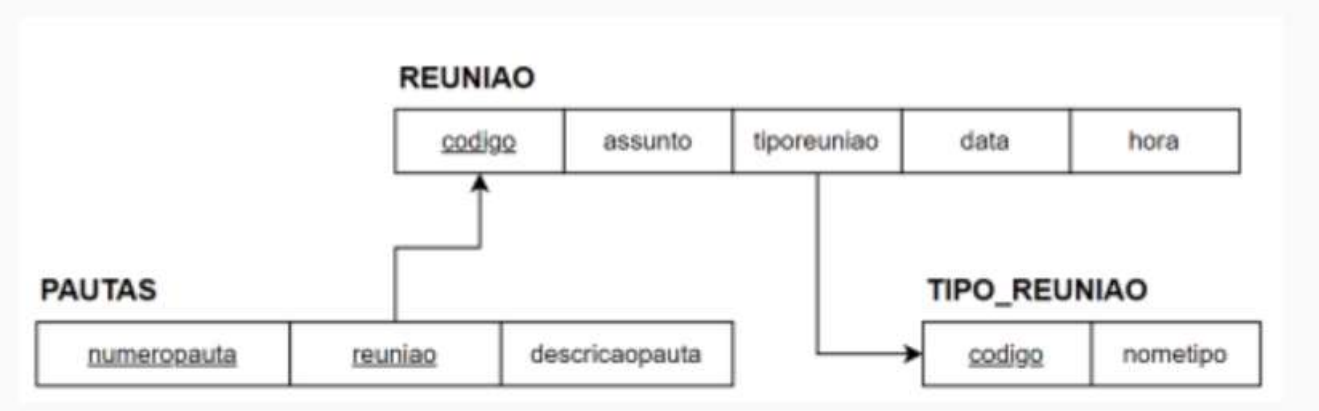
```
select P.CPF, P.NOME
from POLITEC.PESSOA P
where P.CPF exists (select C.CPF from POLITEC.CRIME C where C.CPF=P.CPF and C.TIPO = ‘HEDIONDO’);
```

( ) VERDADEIRO
( ) FALSO



Aqui seria esperado o uso de "in" e não de "exists".

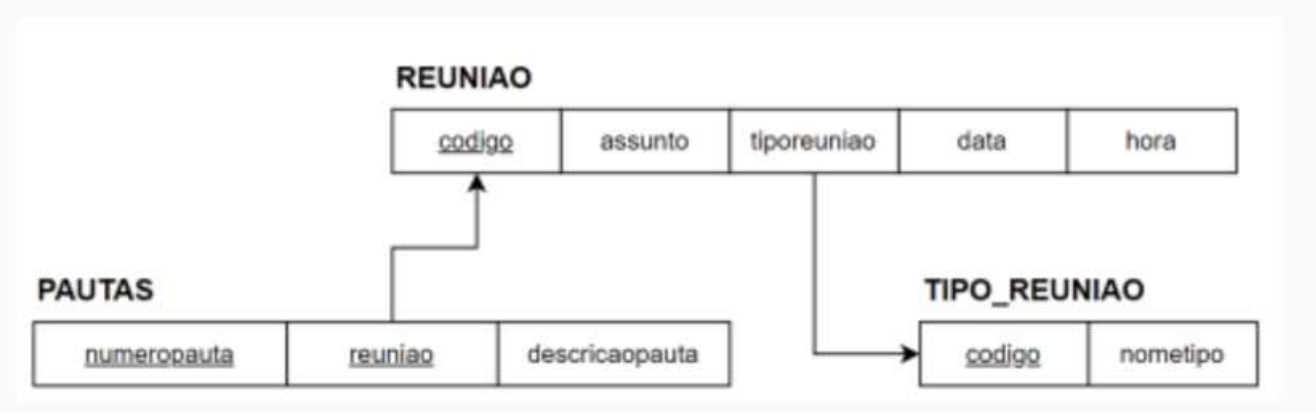
SQL – SELECT – “sub queries”



Escolha a sentença SQL, considerando o padrão ANSI, que **MELHOR** responda à consulta "Listar o codigo e o assunto das reuniões sem pautas cadastradas.":

Demonstração no quadro

SQL – SELECT – “sub queries”



Escolha a sentença SQL que **MELHOR** responda à consulta “**Excluir as pautas da relação PAUTAS cuja reunião correspondente tenha o campo assunto igual a 'X'**”:

Demonstração no quadro

Exemplo: NOT IN

- Uma empresa de investimentos financeiros busca identificar novas oportunidades de negócio para pessoas jurídicas, em especial dentre aquelas que têm como característica a adoção de governança ambiental, social e corporativa (conhecida como ESG, uma sigla em inglês).
- Considere que existe um banco de dados nessa empresa com as seguintes tabelas (todas as chaves primárias são numéricas):
  - Empresa (CNPJ, razaoSocial, endereco)
  - Caracteristica (cod, sigla, nome)
  - Tem (CNPJ, cod)
- Que comando SELECT do SQL retorna apenas o CNPJ e a razão social das empresas que não têm "ESG" como característica?



Exemplo: IN para lista de valores

- Considerando a tabela PRODUTO, apresentada na Figura 2, quantos registros serão excluídos pelo comando SQL abaixo?

MARCA

ID	NOME	GRUPO
1	ABC do Sul	A
2	POA Vintage	B
3	Meia Lua	C
4	Cube	A

PRODUTO

ID	DESCRICAO	ID_MARCA
1	Regata	1
2	Tênis	2
3	Sandália	2
4	Cubo mágico 3x3x3	4
5	Chinelo	2
6	Camiseta manga curta	1

Figura 2 – Tabelas MARCA e PRODUTO

```
DELETE FROM PRODUTO
WHERE ID_MARCA NOT IN (2, 4);
```

A 0

✓ 2

C 3

D 4

E 6

Exemplo: IN para lista de valores

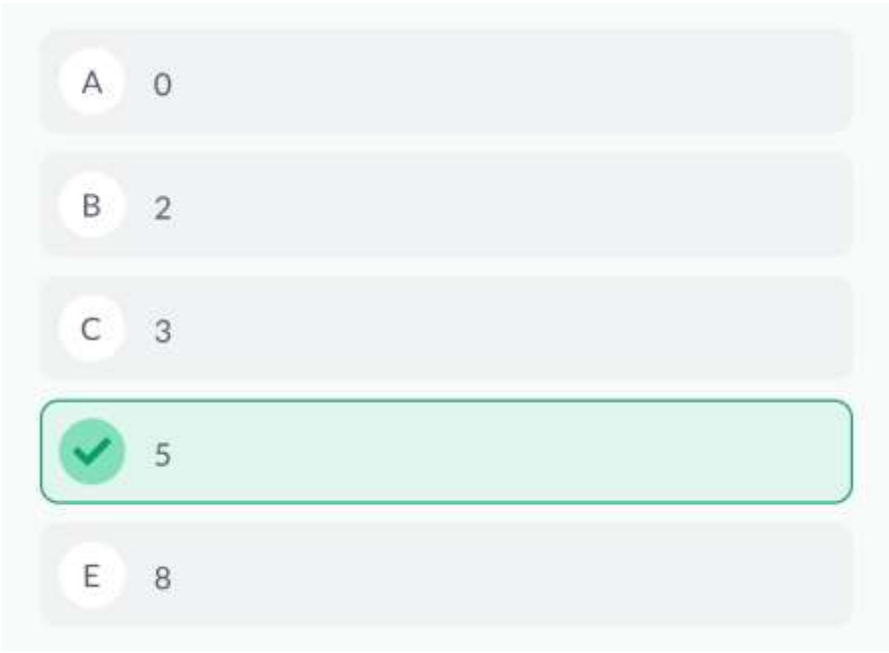
Considerando a tabela "Funcionário", apresentada na Figura 2, quantos registros serão atualizados pelo comando SQL abaixo?

- UPDATE FUNCIONARIO SET ID_FUNCIONARIO_SUPERIOR = 5 WHERE ID_CARGO **IN (2,4)**

CARGO			
ID	DESCRICAO	NIVEL	ID_CARGO_SUPERIOR
1	Gerente Administrativo	B-1	
2	Corretor	A-1	1
3	Analista Administrativo	A-2	1
4	Auxiliar Administrativo	A-2	3

FUNCIONARIO			
ID	NOME	ID_CARGO	ID_FUNCIONARIO_SUPERIOR
1	Agatha Souza	1	
2	João Lucas Fogaça	2	1
3	Thales Rocha	2	1
4	Ana Vitoria Ribeiro	2	1
5	Matheus Araújo	3	1
6	Breno Cunha	3	1
7	Cauã da Cruz	4	5
8	Barbara Cardoso	4	6

Figura 2 – Tabelas "Cargo" e "Funcionário"



Exemplo: comparação de NULL (IS NULL)

- Na questão a seguir, considere a tabela relacional T cuja instância é exibida abaixo. A instalação está configurada para o tratamento de valores NULL como valores desconhecidos (unknown).

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL
3	6	5

Considerando a tabela T da figura 1, analise o comando a seguir.

```
DELETE FROM T
WHERE EXISTS ( SELECT * FROM T t2
        WHERE T.A = t2.A
          and T.B = t2.B
          and T.C = t2.C )
```

O número de linhas removidas pela execução do comando acima é:

- ✓ 0;
- B 1;
- C 2;
- D 3;
- E 4.

Exemplo

Considere as seguintes tabelas que pertencem a parte de um determinado banco de dados:

/*Tabela de Clínicas*/

- Clinica(CodClinica, NomeClinica, LocalClinica)

/* tabela de médicos */

- Medico(CRM, NomeMedico, Telefone)

/* tabela que relaciona médicos a clínicas */

- ClinicaMedico(CodClinica, CRM) CodClinica referencia Clinica, CRM referencia Medico

Utilizando o SQL Server 2019, assinale o comando que lista códigos e nomes dos médicos que não atuam em nenhuma clínica.

A. SELECT Medico.CRM, nomeMedico FROM Medico LEFT JOIN ClinicaMedico ON ClinicaMedico.codClinica IS NULL

B. SELECT Medico.CRM, nomeMedico FROM Medico RIGHT JOIN ClinicaMedico ON ClinicaMedico.codClinica ISNULL

✓ SELECT CRM, nomeMedico FROM Medico WHERE NOT EXISTS (SELECT * FROM ClinicaMedico WHERE CRM = Medico.CRM)

D. SELECT CRM, nomeMedico FROM Medico WHERE CRM NOT IN (SELECT * FROM ClinicaMedico WHERE CRM = Medico.CRM)

Atividade: Sub queries

Dadas as entidades **Unidade** (cidade: varchar(40)), **Produto** (descrição: varchar(50), preço: float), **Vendedor** (nome: varchar(60), matricula: int), **Venda** (unidade: Unidade; produto: Produto, quantidade: float; vendedor: Vendedor, data: Date) faça:

A) Retorne todos os vendedores que não possuem nenhuma venda realizada no último mês.

B) Retorne todos os produtos que tiveram ao menos 1 venda na última semana.

C) Retorne todas as unidades que não possuem vendas no último dia.

Aula 9

Agregação de dados e Junções entre tabelas

Prof. Dr. Rafael Queiroz Gonçalves, PMP

SQL – DML – SELECT - JOIN

```sql
select c.id, c.nome, l.nome_localidade, l.chave
from cliente c
join localidades_clientes l on c.chave_localidade=l.chave;

select c.id, c.nome, l.nome_localidade, l.chave
from cliente c
left join localidades_clientes l on c.chave_localidade=l.chave;

select c.id, c.nome, l.nome_localidade, l.chave
from cliente c
right join localidades_clientes l on c.chave_localidade=l.chave;
```

Exemplo: Produto carteziano

- SELECT A.CODIGO, A.NOME, B.DATA, B.VALOR, C.QTD, D.DESCRIC FROM CLIENTES A, PEDIDOS B, ITENS C, PRODUTOS D WHERE A.CODIGO = B.CODCLIENTE AND B.CODIGO = C.CODPEDIDO AND C.CODPRODUTO = D.CODIGO
- Qual o efeito da execução desse código no banco de dados?

A	No comando acima temos as tabelas clientes e pedidos apenas, unificadas em uma mesma cláusula.
B	No comando acima temos as tabelas, itens e produtos apenas, unificadas em uma mesma cláusula.
C	No comando acima foram atribuídos aliases às tabelas itens e produtos apenas.
D	As tabelas são unificadas, todas elas, por um campo chave, no caso as letras A, B, C e D.
E	As tabelas são unificadas, todas elas, por um campo chave.

Resposta: E

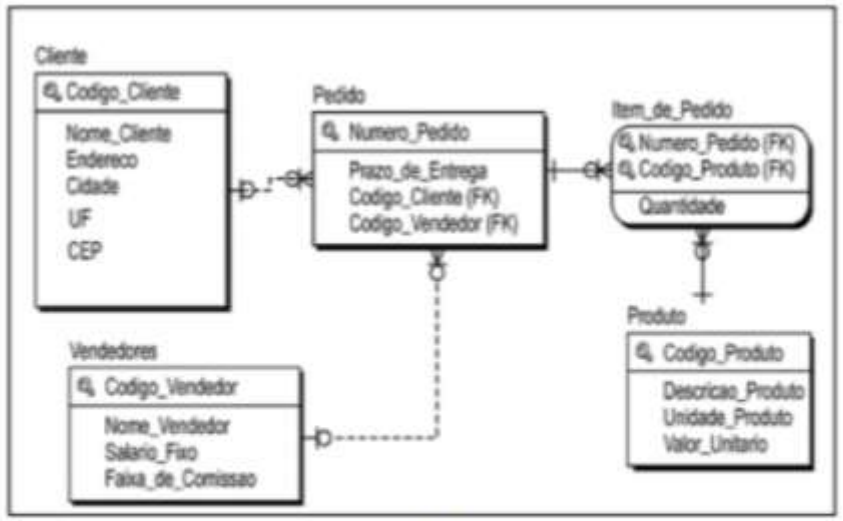


Figura 1: Diagrama de Tabelas Relacionais.

A seguir são apresentados os registros das tabelas: Cliente; Vendedor; e, Pedido:

cod_cliente [PK] integer	nome_cliente character varying (30)	endereço character varying (30)	cidade character varying (20)	uf character (2)	cep integer
20	BETH	AV. OLIMPICO N 40	SÃO PAULO	SP	25679000
110	JORGE	RUA CAIAPÓ 13	CURITIBA	PR	38078500
130	EDMAR	RUA DA PRAIA S/N	SALVADOR	BA	38079000
157	PAULO	TV. MORAES C/3	LONDRINA	PR	[null]
180	LIVIO	AV. BEIRA MAR N 1256	FLORIANOPOLIS	SC	38077500
222	LUCIA	RUA ITABIRA 123 LOJ...	BELO HORIZONTE	MG	22124391
234	JOSI	QUADRA 3 SL. 3 SL 1...	BRASILIA	DF	38046500
260	SUSANA	RUA LOPES MENDES ...	NITERÓI	RJ	38225900
290	RENATO	RUA MEIRELLES N 12...	SÃO PAULO	SP	38438700
390	SEBASTIAO	RUA DA IGREJA N 10	UBERABA	MG	38438700
410	RODOLFO	LARGO DA LAPA 27 S...	RIO DE JANEIRO	RJ	38078900
720	ANA	RUA 17 N. 19	NITERÓI	RJ	24358310
830	MAURICIO	AV. PAULISTA 1236 S...	SÃO PAULO	SP	38126830
870	FLAVIO	AV. PRES. VARGAS 100	SÃO PAULO	SP	22762931

numero_pedido [PK] integer	prazo_de_entrega integer	codigo_cliente integer	codigo_vendedor integer
91	20	260	11
97	20	720	101
98	20	410	209
101	15	720	101
103	20	260	11
104	30	110	101
105	15	180	240
108	15	290	310
111	20	260	240
119	30	390	250
121	20	410	209
127	10	410	11
137	20	720	720
138	20	260	11
143	30	20	111
148	20	720	101
169	15	870	213

Para fazer uma consulta ao banco de dados para listar em ordem alfabética quais clientes têm pedidos com prazo de entrega igual ou superior a quinze dias e que pertencem aos estados de São Paulo ('SP') ou de Minas Gerais ('MG'), o seguinte comando deverá ser utilizado:

A) SELECT cliente.nome_cliente, cliente.UF, pedido.prazo_de_entrega FROM cliente INNER JOIN pedido ON cliente.codigo_cliente = pedido.codigo_cliente WHERE UF IN ('SP', 'MG') AND prazo_de_entrega >= 15;

B) SELECT cliente.nome_cliente, cliente.UF, pedido.prazo_de_entrega FROM cliente INNER JOIN pedido ON cliente.codigo_cliente = pedido.codigo_cliente WHERE UF IN ('SP', 'MG') AND prazo_de_entrega >= 15 ORDER BY 1; **(correta)**

C) SELECT cliente.nome_cliente, cliente.UF, pedido.prazo_de_entrega FROM cliente INNER JOIN pedido ON cliente.codigo_cliente = pedido.codigo_cliente ORDER BY cliente.nome_cliente WHERE UF = ('SP') OR UF = ('MG') AND prazo_de_entrega >= 15;

D) SELECT COUNT(prazo_de_entrega), cliente.nome_cliente, cliente.UF FROM cliente INNER JOIN pedido ON cliente.codigo_cliente = pedido.codigo_cliente GROUP BY cliente.nome_cliente WHERE UF = ('SP') OR UF = ('MG') AND HAVING COUNT(prazo_de_entrega)>=15 ORDER BY cliente.nome_cliente;

E) SELECT SUM(prazo_de_entrega), cliente.nome_cliente, cliente.UF FROM cliente INNER JOIN pedido ON cliente.codigo_cliente = pedido.codigo_cliente GROUP BY cliente.nome_cliente ORDER BY cliente.nome_cliente WHERE UF = ('SP') AND UF = ('MG') AND HAVING SUM(prazo_de_entrega)>=15;

Exemplo: Junção natural

- A junção natural é baseada em todas as colunas nas duas tabelas que possuem o mesmo nome, ela seleciona linhas das duas tabelas que têm valores iguais nas colunas relevantes, por exemplo: **TABELA1.CAMPO1 e TABELA2.CAMPO1**, neste caso, se utilizado o **NATURAL JOIN**.

- Uma escola que oferece aulas de reforço possui um cadastro de professores especializados em alguma matéria e de alunos que precisam de reforço escolar em matérias específicas.

- As tabelas estão desenhadas abaixo.

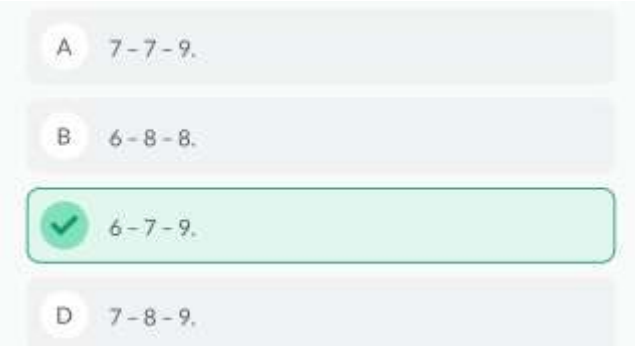
Professores		
codp	**nomep**	**materia**
p1	Joao	matematica
p2	Pedro	ingles
p3	Paula	ingles
p4	Maria	portugues
p5	Rosa	portugues

Alunos		
coda	**nomea**	**materia**
a1	chico	matematica
a2	martha	matematica
a3	lonise	ingles
a4	pedro	ingles
A5	augusto	historia

Solicitaram ao DBA que casasse as necessidades dos alunos com a especialidade dos professores, mas não deram muitos detalhes. O DBA preparou, então, três possibilidades de combinação, representadas pelas consultas SQL (padrão SQL99 ou superior) abaixo:

I. select nomea, nomep from professores **natural join** alunos
II. select nomea, nomep from professores **right join** alunos on professores.materia = alunos.materia
III. select nomea, nomep from professores **full join** alunos using (materia)

Selecione a alternativa que representa, nesta ordem, o número de tuplas resultantes das consultas I, II e III.



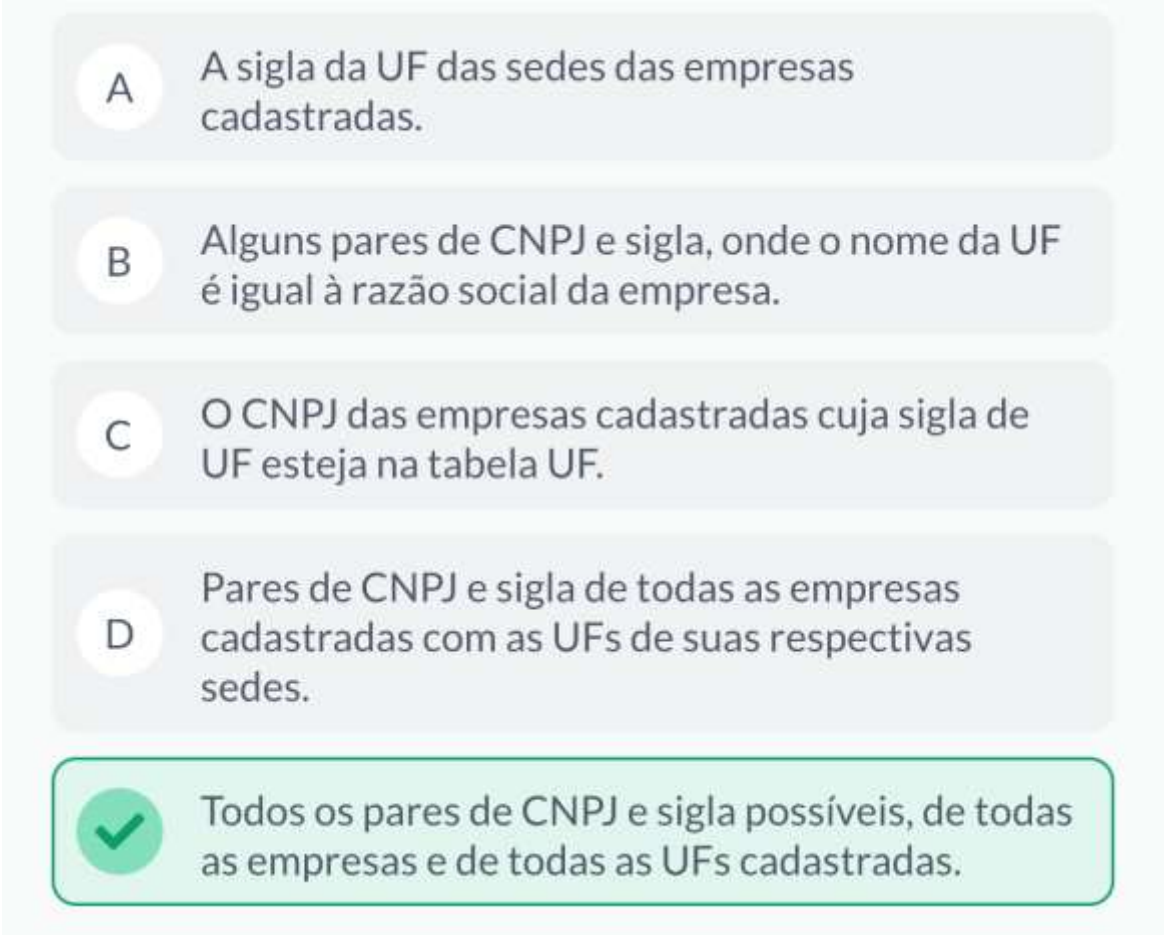
MySQL
- Suporta: natural join
- Não suporta: full join

Exemplo: Produto cartesiano

O banco de dados de uma empresa de investimentos financeiros possui as seguintes tabelas:

- Empresa (CNPJ, razaoSocial, endereco)
- UF (sigla, nome)

O que o comando SQL **"SELECT CNPJ, sigla FROM Empresa, UF"** recupera desse banco de dados?



Exemplo: auto relacionamento

- Considerando as tabelas apresentadas na Figura 2, assinale a alternativa que apresenta um comando SQL que retornará o nome de cada funcionário e o respectivo nome de seu superior.

CARGO			
ID	DESCRICAO	NIVEL	ID_CARGO_SUPERIOR
1	Gerente Administrativo	B-1	
2	Corretor	A-1	1
3	Analista Administrativo	A-2	1
4	Auxiliar Administrativo	A-2	3

FUNCIONARIO			
ID	NOME	ID_CARGO	ID_FUNCIONARIO_SUPERIOR
1	Agatha Souza	1	
2	João Lucas Fogaça	2	1
3	Thales Rocha	2	1
4	Ana Vitoria Ribeiro	2	1
5	Matheus Araújo	3	1
6	Breno Cunha	3	1
7	Cauã da Cruz	4	5
8	Barbara Cardoso	4	6

Figura 2 – Tabelas "Cargo" e "Funcionário"

✓
```
SELECT F.NOME, F2.NOME
FROM FUNCIONARIO F, FUNCIONARIO F2
WHERE F.ID_FUNCIONARIO_SUPERIOR =
F2.ID
```

B
```
SELECT NOME, F2.NOME
FROM FUNCIONARIO, FUNCIONARIO F2
WHERE ID = F2.ID_FUNCIONARIO_SUPERIOR
```

C
```
SELECT FUNC.NOME, SUP.NOME
FROM FUNCIONARIO FUNC, FUNCIONARIO
SUP
WHERE FUNC.ID = SUP.ID
```

D
```
SELECT F1.NOME, F2.NOME
FROM FUNCIONARIO F1, F2
WHERE F.ID_FUNCIONARIO_SUPERIOR IS
F2.ID
```

E
```
SELECT NOME NOME_FUNC, NOME
NOME_SUP
FROM FUNCIONARIO
WHERE ID = ID_FUNCIONARIO_SUPERIOR
```

Exemplo: Join (auto relacionamento)

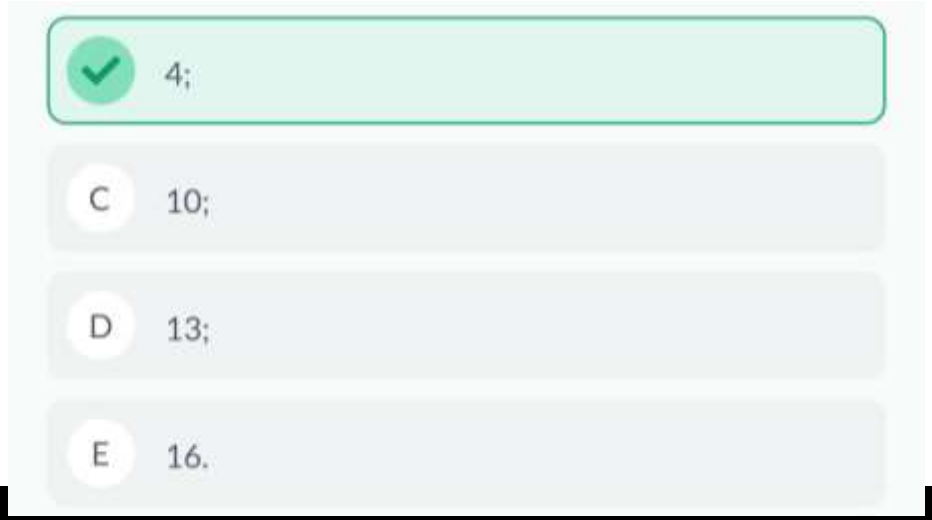
Considerando a tabela T da figura 1, analise a execução do comando SQL a seguir.

- SELECT * FROM **T t1** LEFT JOIN **T t2** ON t1.A = t2.A and t1.B = t2.B and t1.C = t2.C

Além da linha de títulos, o número de linhas produzidas pela execução desse comando é

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL
3	6	5



Exemplo: Produto cartesiano (JOIN)

O banco de dados de uma empresa que comercializa seguros pessoais possui as seguintes tabelas:

- Pessoa (email, nome, unidadeFederativaNascimento, faixaEtaria)
- UF (sigla, nome)
- Faixa (nome, menorIdade, maiorIdade)

A coluna "unidadeFederativaNascimento" da tabela Pessoa é uma chave estrangeira que referencia a coluna "sigla" da tabela UF; a coluna "faixaEtaria" da tabela Pessoa é uma chave estrangeira que aponta para a coluna "nome" da tabela Faixa.

A tabela Faixa possui os seguintes dados:

Tabela: Faixa

nome	menorIdade	maiorIdade
Jovens	- não informada -	19
Adultos	20	59
Idosos	60	- não informada -

Considere o seguinte comando:

- SELECT COUNT(*) FROM Pessoa P, Faixa F WHERE P.faixaEtaria = F.nome AND P.unidadeFederativaNascimento = 'RJ'AND F.maiorIdade <= 19

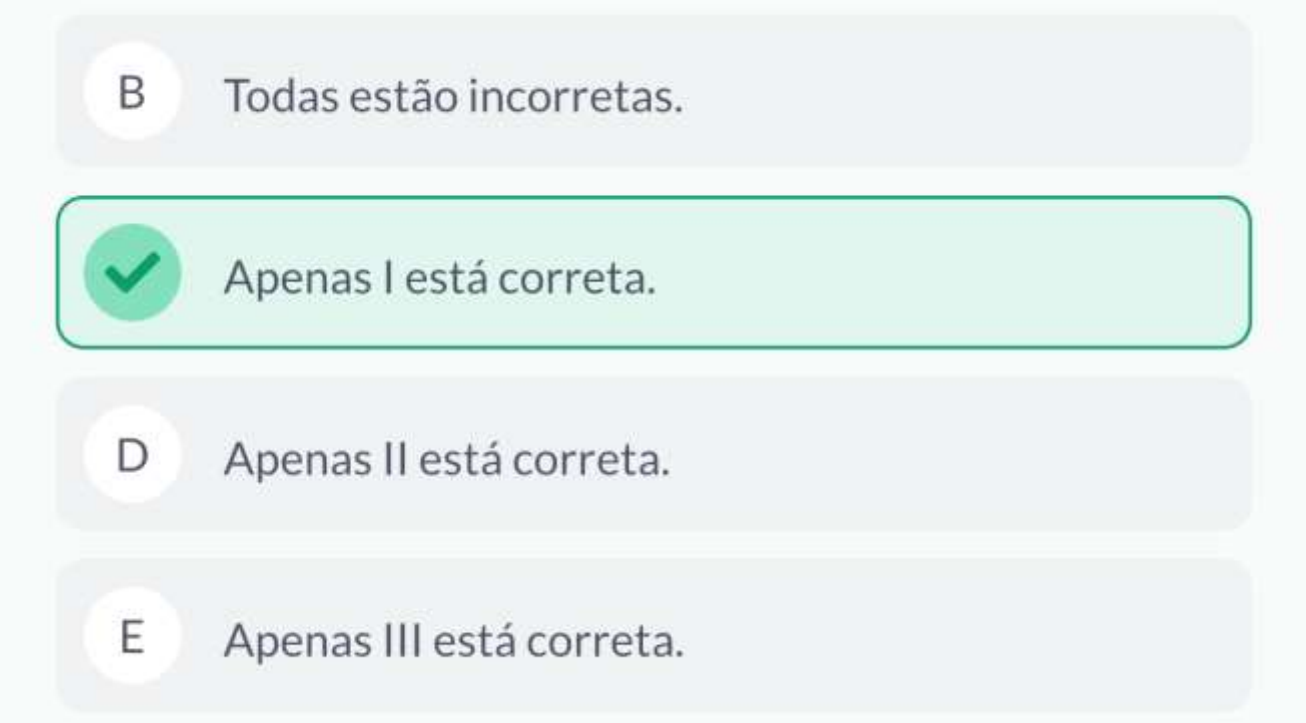
Esse comando SQL :

- ✓ apresenta quantas são as pessoas que estão na tabela Pessoa, que são jovens e que nasceram no estado do Rio de Janeiro.
- B apresenta o nome e o email de jovens nascidos no Rio de Janeiro.
- C agrupa pessoas por faixa etária e mostra quantos são os grupos com pessoas nascidas no Rio de Janeiro.
- D realiza uma operação equivalente à união de dois outros comandos SQL.
- E agrupa pessoas por UF e mostra quantos são os grupos com jovens.

Exemplo: JOIN com colunas nulas

Analise as assertivas abaixo sobre o sistema de banco de dados Microsoft SQL Server 2019 e assinale a alternativa correta.

- I. Durante operações de JOIN, valores nulos (vazios) não são considerados uma combinação.
- II. Os três tipos de JOIN são INNER, OUTER e CROSS.
- III. Colunas usadas em uma condição de JOIN precisam ser do mesmo tipo.



Havendo necessidade de valores nulos serem projetados na resposta, devem ser utilizadas clausulas LEFT ou RIGHT join.

Atividades: Junções

- Dadas as entidades Unidade (cidade: varchar(40)), Produto (descrição: varchar(50), preço: float), Vendedor (nome: varchar(60), matricula: int), Venda (unidade: Unidade; produto: Produto, quantidade: float; vendedor: Vendedor) faça:
  A. Retorne todas as vendas de determinado produto , exibindo detalhes das unidades que realizaram a venda.
  B. Retorne todas as vendas de determinado vendedor, exibindo os detalhes da venda, das unidades, e do vendedor.
  C. Retorne todas as vendas de determinada unidade, realizadas nos últimos 2 anos.
  D. Retorne todos os produtos que não tiveram vendas realizadas no último mês.
  E. Retorne todos os vendedores que não realizaram nenhuma venda nos últimos 20 dias.

SQL – DML - Agregações

```
SELECT
avg(amount),
max(amount),
min(amount) ,
stddev(amount),
count(amount),
sum(amount),
sum(amount)/count(amount),
variance(amount),
sqrt(variance(amount))
FROM sakila.payment
group by customer_id
-- having  avg(amount) > 4
```

JOIN e Group by

Considere as tabelas a seguir em um banco de dados relacional.Qual consulta SQL retorna corretamente o nome do cliente e o valor total dos pedidos, agrupando por cliente e somando o valor total de seus pedidos?

Tabela Clientes

id_cliente	nome	telefone
1	João Silva	1111-1111
2	Maria Souza	2222-2222

Tabela Pedidos

id_pedido	id_cliente	data_pedido	valor_total
1	1	2024-06-20	150.00
2	1	2024-06-21	200.00
3	2	2024-06-22	300.00

A) SELECT nome, valor_totalFROM Clientes JOIN Pedidos ON Clientes.id_cliente = Pedidos.id_clienteGROUP BY nome;

B) SELECT nome, SUM(valor_total) FROM Clientes JOIN Pedidos ON Clientes.id_cliente = Pedidos.id_cliente GROUP BY nome; (gabarito)

C) SELECT nome, SUM(valor_total) FROM Clientes JOIN Pedidos ON Clientes.id_cliente = Pedidos.id_cliente;

D) SELECT nome, COUNT(valor_total) FROM ClientesJOIN Pedidos ON Clientes.id_cliente = Pedidos.id_clienteGROUP BY nome;

E) SELECT nome, valor_totalFROM PedidosGROUP BY nome;

Agregação – Group by

Considerando a SQL:

```
ELECT
    CASE
        WHEN salario > 15000 THEN 'Grupo 1'
        WHEN salario > 10000 THEN 'Grupo 2'
        WHEN salario > 5000 THEN 'Grupo 3'
        ELSE 'Grupo 4'
    END AS grupo_salarial,
    COUNT(*) AS quantidade_funcionarios
ROM Funcionario
ROUP BY grupo_salarial;
```

é possível afirmar que

A) o resultado da consulta gerará uma tabela com quatro colunas.

B) a entidade Funcionário possui os atributos salario e grupo_salarial.

C) o resultado da consulta gerará uma tabela com duas colunas. (gabarito)

D) a entidade Funcionário possui os atributos salario, nome e matricula.

E) o maior salário é maior que quinze mil.

Exemplo:

- Considerando as tabelas apresentadas na Figura 2, qual o comando SQL poderá ser executado para que sejam retornados os nomes das marcas e a quantidade de produtos relacionados a cada marca?

MARCA		
ID	NOME	GRUPO
1	ABC do Sul	A
2	POA Vintage	B
3	Meia Lua	C
4	Cube	A

PRODUTO		
ID	DESCRICAO	ID_MARCA
1	Regata	1
2	Tênis	2
3	Sandália	2
4	Cubo mágico 3x3x3	4
5	Chinelo	2
6	Camiseta manga curta	1

Figura 2 – Tabelas MARCA e PRODUTO

A SELECT M.NOME, SUM(P.DESCRICAO)FROM MARCA M, PRODUTO PWHERE M.ID = P.ID_MARCAHAVING P.DESCRICAO;

✓ SELECT M.NOME, COUNT(P.ID)FROM MARCA M, PRODUTO PWHERE M.ID = P.ID_MARCAGROUP BY M.NOME;

C SELECT M.NOME, SUM(P.ID)FROM MARCA M, PRODUTO PWHERE M.ID = P.ID_MARCAGROUP BY P.DESCRICAO;

D SELECT M.NOME, P.ID_MARCAFROM MARCA M, PRODUTO PWHERE M.ID = P.ID_MARCA;

E SELECT M.NOME, COUNT(P.ID)FROM MARCA M, PRODUTO PWHERE M.ID = P.ID;

Exemplo:

- Considerando a tabela MARCA, apresentada na Figura 2, qual resultado será retornado pelo comando SQL abaixo?

MARCA		
ID	NOME	GRUPO
1	ABC do Sul	A
2	POA Vintage	B
3	Meia Lua	C
4	Cube	A

PRODUTO		
ID	DESCRICAO	ID_MARCA
1	Regata	1
2	Tênis	2
3	Sandália	2
4	Cubo mágico 3x3x3	4
5	Chinelo	2
6	Camiseta manga curta	1

Figura 2 – Tabelas MARCA e PRODUTO

```
SELECT COUNT(DISTINCT GRUPO)
FROM MARCA
```

A 0

B 1

C 2

✓ 3

E 4

Exemplo: HAVING

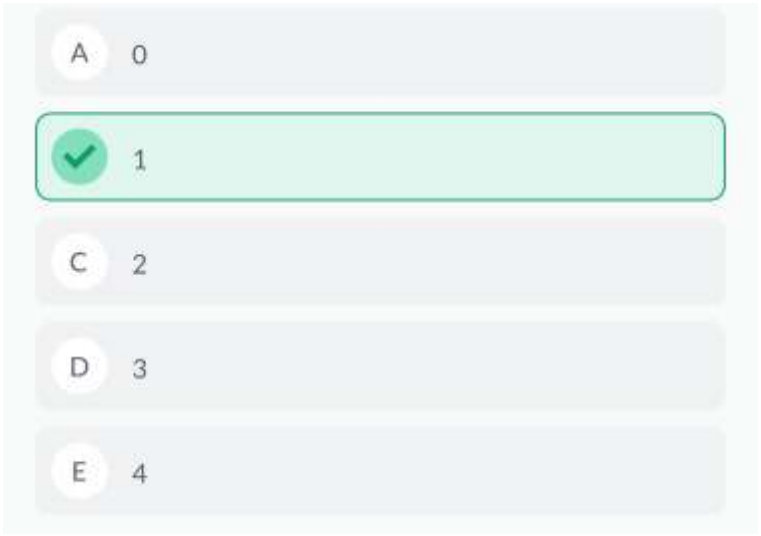
Considerando a tabela "Cargo", apresentada na Figura 2, quantas linhas serão retornadas pelo comando SQL abaixo?

SELECT COUNT(*), NIVEL FROM CARGO **GROUP BY** NIVEL **HAVING** COUNT(*) > 1

CARGO			
ID	DESCRICAO	NIVEL	ID_CARGO_SUPERIOR
1	Gerente Administrativo	B-1	
2	Corretor	A-1	1
3	Analista Administrativo	A-2	1
4	Auxiliar Administrativo	A-2	3

FUNCIONARIO			
ID	NOME	ID_CARGO	ID_FUNCIONARIO_SUPERIOR
1	Agatha Souza	1	
2	João Lucas Fogaça	2	1
3	Thales Rocha	2	1
4	Ana Vitoria Ribeiro	2	1
5	Matheus Araújo	3	1
6	Breno Cunha	3	1
7	Cauã da Cruz	4	5
8	Barbara Cardoso	4	6

Figura 2 – Tabelas "Cargo" e "Funcionário"



Exemplo: Group by

- Qual o resultado correto ao executar o comando SQL abaixo?

```
SELECT ALUNO.Matricula, ALUNO.Nome, CURSO.Nome AS Curso SUM(DISCIPLINA.Carga_horaria) AS "Total de horas" FROM ALUNO,
ALUNO_CURSA, DISCIPLINA, CURSO
 WHERE ALUNO.Matricula = ALUNO_CURSA.Cod_aluno
       AND ALUNO_CURSA.Cod_disciplina = DISCIPLINA.Codigo
       AND DISCIPLINA.Cod_curso = CURSO.Codigo
 GROUP BY ALUNO.Matricula, CURSO.Nome
 ORDER BY ALUNO.Nome;
```

ALUNO

Matricula	Nome	Cod_curso
1001	Carolina Souza	1
1002	Jorge Andrade	1
1003	Cassio Gomes	2

CURSO

Codigo	Nome
1	Ciência da Computação
2	Análise de Sistemas

DISCIPLINA

Codigo	Nome	Cod_curso	Carga_horaria
2001	Lógica de Programação	1	200
2002	Cálculo Numérico	1	180
2003	Programação	2	240

ALUNO_CURSA

Cod_aluno	*Cod_disciplina*
1001	2001
1001	2002
1002	2002
1003	2003

A

Matricula	Nome	Curso	Total de horas
1001	Carolina Souza	Ciência da Computação	380
1002	Jorge Andrade	Ciência da Computação	180
1003	Cassio Gomes	Análise de Sistemas	240

B

Matricula	Nome	Curso	Total de horas
1001	Carolina Souza	Ciência da Computação	560
1003	Cassio Gomes	Análise de Sistemas	240

C

ALUNO.Matricula	ALUNO.Nome	Curso.Nome	Total de horas
1001	Carolina Souza	Ciência da Computação	380
1003	Cassio Gomes	Análise de Sistemas	240
1002	Jorge Andrade	Ciência da Computação	180

D

Matricula	Nome	Curso	Total de horas
1001	Carolina Souza	Ciência da Computação	200
1001	Carolina Souza	Ciência da Computação	180
1003	Cassio Gomes	Análise de Sistemas	240
1002	Jorge Andrade	Ciência da Computação	180

✓

Matricula	Nome	Curso	Total de horas
1001	Carolina Souza	Ciência da Computação	380
1003	Cassio Gomes	Análise de Sistemas	240
1002	Jorge Andrade	Ciência da Computação	180

Junção + Agregação

Um técnico de informática no MPRS recebeu a tarefa de gerar um relatório sobre funcionários que atendem a critérios específicos. O levantamento deve listar os funcionários do MPRS que:

• são naturais de Porto Alegre;

• possuem pelo menos um dependente;

• estão entre os três com os maiores salários;

• se autodeclaram pardos.

Com base nesses critérios, assinale a alternativa que apresenta a consulta SQL (Structured Query Language) correta para atender à solicitação.

A) SELECT f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidadeFROM funcionarios f LEFT JOIN dependentes d ON f.id = d.funcionario_id WHERE f.orgao = 'Ministério Público do Rio Grande do Sul'AND f.naturalidade = 'Porto Alegre' AND f.cor_raca = 'Pardo' ORDER BY f.salario DESCLIMIT 3;

B) BSELECT f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidadeFROM funcionarios f JOIN dependentes d ON f.id = d.funcionario_idHAVING f.orgao = 'Ministério Público do Rio Grande do Sul' AND f.naturalidade = 'Porto Alegre' AND f.cor_raca = 'Pardo' ORDER BY f.salario DESCLIMIT 3;

C) SELECT f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidadeFROM funcionarios f JOIN dependentes d ON f.id = d.funcionario_id WHERE f.orgao = 'Ministério Público do Rio Grande do Sul'AND f.naturalidade = 'Porto Alegre'AND f.cor_raca = 'Pardo' GROUP BY f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidade ORDER BY f.salario DESC LIMIT 3; (gabarito)

D) SELECT f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidade FROM funcionarios fJOIN dependentes d ON f.id = d.funcionario_id WHERE f.orgao = 'Ministério Público do Rio Grande do Sul' OR f.naturalidade = 'Porto Alegre' OR f.cor_raca = 'Pardo' ORDER BY f.salario DESCLIMIT 3;

E) SELECT f.id, f.nome, f.cargo, f.salario, f.cor_raca, f.naturalidade FROM funcionarios f JOIN dependentes d ON f.id = d.funcionario_idWHERE f.orgao != 'Ministério Público do Rio Grande do Sul' AND f.naturalidade != 'Porto Alegre'AND f.cor_raca != 'Pardo' ORDER BY f.salario DESCLIMIT 3;

Atividades: Agregações

- Dadas as entidades Unidade (cidade: varchar(40)), Produto (descrição: varchar(50), preço: float), Vendedor (nome: varchar(60), matricula: int), Venda (unidade: Unidade; produto: Produto, quantidade: float; vendedor: Vendedor) faça:
  A. Retorne a quantidade de vendas de determinado produto e o valor total destas vendas.
  B. Retorne a quantidade de vendas de determinado vendedor, exibindo o valor destas vendas e a média dos valores das vendas.
  C. Retorne todas as unidades que tiveram o valor total de vendas superior a 100.000 no último ano.
  D. Retorne todos os produtos que tiveram total de vendas nos últimos 3 meses inferior
  E. Retorne todos os vendedores que no último ano tiveram valor de vendas acumulado acima da média das vendas de todos os vendedores.

Aula 10

DCL – Segurança no SGBD – Autenticação e Autorização

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Contextualização do Problema

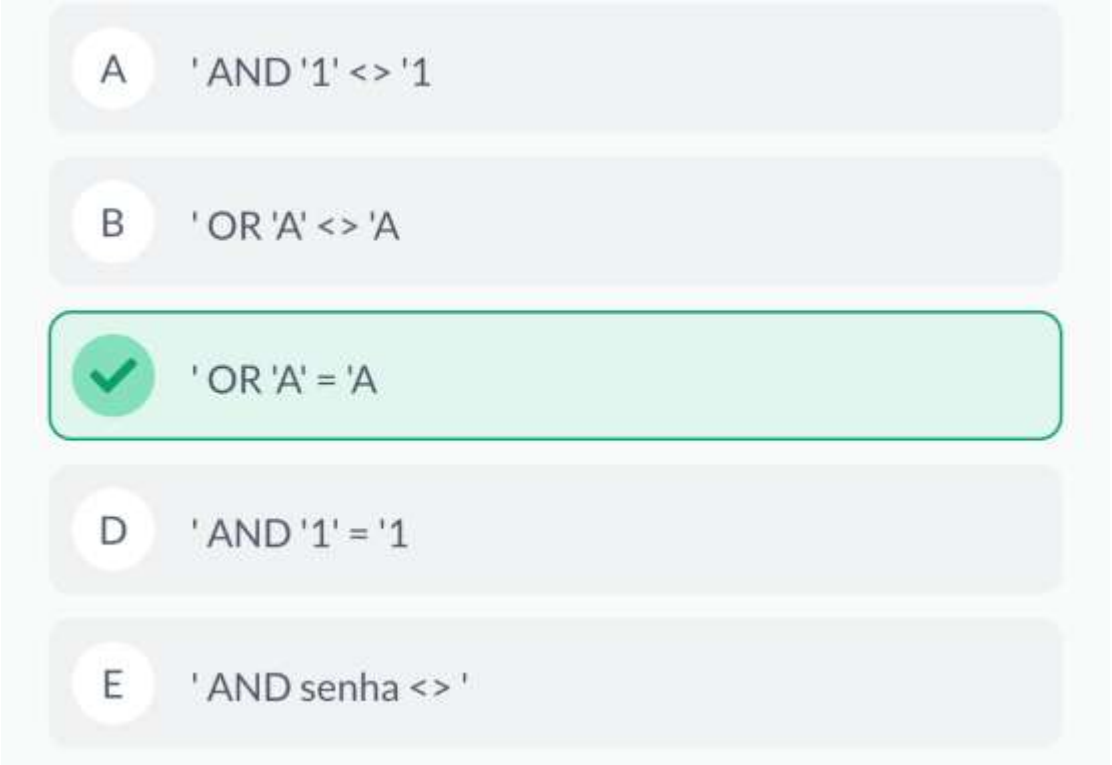
- SQL injection



Fonte: https://www.pfi-security.co.uk/wp-content/uploads/2015/11/SQL-Injection.png

SQL Injection

- Suspeita-se que um sistema web esteja sujeito a ataques do tipo SQL Injection.
- Sua página de acesso possui campos para digitação de login e senha e sabe-se que o script responsável por autenticar o usuário o faz por meio de uma única query do tipo SELECT, acessando a tabela de usuários da aplicação e fazendo uma busca por login e senha em texto claro (sem usar hash ou encriptação).
- Um procedimento que tem o potencial de fornecer acesso ao sistema, sem o conhecimento de um login e senha válidos (considerando que para conseguir acesso a query deva trazer algum resultado), seria digitar o seguinte conteúdo em um dos dois campos (podendo o atacante tentar nos dois):



Contextualização do Problema

Qual seria o impacto de um ataque *SQL injection*, se a aplicação tivesse a finalidade exclusiva de consulta, e um usuário apenas com direito de SELECT nas tabelas que de fato precisam ter seus dados acessados?

Contextualização do Problema

- Usuários uma vez conhecendo as credenciais do banco de dados podem acessá-lo diretamente.

Comum em aplicações desktop!

- Independente das consultas prontas da aplicação, o usuário terá total liberdade no banco de dados.

Boa prática de Segurança

- Boa práticas
  - Não utilizar o usuário root, como usuário de aplicação.
  - Criar um usuário para cada aplicação!
    - Com os direitos de acesso de banco de dados necessários.
  - Troca de senha periódica, atualizando as configurações na aplicação.

DCL

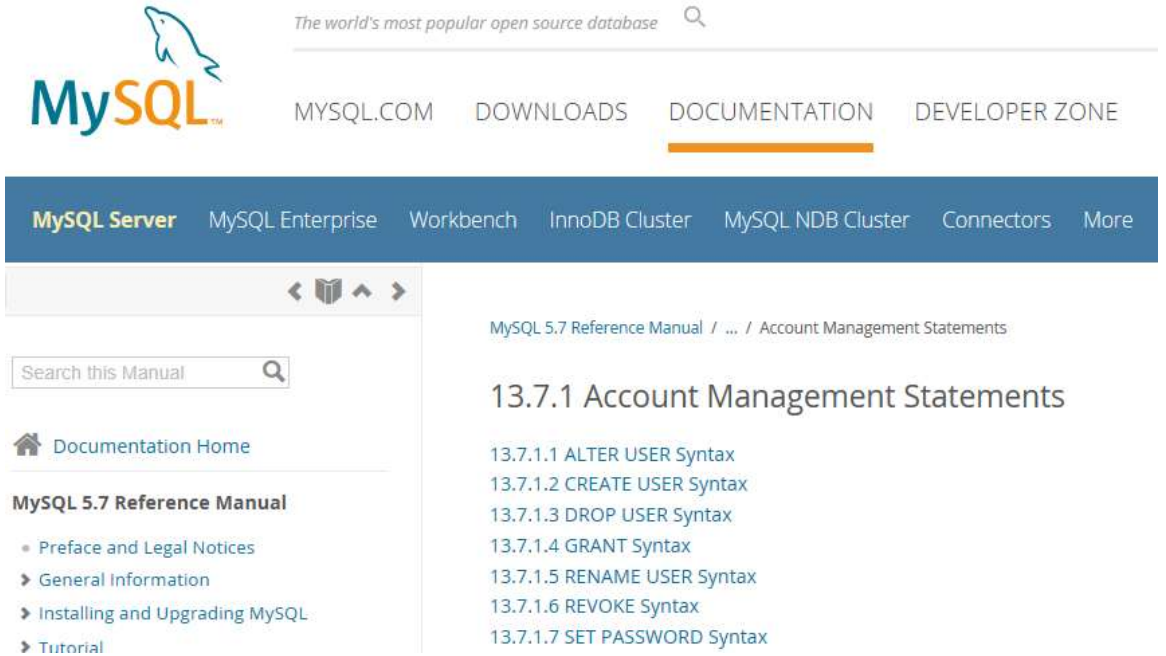
Principais Instruções

- GRANT para permitir que usuários especificados realizem tarefas especificadas.

- REVOKE para cancelar permissões previamente concedidas ou negadas.

MySQL

- Documentação oficial das instruções DCL no MySQL.



https://dev.mysql.com/doc/refman/5.7/en/account-management-sql.html

DCL – Criação de usuários

```
CREATE USER 'jeffrey'@'localhost' IDENTIFIED BY 'password';
```

DCL – Criação de usuários

```
CREATE USER 'jeffrey'@'localhost' IDENTIFIED WITH
sha256_password BY 'new_password' PASSWORD
EXPIRE INTERVAL 180 DAY;
```

DCL – Deleção de usuários

```
DROP USER 'jeffrey'@'localhost';
```

DCL

- Privilégios
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
  - EXECUTE
  - DROP
  - CREATE

Por que gerenciar privilêgios?:

O analista Júnior, recém-chegado ao time de DBAs, recebeu a tarefa de criar um usuário no banco de dados para a estagiária Carla. Carla precisará apenas de permissão para executar consultas SELECT em duas tabelas específicas do esquema de Vendas. Preocupado com a segurança, Júnior deve seguir um princípio fundamental que determina que usuários devem ter apenas os privilégios estritamente necessários para realizar suas tarefas. Qual é esse princípio?

- **A) Princípio do menor privilégio. (gabarito)**
- B) Verificação de integridade referencial.
- C) Criptografia de dados em repouso.
- D) Autenticação de dois fatores.
- E) Isolamento de transação.

Sintaxe básica - GRANT

GRANT *priv_type* [(*column_list*)] ...

ON [*object_type*] ...

TO *user* ...

[WITH GRANT OPTION]

GRANT - Exemplos

```
GRANT ALL ON db1.* TO 'jeffrey'@'localhost';

GRANT SELECT ON db2.invoice TO 'jeffrey'@'localhost'
```

GRANT Exemplos

```
GRANT ALL ON *.* TO 'someuser'@'somehost';
```

```
GRANT SELECT, INSERT ON *.* TO 'someuser'@'somehost';
```

Criando usuário para aplicação:

```
CREATE USER user_name'@'localhost' IDENTIFIED BY 'superhardpass';
GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'user_name '@'localhost';
```

Sintaxe básica - Revoke

REVOKE *priv_type* [(*column_list*)] ...

ON [*object_type*]

FROM *user* [, *user*] ...

Revoke Exemplos

```
REVOKE INSERT ON *.* FROM 'jeffrey'@'localhost';
```

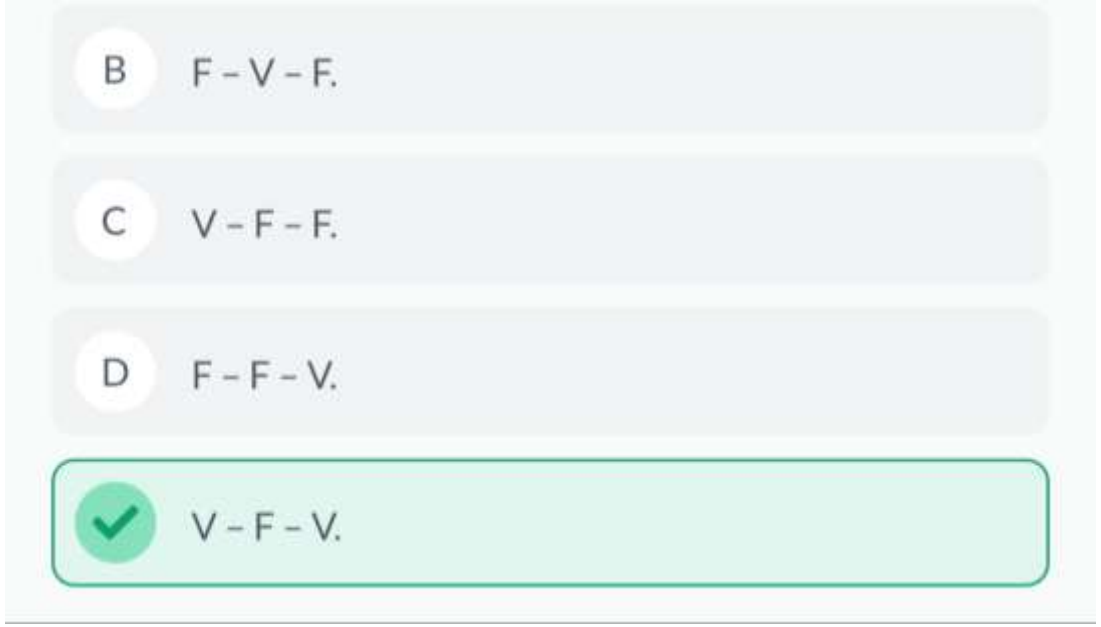
Exemplo - GRANT

Imagine que em um banco de dados ORACLE existe um usuário USUARIO1 e uma tabela de nome PROJETOS. Considere que o DBA emitiu o seguinte comando SQL nesse banco de dados: GRANT ALL PRIVILEGES ON PROJETOS TO USUARIO1;

Analise as assertivas abaixo, sobre os privilégios concedidos a USUARIO1 com esse comando, assinalando V, se verdadeiras, ou F, se falsas.

- ( ) USUARIO1 pode remover tuplas da tabela PROJETOS.
- ( ) USUARIO1 pode conceder a outros usuários um ou mais privilégios recebidos sobre a tabela PROJETOS através de comandos GRANT.
- ( ) USUARIO1 pode definir um gatilho sobre a tabela PROJETOS.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:



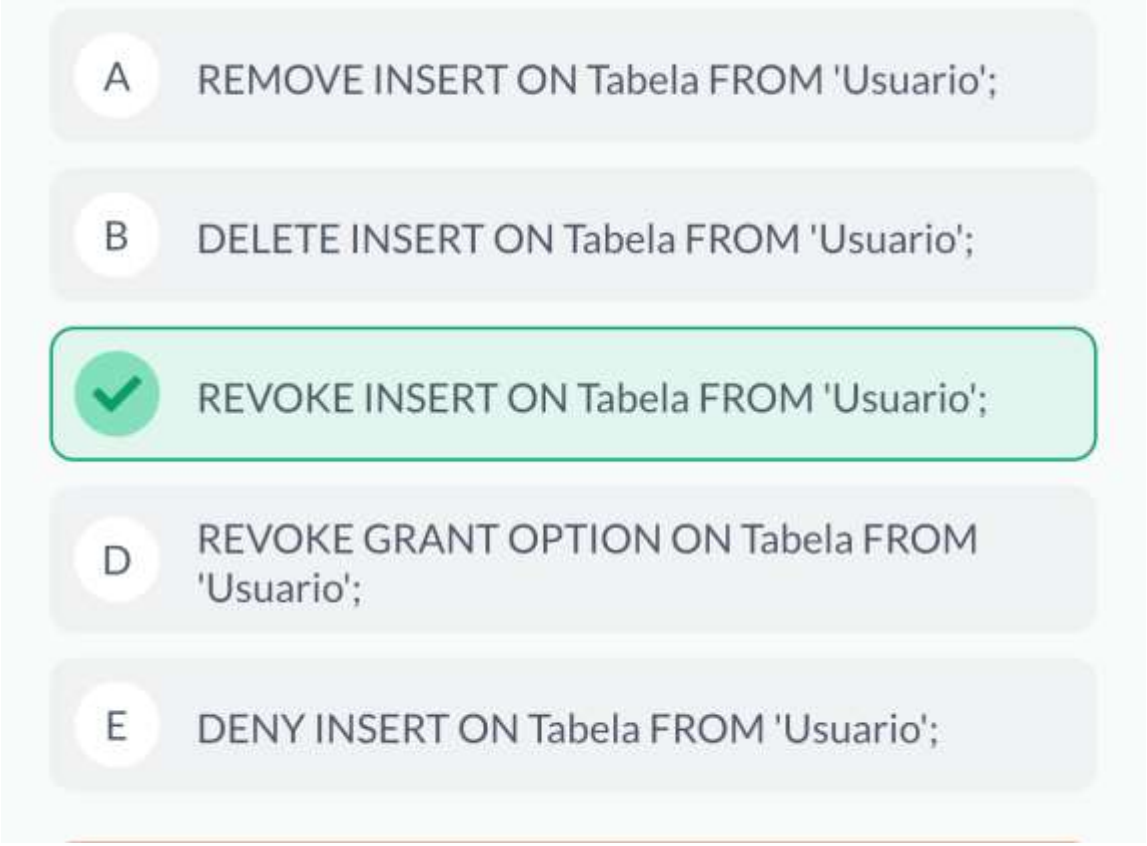
Exemplo - REVOKE

- Qual das declarações SQL abaixo revogaria todos os privilégios de um usuário “usuario1” em uma tabela “tabela1”?



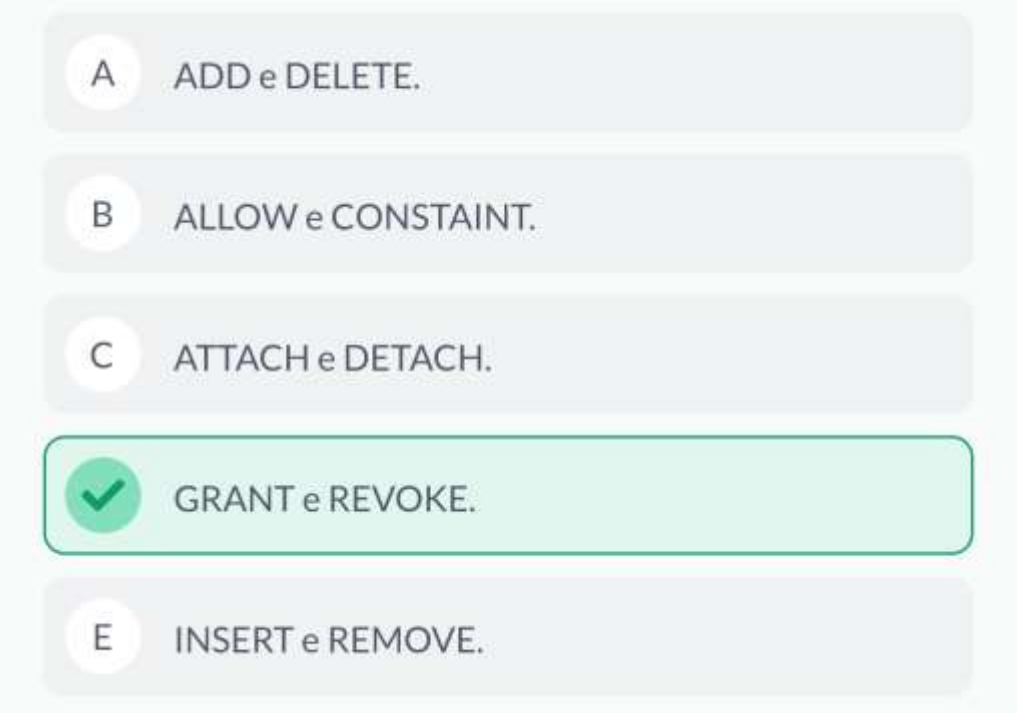
Exemplo - Revoke

- É um comando MySQL 8 que, quando executado, remove a permissão do usuário Usuario de inserir na tabela Tabela?



Exemplo - DCL

- No contexto da concessão de privilégios/permissões na administração de bancos de dados relacionais, assinale a opção que indica dois dos comandos básicos disponíveis na maior parte dos SGDB.



Atividade: DCL

1. Crie um usuário para testar as instruções DCL.

2. Criação de usuário com direto de acesso global para inserção, seleção, e update. (sem create, drop, delete, etc.)

    1.1. Testar um comando de drop table mal sucedido.

    1.2. Testar um comando de insert bem sucedido.

3. Criação de tabela configurações. Proibir operações de drop, insert, updade, ou delete nesta tabela (apenas select).

    2.1. Testar uma operação de consulta bem sucedida.

    2.2. Testar uma operação de inserção mal sucedida.

4. Excluir o usuário criado.

5. Criar view a partir de uma tabela com valores sensíveis (ex. preço) excluindo estas colunas na view. Criar usuário com acesso de select na view e sem acesso algum na tabela.

POSTAR:

1. Instruções DDL para toda estrutura do banco.
2. Instrução DCL dos exercícios 1, 2, e 3.
3. 1 print do teste de cada instrução e de seu resultado. (1.1, 1.2, 2.1, 2.2)

Parte III



Aula 13

Functions e store procedures

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Functions and store procedures

- Componentes de banco de dados capazes de encapsular trechos de código de forma reutilizável.
- Funções possuem tipo e valor de retorno.
- Procedures não possuem retorno.

Exemplo procedure

```
DELIMITER $$
CREATE PROCEDURE seleciona_produtos(IN quantidade INT)
BEGIN
    SELECT * FROM PRODUTO LIMIT quantidade;
END $$
DELIMITER ;
CALL seleciona_produtos(2)
```

Exemplo procedure

```
DELIMITER $$
CREATE PROCEDURE verifica_quantidade_produtos (OUT quantidade INT)
BEGIN
SELECT COUNT(*) INTO quantidade FROM PRODUTO;
END $$
DELIMITER ;
call verifica_quantidade_produtos(@qtdProd);
select @qtdProd
```

Functions and store procedures

Exemplo de procedure:

```
delimiter //
CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities INT)
BEGIN
     SELECT COUNT(*) INTO cities FROM world.city
     WHERE CountryCode = country;
END//
delimiter ;
```

```
CALL citycount('JPN', @cities); -- cities in Japan
```

Exemplo procedure

```
DELIMITER $$
CREATE PROCEDURE quadrado(INOUT numero INT)
BEGIN
    SET numero = numero * numero;
END $$
DELIMITER ;

SET @valor = 5;
CALL quadrado(@valor);
SELECT @valor;
```

Exemplo IF

```
DELIMITER $$
CREATE PROCEDURE modulo(INOUT numero INT)
BEGIN

    IF numero < 0 THEN
        SET numero = numero * -1;
    ELSE
        SET numero = numero;
    END IF;
END $$
DELIMITER ;
SET @valor = -2;
CALL modulo(@valor);
SELECT @valor;

SET @valor = 2;
CALL modulo(@valor);
SELECT @valor;
```

Procedures

```
SHOW PROCEDURE STATUS;

DROP PROCEDURE <name>;
```

Procedures podem ser úteis para ser scripts de automatização de rotinas ou para ser invocados por programas.

Functions and store procedures

Exemplo de função

```
CREATE FUNCTION hello (s CHAR(20)) RETURNS CHAR(50) DETERMINISTIC
RETURN CONCAT('Hello, ',s,'!');
SELECT hello('world');
```

Deterministic: possibilita otimizações (cache de resultados)

Function

```
DELIMITER $$
CREATE FUNCTION customer_level( credit DECIMAL(10,2) ) RETURNS
VARCHAR(20) DETERMINISTIC
BEGIN
    DECLARE customerLevel VARCHAR(20);

    IF credit > 50000 THEN
        SET customerLevel = 'PLATINUM';
    ELSEIF (credit >= 50000 AND  credit <= 10000) THEN
        SET customerLevel = 'GOLD';
    ELSEIF credit < 10000 THEN
        SET customerLevel = 'SILVER';
    END IF;
    RETURN (customerLevel); -- return the customer level
END$$
DELIMITER ;
SET @level = customer_level(80000);
select @level
```

Function – while example

```
DELIMITER $$
CREATE FUNCTION fatorial(
    num INT
) RETURNS INT DETERMINISTIC
BEGIN
    DECLARE counter INT DEFAULT num;
    WHILE num > 1 DO
        SET num = num -1;
        SET counter = counter * num;
    END WHILE;
    RETURN counter;
END$$
DELIMITER ;
select fatorial(5)
```

Function

```
SHOW FUNCTION STATUS
```

```
SELECT
    customerName,
    CustomerLevel(creditLimit)
FROM
    customers
ORDER BY
    customerName;
```

customerName	CustomerLevel(creditLimit)
Alpha Cognac	PLATINUM
American Souvenirs Inc	SILVER
Amica Models & Co.	PLATINUM
ANG Resellers	SILVER
Anna's Decorations, Ltd	PLATINUM
Anton Designs, Ltd.	SILVER
Asian Shopping Network, Co	SILVER
Asian Treasures, Inc.	SILVER
Atelier graphique	GOLD
Australian Collectables, Ltd	PLATINUM
Australian Collectors, Co.	PLATINUM

```
DELIMITER $$

CREATE PROCEDURE GetCustomerLevel(
    IN  customerNo INT,
    OUT customerLevel VARCHAR(20)
)
BEGIN

        DECLARE credit DEC(10,2) DEFAULT 0;

    -- get credit limit of a customer
    SELECT
                creditLimit
        INTO credit
    FROM customers
    WHERE
                customerNumber = customerNo;

    -- call the function
    SET customerLevel = CustomerLevel(credit);
END$$

DELIMITER ;
```

Uma function pode ser chamada em um select ou dentro de outra function ou procedure.

Usuário com acesso exclusivo à Procedure

```
DELIMITER $$
CREATE PROCEDURE  valorTotalPagoCliente (IN id_cliente INT, OUT valor DECIMAL)

BEGIN
	SELECT sum(amount) INTO valor FROM payment where customer_id = id_cliente;
END $$
DELIMITER ;

CREATE USER 'jeffrey'@'localhost' IDENTIFIED BY '123’;
GRANT EXECUTE ON PROCEDURE sakila.valorTotalPagoCliente TO 'jeffrey'@'localhost'
```

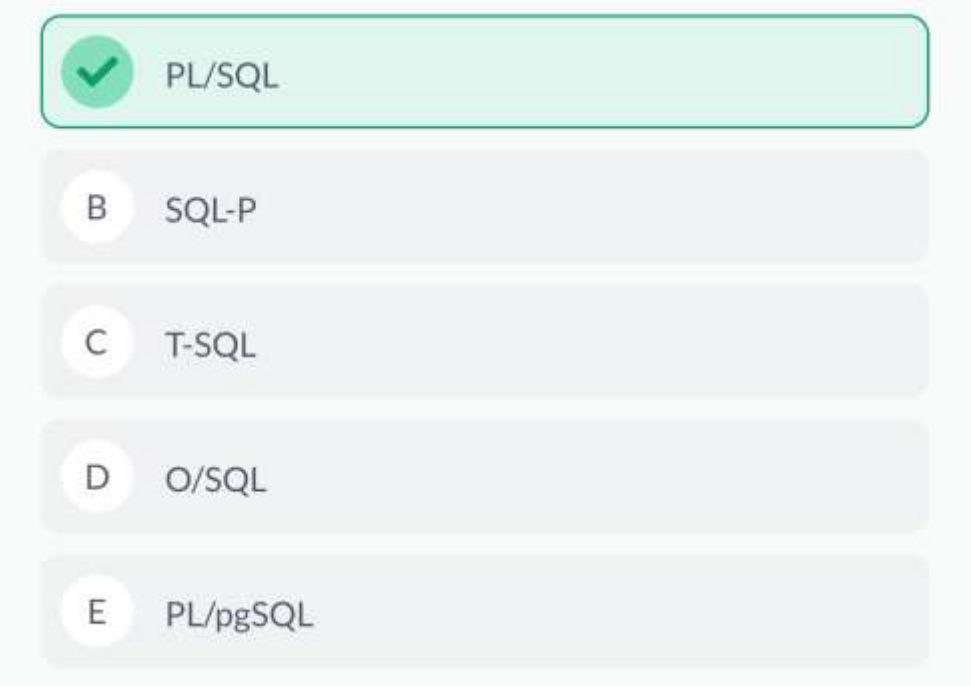
Linguagem utilizada

- MySQL stored procedures are written in the **MySQL stored program language**, which is ANSI SQL:2003 compliant. It has a block structure similar to Oracle PL/SQL and IBM DB2 SQL.

Source: https://www.databasedesign-resource.com/mysql-stored-procedures.html

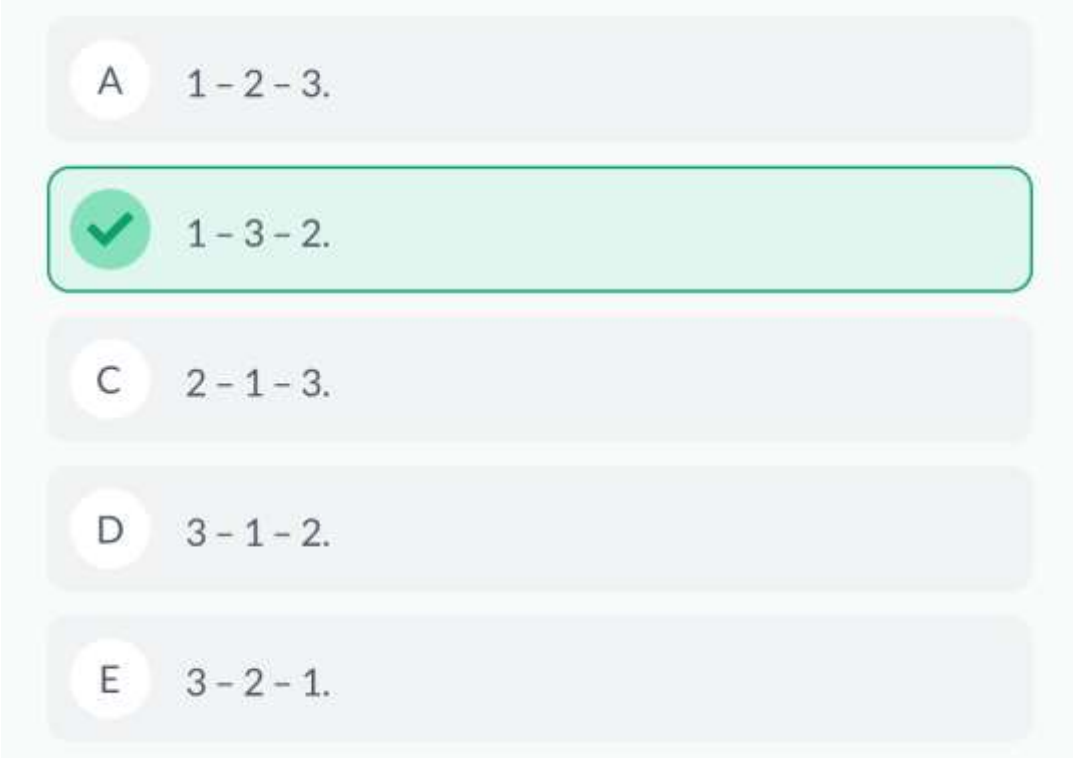
Exemplo - Linguagem

- A _____ é uma linguagem procedural disponível, por padrão, em SGBDs Oracle. Assinale a alternativa que preenche corretamente a lacuna do trecho acima



Exemplo

- Relacione a Coluna 1 à Coluna 2, associando os SGDBs aos tipos de extensões SQL utilizadas por esses SGBDs.
- Coluna 1
- 1. Oracle
- 2. PostgreSQL
- 3. SQL Server
- Coluna 2
- ( ) PL/SQL
- ( ) T-SQL
- ( ) PL/pgSQL
- A ordem correta de preenchimento dos parênteses, de cima para baixo, é:



Functions and store procedures

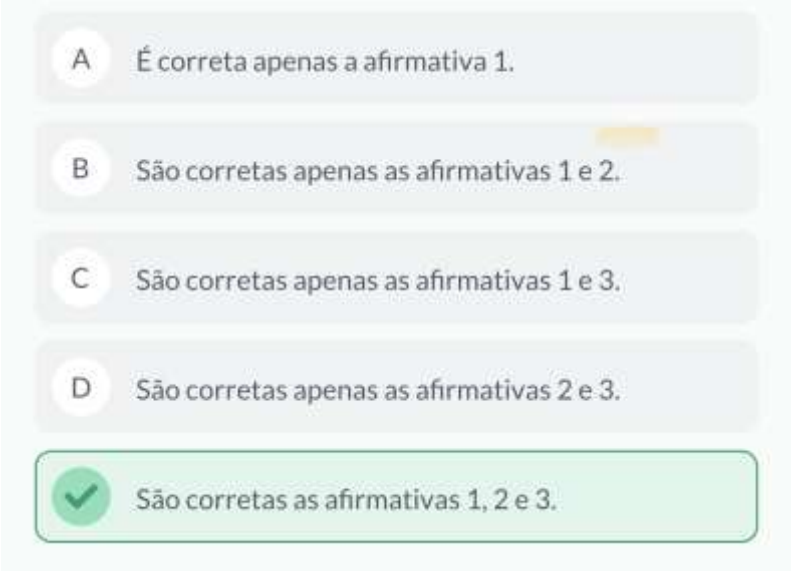
É comum, em um sistema de informação que utiliza banco de dados relacional, como por exemplo o MySQL, executar diferentes comandos SQL em sequência para obter um resultado.A depender do que será realizado, muitos SQL em sequência podem ser executados em módulos diferentes do mesmo sistema. Para estimular a reutilização de código, tornar a programação mais simples, além de compartilhar com os servidores de banco de dados o uso de recursos computacionais para executar os comandos SQL, o SGBD permite criar objetos chamados Stored Procedure.
Os SGBDs possuem regras estabelecidas para a criação de Stored Procedure. Ao utilizar o MySQL, analise as afirmativas a seguir.

I. Na construção de uma Stored Procedure, o MySQL permite o uso do comando RETURN para retornar ao programa principal o valor do dado tratado.
II. No MySQL, os parâmetros recebidos por uma Stored Procedure podem ser do tipo IN, que informa que somente aceita dados de entrada, OUT, que informa que o parâmetro é utilizado apenas para retorno, ou INOUT, em que o parâmetro é utilizado para entrada e retorno.
III. No MySQL, uma Stored Procedure é finalizada ao utilizar o comando END, uma vez que o sinal de ponto e vírgula delimita o fim da instrução SQL.
Está correto o que se afirma em:
a) II e III b) III apenas c) I e II d) I e III **e) II apenas**

Exemplo:

Analise as afirmativas abaixo sobre as funções de tabela em PL/SQL.

- 1. São usadas para retornar coleções PL/SQL que simulam ou se comportam como tabelas.
- 2. Podem ser consultadas como uma tabela regular, utilizando o operador de tabela TABLE da cláusula FROM.
- 3. Funções de tabela regulares requerem que coleções sejam totalmente populadas antes que sejam retornadas. Assinale a alternativa que indica todas as afirmativas correta



Exemplo: functions para default values

- CURRENT_TIMESTAMP();

```
CREATE TABLE Orders (
    ID int NOT NULL,
    OrderNumber int NOT NULL,
    OrderDate date DEFAULT CURRENT_DATE()
);
```

Atividades: Functions e procedures

- Considerando a entidade cliente (data_registro:date_time, nome: varchar(100), email: varchar(50)) faça:
  a) Uma procedure para registrar novo cliente recebendo o valor de cada um de seus atributos como parâmetro.
  b) Uma procedure para atualizar o email de determinado cliente.
  c) Uma prodedure para deletar todos os clientes registrados antes de determinada data passada por parâmetro.
  d) Uma function para retornar a quantidade de clientes registrados nos últimos 30 dias.
  e) Uma function para retornar a quantidade de clientes registrados no ano corrente.

Aula 14

Views e Triggers

Prof. Dr. Rafael Queiroz Gonçalves, PMP

Views

O que são views?

São resultados de consultas (SELECT) que podem ser facilmente acessados por um único nome (sendo este o nome da view).

Como a view pode ser utilizada?

Com base no nome da view é possível realizar novas operações de consulta, como filtros, projeção, ordenação, agrupamento, etc.

Views

Posso inserir registros em uma view?

Apenas para views compostas por uma única tabela, que contenha as colunas mínimas necessárias para inserção de registro.

Quais os benefícios do uso de uma view?

- **Segurança:** não fornecer dados a todas as colunas de uma tabela.
- **Performance:** devolver apenas o conjunto mínimo de atributos necessários para determinada funcionalidade.

Views

Sintaxe básica:

- CREATE VIEW test.v AS SELECT * FROM t;

View example

```sql
CREATE VIEW brazilan_customers AS
SELECT CustomerName, ContactName
FROM Customers
WHERE Country = 'Brazil';
```

```sql
CREATE VIEW products_above_average_price AS
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

Exemplo:

Considere a tabela PILOTOS e a visão PILOTOSA320 definidas pelos comandos SQL abaixo (padrão SQL99 ou superior).

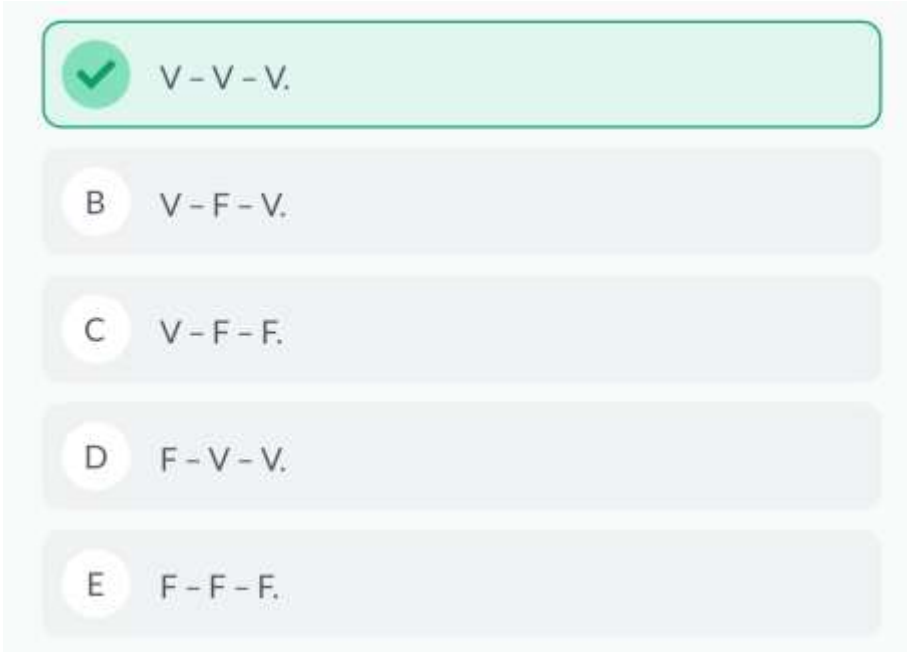
```
create table PILOTOS
(codp int not null,
nomep varchar(200) not null,
salario float,
companhia varchar(200),
aviao char(5) not null default '777',
primary key(codp));
```

```
create view PILOTOSA320
as (select codp, nomep, companhia, aviao
   from PILOTOS
   where aviao = 'A320');
```

Considere os comandos SQL abaixo no mesmo padrão, em que cada comando corresponde a uma transação. Assinale com V, se o comando executa corretamente, ou com F, se o comando resulta em erro.

- ( ) insert into PILOTOSA320(codp, nomep, companhia, aviao) values (1, 'jose', 'gol', 'A320')
- ( ) insert into PILOTOSA320(codp, nomep, companhia, aviao) values (2, 'maria', 'tam', '777');
- ( ) insert into PILOTOSA320(codp, nomep, aviao) values (2, 'jose', 'A320');

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:



Atividades: Views

1) Criar view mesclando dados de duas tabelas por meio de JOIN.
   - Tentar inserir registros e observar o resultado.
2) Criar view flat: utilizando dados de apenas uma tabela.
   - Tentar inserir registros e observar o resultado.
3) Projeção de colunas em view.
   - Selecionar um subconjunto de atributos em uma view.
4) Aplicar group by em view, utilizando filtros having.
5) Criar view utilizando alias para o nome das colunas.
   - Tentar selecionar registros utilizando nome original das colunas e observar o resultado.

6) Em uma tabela de prestadores de serviços (nome, email, valor mensal, descrição), criar uma view que não contenha a coluna valor. Criar um usuário chamado desenvolvedor e atribuir acesso de leitura apenas na view, e remover acessos de leitura na tabela.
   - Tente acessar a tabela e observe o resultado.
   - Tenta acessar a view e observe o resultado.

Trigger

- Execução de código (instruções) automatizada a ser executada imediatamente antes (BEFORE) ou depois (AFTER) da execução de determinado evento.
- Eventos incluem instruções de inserção ou modificação de dados como: INSERT, UPDATE e DELETE.
- No corpo da instrução a ser executada é possível para cada registro afetado conhecer os valores dos atributos envolvidos por meio de variáveis OLD e NEW.

Trigger

- SHOW TRIGGERS
- DROP TRIGGER <trigger_name>

Trigger	Event	Table	Statement	Timing	Created	sql_mode
customer_create_date	INSERT	customer	SET NEW.create_date = NOW()	BEFORE	2019-12-27 20:49:11.40	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...
ins_film	INSERT	film	BEGIN INSERT INTO film_text (film_id, title, d...	AFTER	2019-12-27 20:49:10.12	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...
upd_film	UPDATE	film	BEGIN IF (old.title != new.title) OR (old.desc...	AFTER	2019-12-27 20:49:10.14	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...
del_film	DELETE	film	BEGIN DELETE FROM film_text WHERE film_id...	AFTER	2019-12-27 20:49:10.16	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...
payment_date	INSERT	payment	SET NEW.payment_date = NOW()	BEFORE	2019-12-27 20:49:12.54	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...
rental_date	INSERT	rental	SET NEW.rental_date = NOW()	BEFORE	2019-12-27 20:49:13.33	STRICT_TRANS_TABLES,STRICT_ALL_TABLES,...

Trigger – Estrutura do código

```
CREATE TRIGGER <nome> momento evento
ON <tabela>
FOR EACH ROW
BEGIN
/*corpo do código*/
END
```

```
CREATE TRIGGER <nome> [BEFORE | AFTER] [INSERT | UPDATE | DELETE]
ON <tabela>
FOR EACH ROW
BEGIN
/*corpo do código*/
END
```

Trigger – INSERT - Exemplo

Preencher ou calcular campos automaticamente.

```
DELIMITER $
CREATE TRIGGER city_create_date before insert
ON city
FOR EACH ROW
BEGIN
    SET NEW.create_date = now();
END$
DELIMITER ;
```

Trigger – DELETE - Exemplo

Bloquear deleções ao não atender alguma condição lógica.

```
DELIMITER $
CREATE TRIGGER city_control_deletions
BEFORE DELETE ON city
FOR EACH ROW
    IF OLD.create_date is not NULL AND year(OLD.create_date) >= 2023 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'It is not allowed to delete entries
created after 2023';
    END IF;
END$
DELIMITER ;
```

Trigger – INSERT - Exemplo

```
CREATE TRIGGER no_duplicate_last_name
BEFORE INSERT ON actor
FOR EACH ROW
BEGIN
  DECLARE nLastName INT;
  select count(*) into nLastName from actor where last_name=NEW.last_name;
  IF nLastName > 0  THEN
   SIGNAL SQLSTATE '02000' SET MESSAGE_TEXT = 'It is not allowed duplicated last names!';
  END IF;
  SET NEW.last_update = now();
  SET NEW.first_name = ucase(NEW.first_name);
  SET NEW.last_name = ucase(NEW.last_name);
END$$
DELIMITER ;
```

Trigger – more examples

```
DELIMITER $

CREATE TRIGGER Tgr_ItensVenda_Insert AFTER INSERT
ON ItensVenda
FOR EACH ROW
BEGIN
  UPDATE Produtos SET Estoque = Estoque - NEW.Quantidade
WHERE Referencia = NEW.Produto;
END$

CREATE TRIGGER Tgr_ItensVenda_Delete AFTER DELETE
ON ItensVenda
FOR EACH ROW
BEGIN
  UPDATE Produtos SET Estoque = Estoque + OLD.Quantidade
WHERE Referencia = OLD.Produto;
END$

DELIMITER ;
```

Exemplo Trigger

Considere as tabelas criadas com os comandos SQL abaixo, e os comandos de inserção de tuplas SQL (padrão SQL99 ou superior).

- Create table A (pka int primary key, a1 integer);
- Create table B (pkb int primary key, b1 integer);
- Create table C (pkc int primary key, c1 integer);
- insert into A values (1,10);
- insert into A values (2, 10);
- insert into A values (3, 2);
- insert into A values (4, 3);
- insert into B values (10,0);
- insert into C values (100,0);

Suponha a criação dos gatilhos T1 e T2 definidos abaixo cujas cláusulas estão de acordo com padrão, e cujo código disparado foi escrito conforme sintaxe PL/SQL (Oracle).

Create Trigger T1	Create Trigger T2
Before delete on A begin Update B set b1 = b1 + 1; end;	After delete on A for each row begin Update C set c1 = c1 + 1; end;

Considere que os três comandos SQL de remoção de tuplas abaixo foram executados sem erro:

I. delete from A where a1>=10;

II. delete from A where a1<10;

III. delete from A where a1 is null;

Finalmente, foram executadas as duas consultas SQL abaixo, cada qual retornando uma única tupla:

- (1) SELECT b1 FROM B;
- (2) SELECT c1 FROM C;

Selecione a alternativa que representa, nesta ordem, o valor do atributo b1 resultante da consulta (1), e o valor do atributo c1 resultante da consulta (2).

B 2 – 3.

C 2 – 4.

D 3 – 2.

✓ 3 – 4.

Trigger Storms

Em um sistema de gerenciamento de banco de dados relacional (SGBD), triggers são mecanismos que executam ações automáticas em resposta a eventos como inserções, atualizações ou exclusões em tabelas. Quando triggers são mal projetados, especialmente em cenários que envolvem múltiplas tabelas, eles podem resultar em "cascading triggers" ou "trigger storms", um comportamento em que um trigger dispara outro que, por sua vez, dispara mais triggers, potencialmente levando a loops infinitos, desempenho degradado ou dificuldades de depuração. Considere um SGBD que permite triggers aninhados sem limite estrito de profundidade. Avalie as seguintes afirmações sobre cenários de definição de triggers em um banco de dados relacional e assinale aquela que descreve um cenário de cascading triggers ou trigger storms:

A) Um trigger BEFORE UPDATE em uma tabela que impede a modificação de uma chave primária.

B) Um trigger BEFORE INSERT em uma tabela que atualiza um campo na mesma tabela.

C) Um trigger AFTER INSERT em uma tabela A que causa uma inserção em uma tabela B que, por sua vez, tem um trigger AFTER INSERT que atualiza a tabela A. (gabarito)

D) Um trigger AFTER UPDATE em uma tabela que insere um registro em uma tabela de auditoria relacionada.

E) Um trigger AFTER DELETE em uma tabela que atualiza uma estatística agregada em outra tabela.

Atividades: Triggers

Dada uma tabela Cliente (id, nome, email, telefone, last_update_on)

a) Na atualização preencher automaticamente o “last update”.

b) Na atualização, deleção, ou inserção criar uma tabela de log das mudanças: ação, quando, registro afetado.

c) Na atualização logar em uma tabela de auditoria todos os campos que foram alterados (apenas os campos alterados: valor antigo e o valor novo)

d) Na deleção – não possibilitar registros a deleção de registros que já tenham histórico de atualizações realizadas.

Materialized view

Determinado analista de sistema da Câmara Municipal está otimizando uma consulta SQL para gerar um relatório de solicitações processadas por departamento. A tabela Solicitações possui os seguintes campos:

* id_solicitacao (chave primária) * id_departamento (chave estrangeira)* data_solicitacao * status ('pendente', 'em andamento', 'concluída')

A consulta a seguir foi implementada para contar o número de solicitações concluídas por departamento:

SELECT id_departamento, COUNT(*) AS total_concluidas FROM Solicitações WHERE status = 'concluída' GROUP BY id_departamento;

A equipe identificou que a consulta está impactando o desempenho do banco de dados quando acessada simultaneamente por múltiplos usuários. Considerando o impacto causado por acessos concorrentes a uma consulta de leitura com agregação, qual das estratégias a seguir representa a solução mais eficaz para otimizar o desempenho e reduzir a carga sobre o banco de dados?

A) Criar uma indexação no campo status para acelerar a filtragem das solicitações concluídas.

B)Substituir a consulta por uma visão materializada, armazenando os resultados pré-calculados para reduzir a carga computacional da consulta. (gabarito)

C) Utilizar o nível de isolamento SERIALIZABLE, garantindo que cada transação leia os dados sem interferência de outras, mesmo que isso reduza a concorrência.

D)Implementar um bloqueio exclusivo (LOCK) na tabela Solicitações durante a execução da consulta para evitar inconsistências, impedindo leituras concorrentes.

Materialized view: the query result is cached as a concrete ("materialized") table (rather than a view as such) that may be updated from the original base tables from time to time. This enables much more efficient access, at the cost of extra storage and of some data being potentially out-of-date.

MySQL doesn't natively support materialized views. They are not a built-in feature, unlike regular views.

Aula 15

Sequences

Conectando em SGBD por aplicação - JAVA

Prof. Dr. Rafael Queiroz Gonçalves, PMP

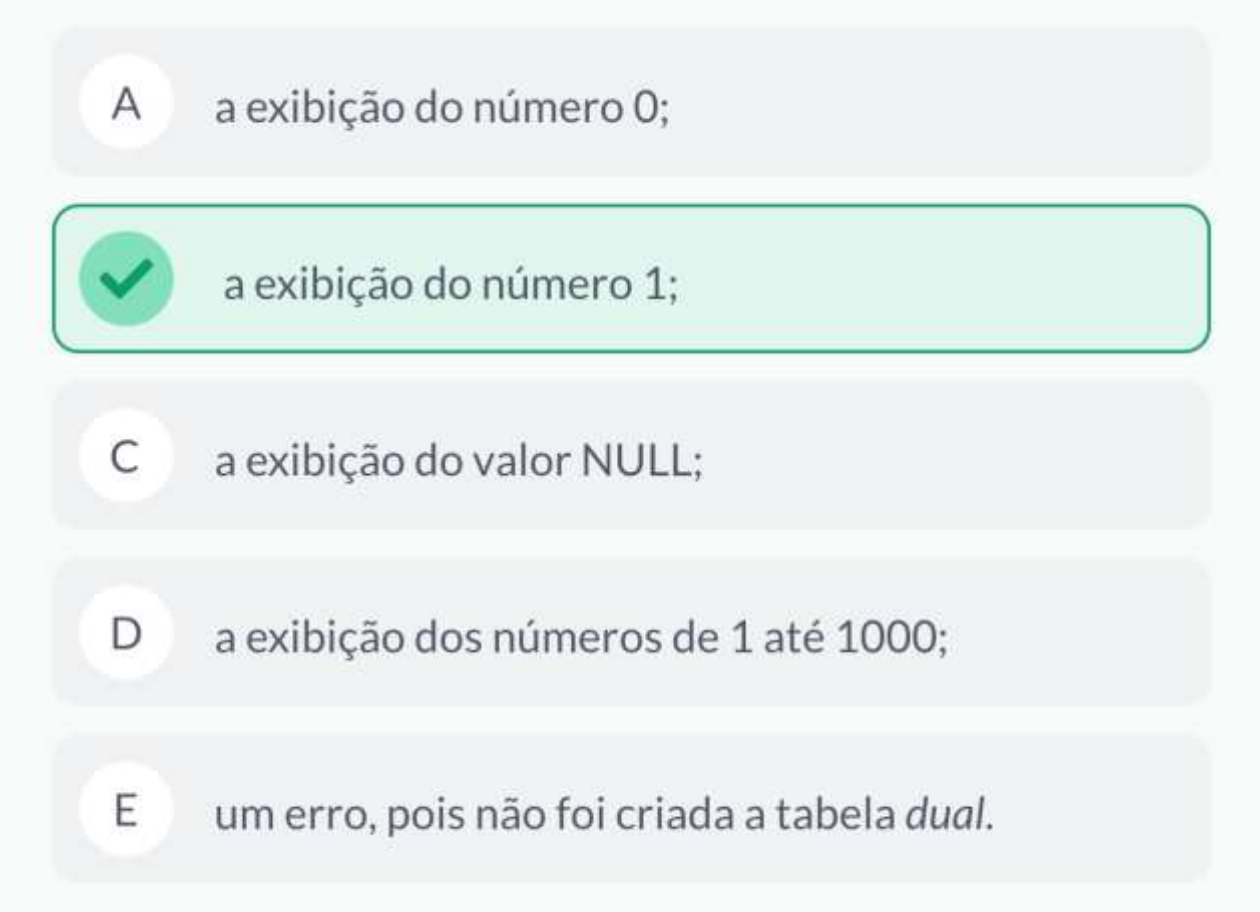
Sequence

Numa instalação Oracle, considere o script a seguir.

create sequence xxxx;

select xxxx.nextval from dual;

A execução desse script provoca:



Driver de conexão com SGBD

Todo SGDB fornece um driver de conexão para a linguagem de programação estabelecer a conexão e intercambiar dados..

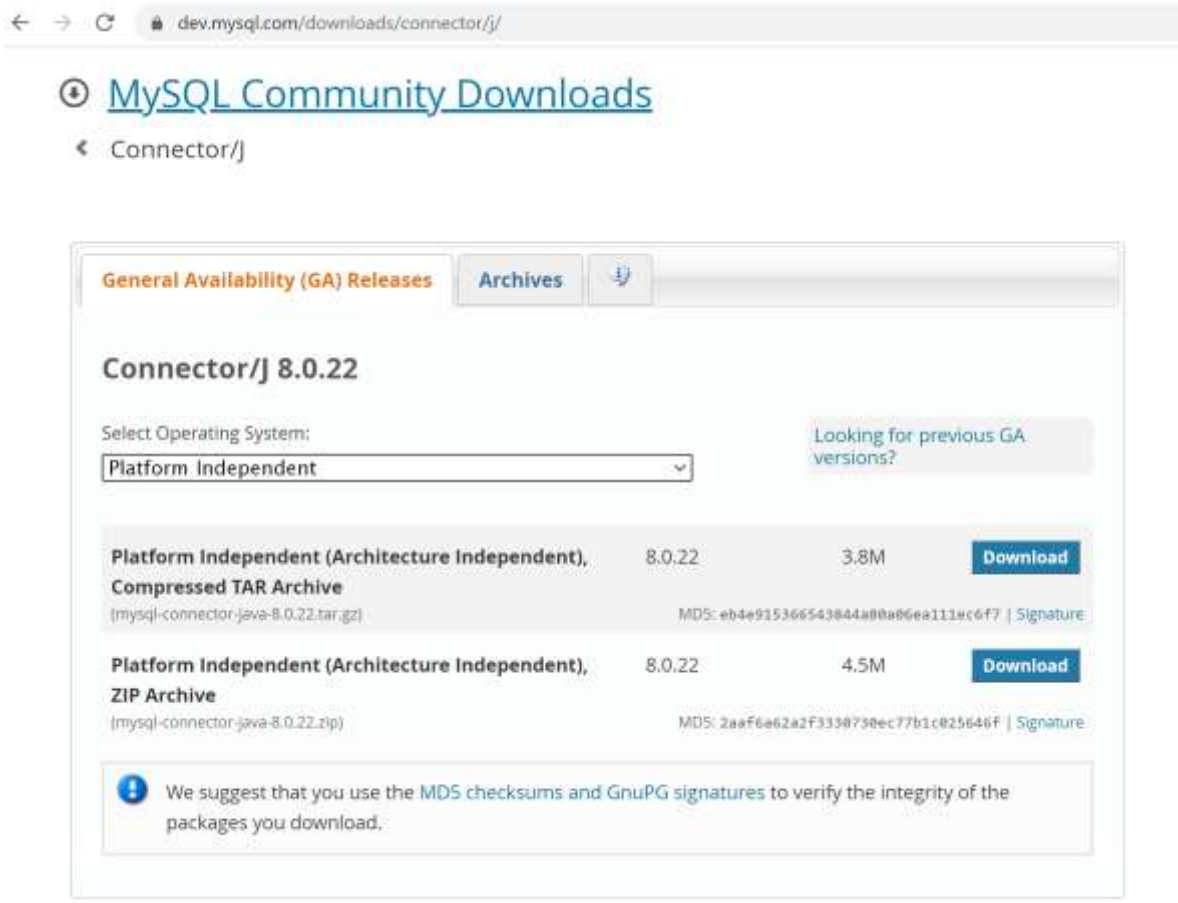
O Driver é um arquivo .jar (biblioteca) que precisa ser incluida no projeto.

Exemplo de inclusão do driver do MySQL via maven.

```
 <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.22</version>
</dependency>
```

Driver de conexão com SGBD

https://dev.mysql.com/downloads/connector/j/



Estabelecendo conexão

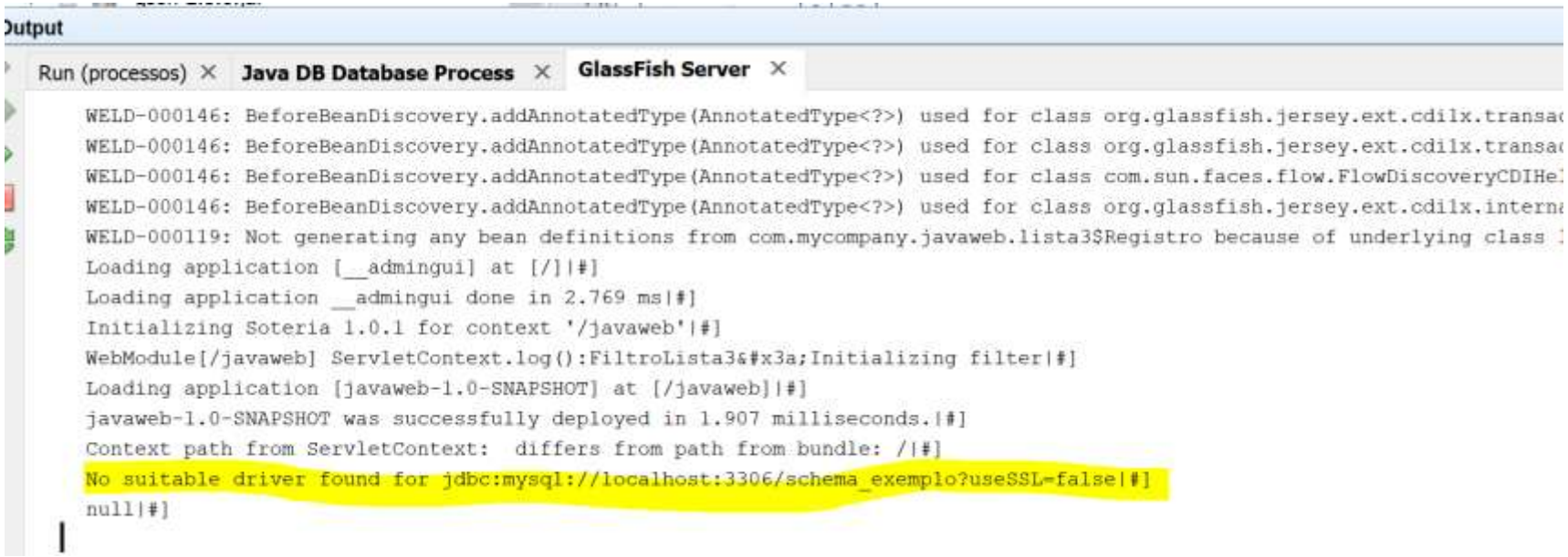
```
import java.sql.Connection;
import java.sql.DriverManager;
```

```
 public class DataSourceMySQL {
    public  Connection conectaBD() throws SQLException, ClassNotFoundException {
        String url ="jdbc:mysql://localhost:3306/schema_exemplo?useSSL=false&serverTimezone=UTC";
        String user ="root";
        String passwd = "root";
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(url, user, passwd);
        return con ;
     }
}
```

Driver de conexão com SGBD

A ausência do driver implica em exceção no momento da conexão.

```
Connection con = DriverManager.getConnection(url, user, passwd);
```



Leitura de dados

```
public ArrayList<Processo> buscaProcessos() {
    ArrayList<Processo> lista = new ArrayList<Processo>();
            try {
                DataSourceMySQL ds = new DataSourceMySQL();
                String sql ="select numero, assunto from processos";
                Connection c = ds.conectaBD();
                PreparedStatement ps = c.prepareStatement(sql);
                ResultSet rs = ps.executeQuery();
                while (rs.next()) {
                    Processo obj = new Processo();
                    obj.setNumero(rs.getString("numero"));
                    obj.setAssunto(rs.getString("assunto"));
                    lista.add(obj);
                }
            } catch (Exception e) {
                System.err.println(e.getMessage());
            }
            return lista;
    }
```

Inserção de dados

```
public boolean insereProcesso(Processo p) {
    boolean result=false;
    try {
        DataSourceMySQL ds = new DataSourceMySQL();
        Connection c = ds.conectaBD();;
        String sql = "insert into processos (numero, assunto) values(?,?)";
        PreparedStatement ps = c.prepareStatement(sql);
        ps.setString(1, p.getNumero());
        ps.setString(2, p.getAssunto());
        ps.execute();
        result=true;
    } catch (Exception e) {
        System.err.println(e.getMessage());
    }
    return result;
}
```

Atualização de dados

```
public boolean atualizaProcesso(Processo p) {
        boolean result=false;
        try {
            DataSourceMySQL ds = new DataSourceMySQL();
            Connection c = ds.conectaBD();
            String sql = "update processos set assunto=? where numero=?";
            PreparedStatement ps = c.prepareStatement(sql);
            ps.setString(1, p.getAssunto());
            ps.setString(2, p.getNumero());
            ps.execute();
            result=true;
        } catch (Exception e) {
            System.err.println(e.getMessage());
        }
        return result;
    }
```

Exclusão de dados

```
public boolean excluiProcesso(String numero) {
     boolean result=false;
     try {
         DataSourceMySQL ds = new DataSourceMySQL();
          Connection c = ds.conectaBD();
         String sql = "delete from processos where numero=?";
         PreparedStatement ps = c.prepareStatement(sql);
         ps.setString(1, numero);
         ps.execute();
         result=true;
     } catch (Exception e) {
         System.err.println(e.getMessage());
     }
     return result;
 }
```

Trabalho com Blob

```
public void atualizaFoto(int idStaff, File foto) throws SQLException, ClassNotFoundException, FileNotFoundException{
    Connection c = this.conectaBD();
    PreparedStatement ps = c.prepareStatement("update staff set picture= ? where staff_id=?");
    ps.setBlob(1,  new FileInputStream(foto));
    ps.setInt(2, idStaff);
    ps.execute();
}

  public void baixarFoto(int idStaff) throws SQLException, ClassNotFoundException, FileNotFoundException, IOException{
    Connection c = this.conectaBD();
    PreparedStatement ps = c.prepareStatement("select picture from staff where staff_id=?");
    ps.setInt(1, idStaff);
    ResultSet rs = ps.executeQuery();
     while (rs.next()){
        Blob blob = rs.getBlob("picture");
        byte[] dados = blob.getBytes(1, (int) blob.length());
        File f = new File("C:\\Users\\rafae\\OneDrive\\Imagens\\download.jpg");
        FileOutputStream fos = new FileOutputStream(f);
        fos.write(dados);
        fos.close();
    }
}
```

Implantação: Aplicação <-> Banco de dados

Um administrador de sistemas está configurando um ambiente para hospedar uma aplicação web corporativa que utiliza um banco de dados relacional MySQL e será executada em um servidor de aplicação Apache Tomcat.

Sobre a instalação e configuração desses componentes, analise as afirmativas:

I. Durante a instalação do MySQL, é recomendável configurar um usuário administrador com privilégios restritos para evitar que aplicações tenham acesso irrestrito ao banco de dados, aumentando a segurança.

II. O Apache Tomcat, por ser um servidor de aplicação leve, não suporta conexões simultâneas e, portanto, não é indicado para ambientes corporativos que demandam alta escalabilidade.

III. A configuração do pool de conexões no servidor de aplicação é fundamental para otimizar o uso dos recursos e melhorar a performance na comunicação entre a aplicação e o banco de dados.

IV. Para garantir a segurança dos dados, as conexões entre o servidor de aplicação e o banco de dados devem ser criptografadas, o que pode ser configurado ativando o SSL/TLS no MySQL e no Tomcat.

V. Durante a instalação do MySQL, o diretório dos arquivos de dados deve ser configurado preferencialmente em um disco SSD para melhorar o desempenho das operações de leitura e gravação.

Assinale a alternativa CORRETA:

A) Apenas as afirmativas I, III e IV estão corretas. (gabarito)

B) Apenas as afirmativas II e V estão corretas.

C) Apenas as afirmativas I, II, IV e V estão corretas.

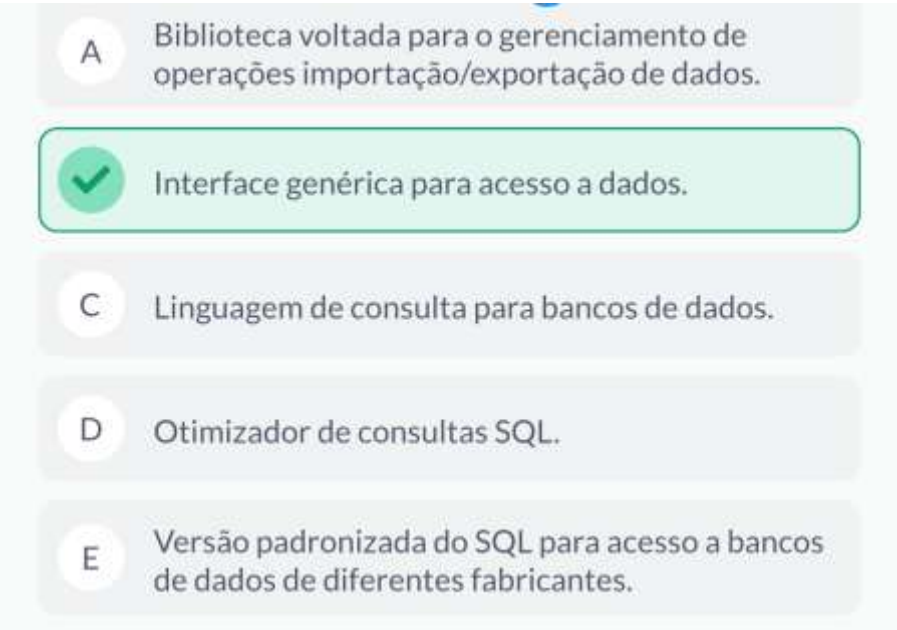
D) Todas as afirmativas estão corretas.

E) Apenas as afirmativas I, III e V estão corretas.O

Uso de HDDs é recomendável pois atingem maiores espaços de armazenamento e durabilidade no contexto de banco de dados.

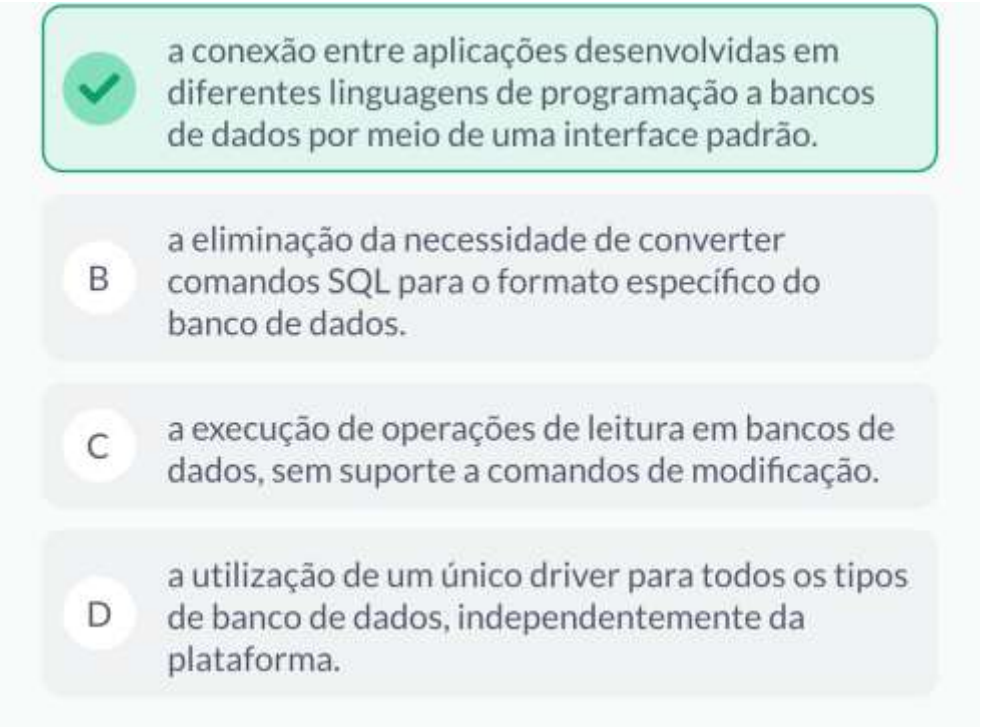
O que é ODBC?

- A tecnologia ODBC é amplamente comentada e utilizada quando o assunto é banco de dados. A definição correta para ODBC, é



ODBC

- O Open Database Connectivity (ODBC) é uma tecnologia que permite



Aula 16

Conectando em SGBD por aplicação – NODE.JS

Prof. Dr. Rafael Queiroz Gonçalves, PMP

MySQL – node.js

```
const mysql = require('mysql');
//initialize mysql connection
const MYSQL_IP="localhost";
const MYSQL_LOGIN="root";
const MYSQL_PASSWORD="root";

let con = mysql.createConnection({
  host:  MYSQL_IP,
  user: MYSQL_LOGIN,
  password: MYSQL_PASSWORD,
  database: "messages_app"
});

con.connect(function(err) {
  if (err){
    console.log(err);
    throw err;
  }
  console.log("Connection with mysql established");
});

  //without parameters
  const nameToFilter="Luke";
  let sql="SELECT * FROM messages_app.contact where first_name='"+nameToFilter+"';";
  con.query(sql, function (err, result) {
    if (err) throw err;
    result.forEach(record=>{
       console.log("Result without parameters:",record);
    });
  });

  //with parameters
  sql="SELECT * FROM messages_app.contact where first_name=?;";
  const parameters=[nameToFilter];
  con.query(sql,parameters, function (err, result) {
    if (err) throw err;
    result.forEach(record=>{
       console.log("Result with parameters:",record);
    });
  });
```

MySQL – Node.js - Blob read

```
const query = 'SELECT id, blob_column_name FROM your_table WHERE id = ?';
const recordId = 1; // Example ID
connection.query(query, [recordId], (err, results) => {
    if (err) throw err;
    if (results.length > 0) {
        const row = results[0];
        const blobData = row.blob_column_name; // This will be a Buffer object
        console.log('BLOB data retrieved:', blobData);

         const fs = require('fs');
         fs.writeFile('output_image.jpg', blobData, (err) => {
             if (err) throw err;
             console.log('Image saved successfully!');
         });

    } else {
        console.log('No record found with the specified ID.');
    }
    connection.end(); // Close the connection when done
});
```

MySQL – Node.js - Blob write

```
const mysql = require('mysql');

// Database connection details
const connection = mysql.createConnection({
    host: 'localhost',
    user: 'your_username',
    password: 'your_password',
    database: 'your_database'
});

connection.connect((err) => {
    if (err) throw err;
    console.log('Connected to MySQL database!');
    // Assuming you have a table named 'files' with a 'file_data' BLOB column
    const sql = 'INSERT INTO files (file_name, file_data) VALUES (?, ?)';
    const fileName = 'my_image.jpg';
    // Replace imageBuffer with your actual Buffer containing the BLOB data
    connection.query(sql, [fileName, imageBuffer], (err, result) => {
        if (err) throw err;
        console.log('BLOB data inserted successfully!');
        connection.end(); // Close the connection after the operation
    });
});
```

Demonstração

Conectando em um SGBD por linguagem de programação:

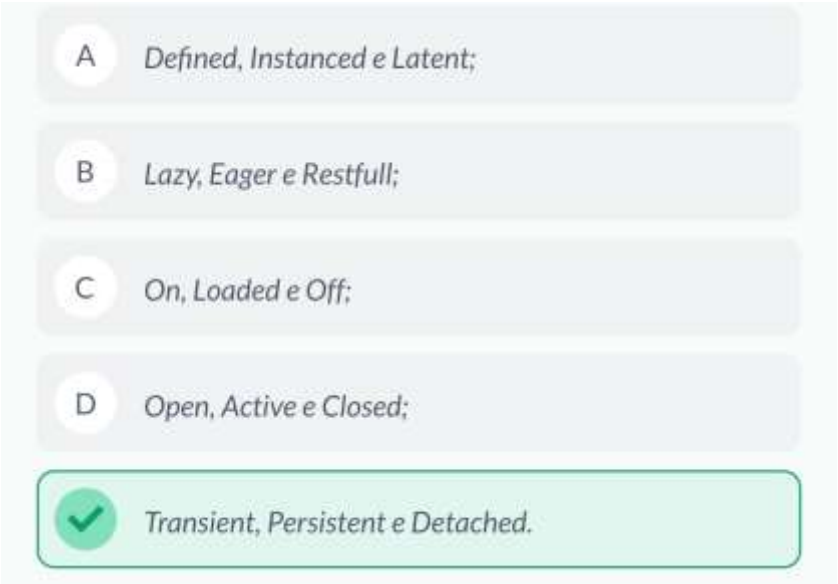
a) Em um schema existente, acesse os dados de uma tabela (SELECT) que contenha dados de mais de um registro, e exiba os dados de todas as linhas e colunas desta tabela.

b) Aplicar um filtro na consulta feita pela aplicação, cujo parâmetro é informado pelo usuário.

c) Faça a inserção de um novo registro com base em valores informados pelo usuário.

ORM - Exemplo

- No contexto do Hibernate, considere a declaração a seguir.
- Estudante estudante= new Estudante();
- É correto afirmar que o objeto estudante, ao longo da sua existência/utilização, pode estar em algum dos estados:



Transient: O objeto é criado e não está associado a uma sessão do Hibernate. Ele ainda não possui uma representação no banco de dados.

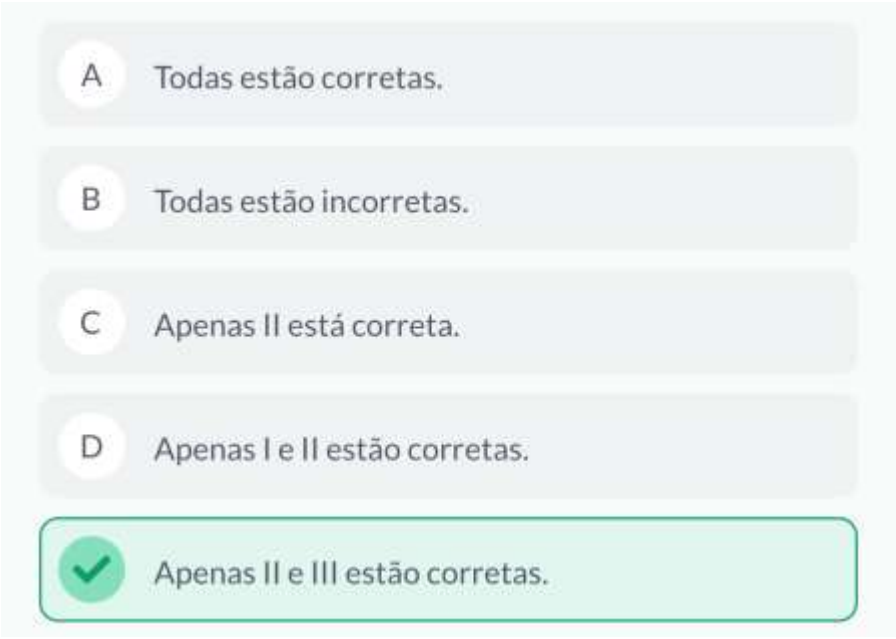
Persistent: O objeto está associado a uma sessão do Hibernate e possui uma representação correspondente no banco de dados. Qualquer alteração feita no objeto será refletida no banco de dados quando a transação for commitada.

Detached: O objeto já esteve associado a uma sessão do Hibernate, mas a sessão foi fechada ou o objeto foi explicitamente removido da sessão. Embora não esteja mais associado a uma sessão, o objeto ainda possui uma representação no banco de dados. Alterações feitas no objeto após ser desanexado não serão automaticamente sincronizadas com o banco de dados, a menos que o objeto seja reanexado a uma nova sessão.

Monitoramento

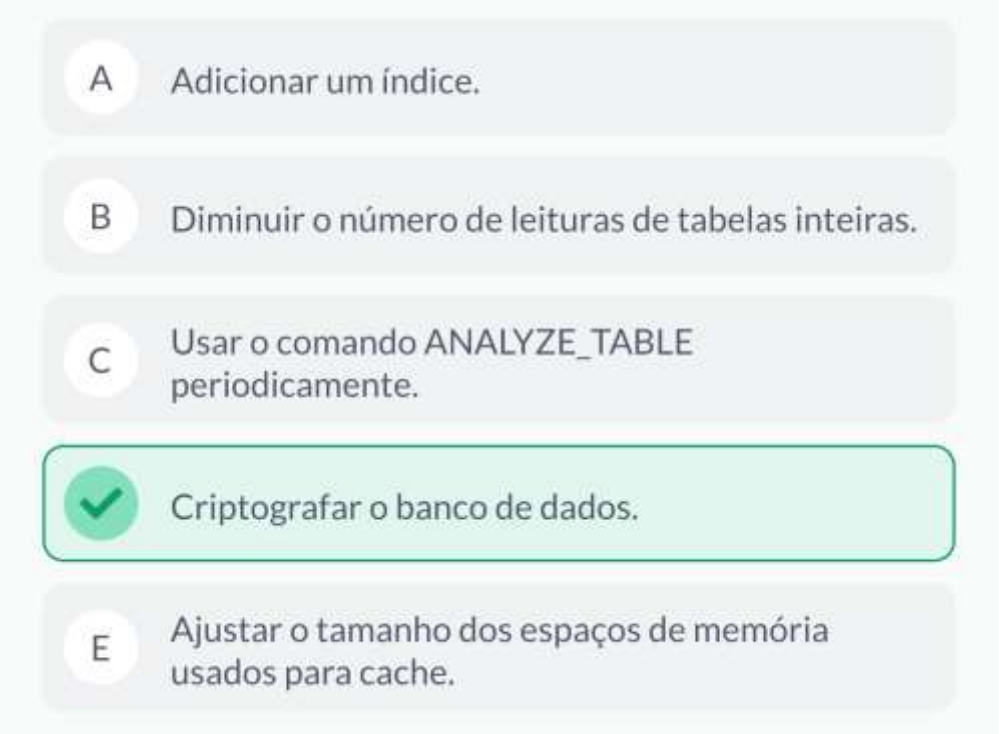
Analise as assertivas abaixo sobre os casos de uso do monitoramento de operação de banco de dados (Monitoring Database Operations) do Oracle 19c e assinale a alternativa correta.

- I. O monitoramento não tem suporte a cláusulas SQL em paralelo, por isso não pode ser usado para analisar problemas de balanceamento de carga.
- II. Uma cláusula SQL que começou a levar mais tempo para executar pode ser analisada com o monitoramento.
- III. Uma sessão de banco de dados que está com performance abaixo do esperado pode precisar ser analisada pelo monitoramento.



Tunning

- Assinale a alternativa que NÃO corresponde à técnica que pode ser usada para otimização de instruções do tipo SELECT no MySQL 8.



Replicação

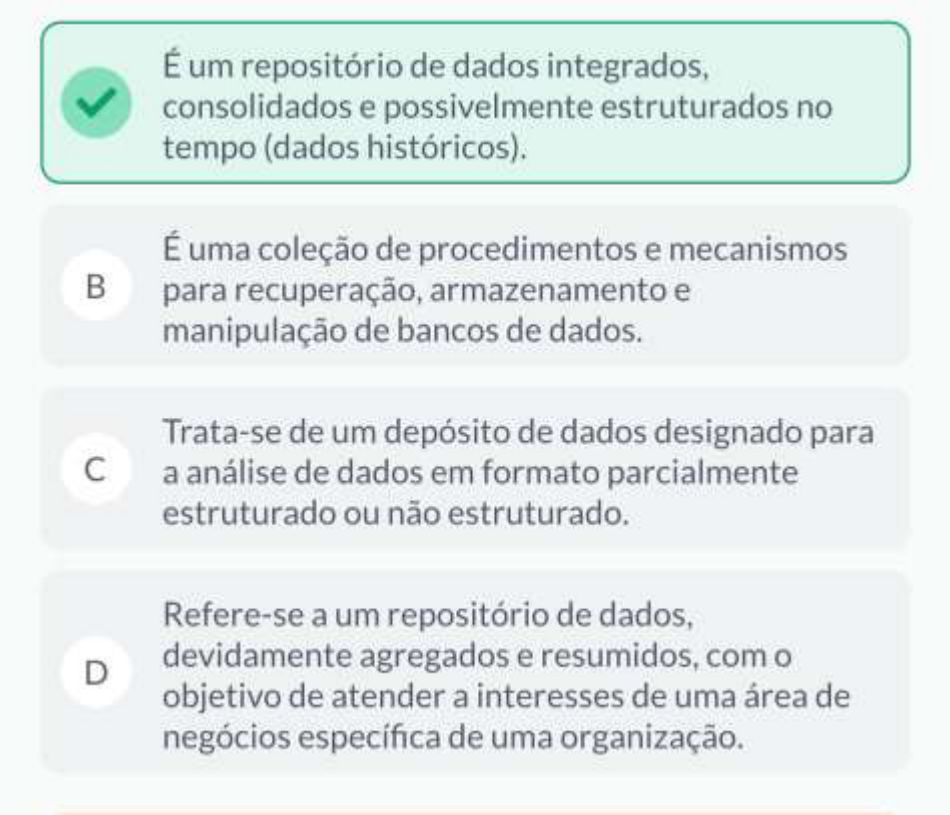
- Julgue o item subsecutivo, que se referem a conceitos de programação e banco de dados.

- Replicação e duplicação são processos de manutenção e atualização de bancos de dados distribuídos.

- Certo

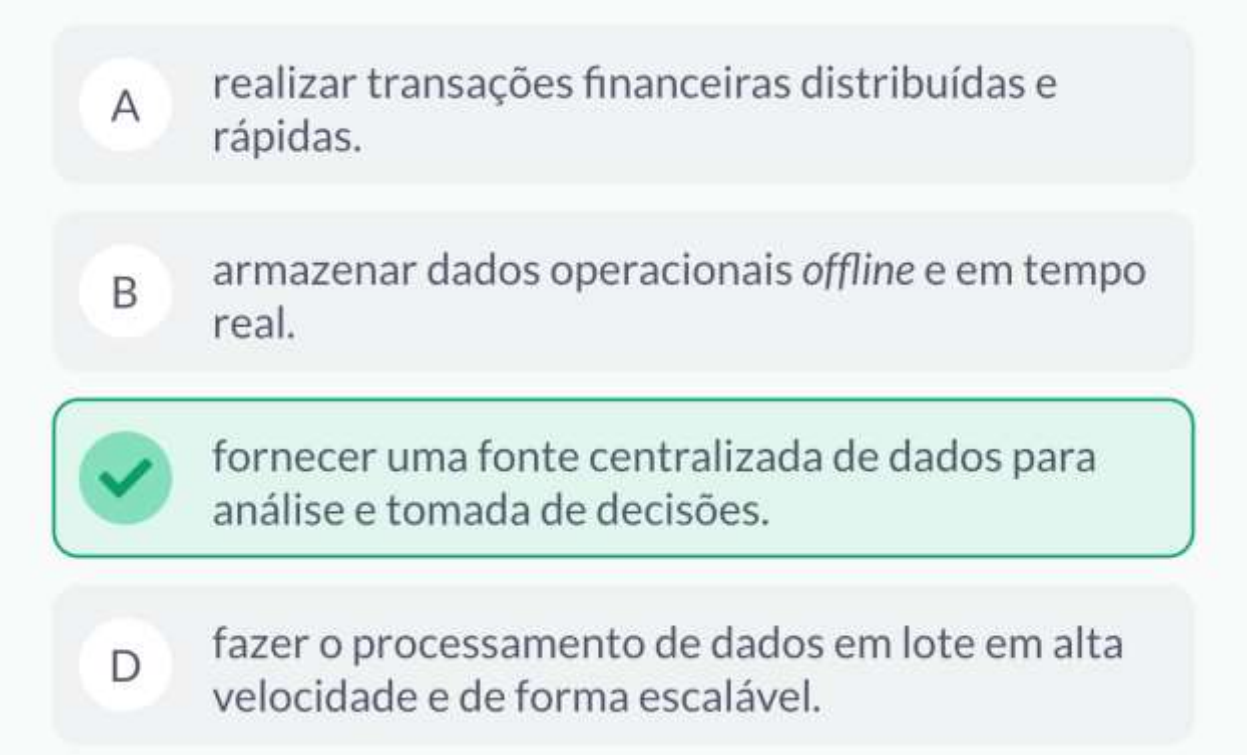
Data warehouse

- Sobre Data Warehouse, assinale a afirmativa correta



Dataware house

- A principal finalidade de um Data Warehouse é



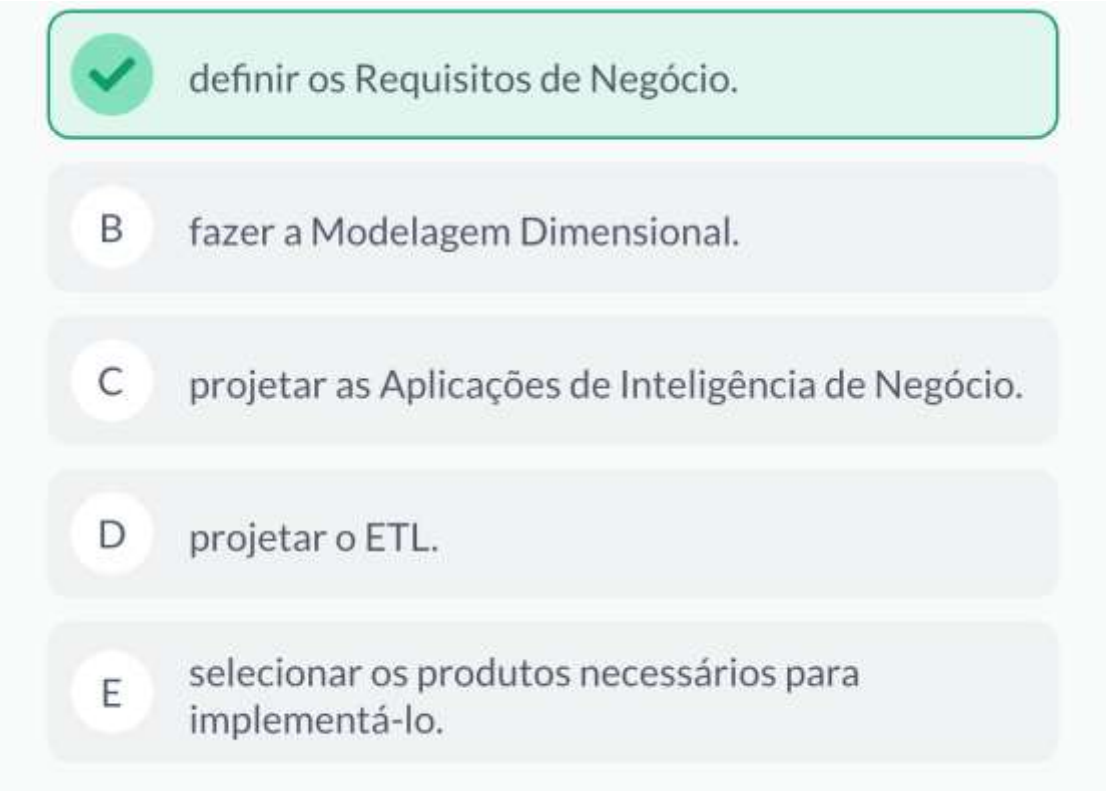
Data warehouse

- A gestão do TJRN é apoiada por sistemas de informações digitais que estão em produção há mais de dez anos abrangendo diversos contextos, como gestão de pessoal, gestão orçamentária, pedidos de serviço, controle de viaturas etc. Para apoiar a tomada de decisão de alto nível do Tribunal, é necessário o desenvolvimento de um banco de dados analítico que seja orientado a assunto, não volátil e histórico, integrando dados estruturados dos diversos sistemas e contextos.
  O banco de dados a ser desenvolvido é um Data:



Data warehouse

- Uma organização percebeu que seus dados estavam distribuídos entre vários sistemas, o que dificultava muito a criação de um dashboard para analisar sua produtividade. Resolveu, então, criar um Data Warehouse, seguindo o Ciclo de Vida da metodologia de Kimball. A partir desse momento, iniciou um projeto pela fase de planejamento de projeto.
  A seguir, essa organização deve continuar o projeto por



Data warehouse

No ambiente de Data Warehousing da 'SMFP/RJ, os dados são extraídos de diversas fontes e integrados para apoiar a fiscalização de rendas por meio do desenvolvimento de diversos artefatos de dados. Antes de começar as análises nesse ambiente, o fiscal de rendas Inácio fez consultas sobre o seu conteúdo, por exemplo:

- DW_Tributos, banco de dados analítico do tipo Data Warehouse que integra dados sobre os tributos arrecadados do Município do Rio de Janeiro.
- TP_EMPRESA, Caractere, 1, atributo que descreve o tipo da empresa contendo os seguintes valores: M - MEI ou S - Simples Nacional, e faz parte da tabela TB _EMPRESA.
- RL_Sit_Fiscal, relatório sobre a situação fiscal das empresas do Município do Rio de Janeiro.

O componente do ambiente de Data Warehousing, utilizado por Inácio, que foi desenvolvido para apoiar consultas sobre a descrição de cada artefato de dado, é:

A) ETL;

B) Métricas;

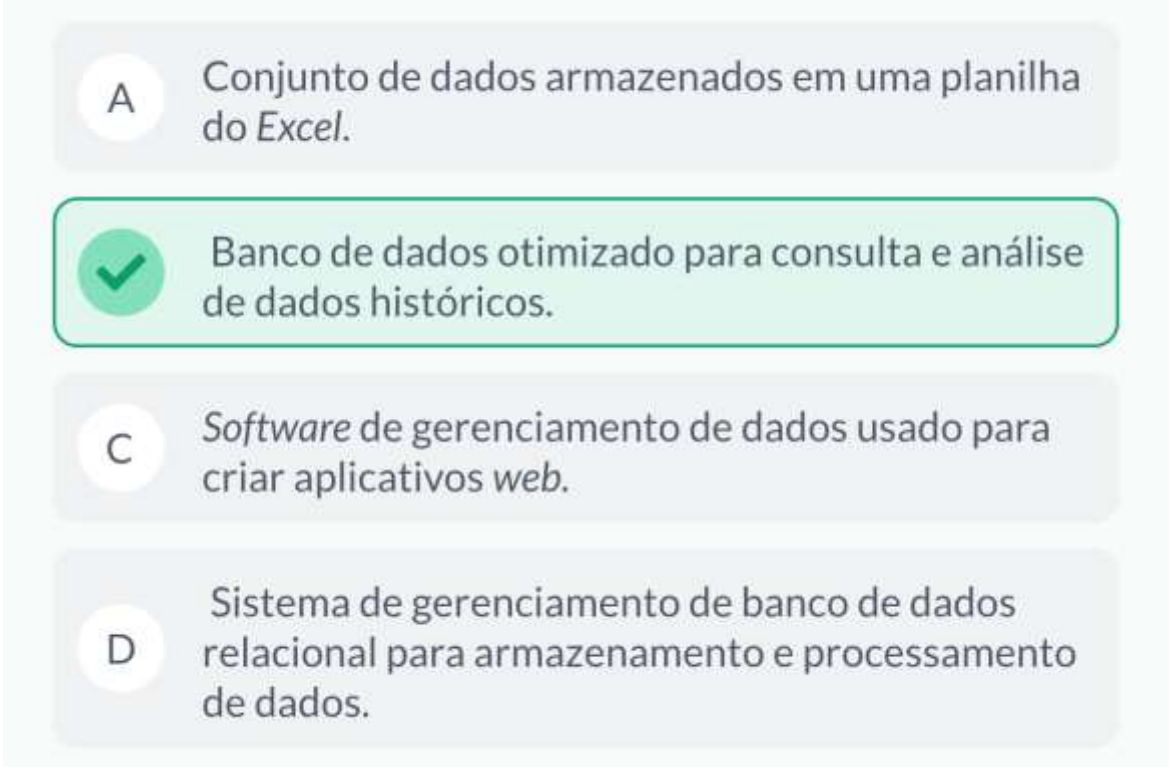
C) Dashboard;

D) Dimensões de Dados;

e) Repositório de Metadado (correto)

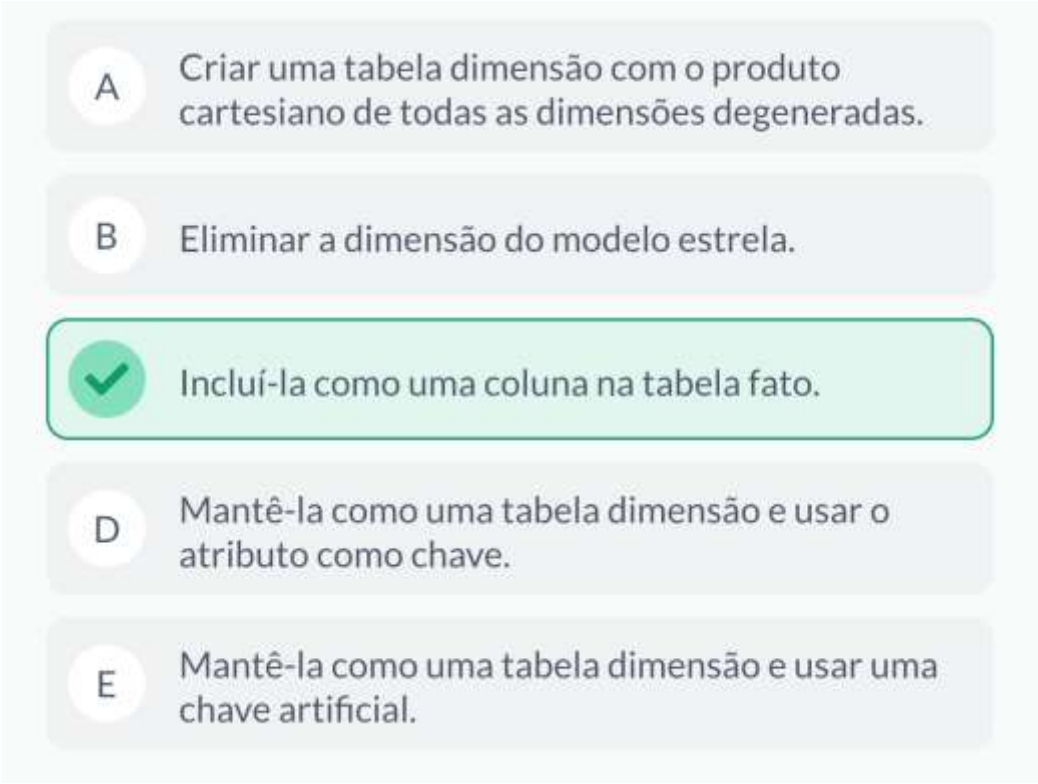
Data warehouse

- Um Data Warehouse é projetado para análises de dados, que envolvem a leitura de grandes quantidades de dados para compreender relações e tendências entre eles. A definição correta de Data Warehouse é:



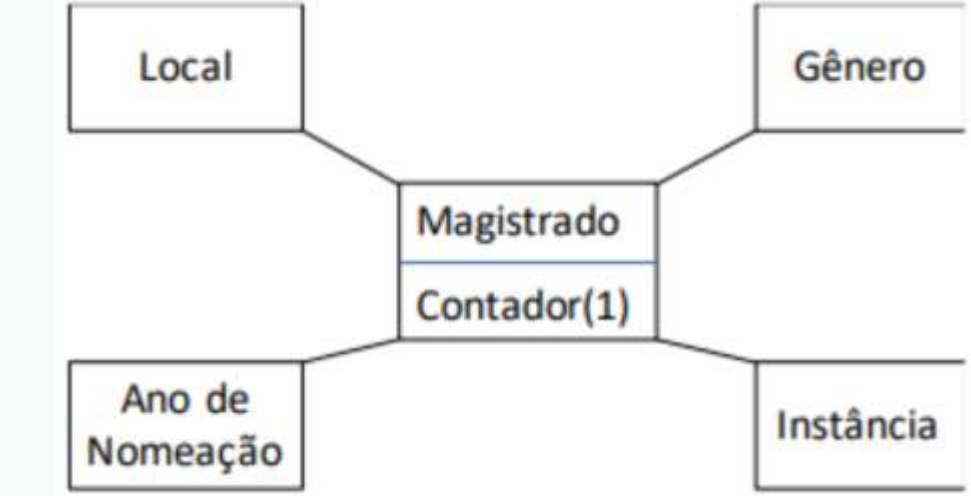
Data warehouse

- Ao construir um modelo de dados usando a modelagem dimensional para um Data Warehouse, seguindo um modelo estrela, um engenheiro de dados encontrou uma dimensão que se encaixava perfeitamente no conceito de dimensão degenerada, de apenas um atributo.
  Qual é a maneira correta de tratar uma dimensão degenerada?



Data warehouse

- Observe o modelo multidimensional de dados de um Data Warehouse a seguir.

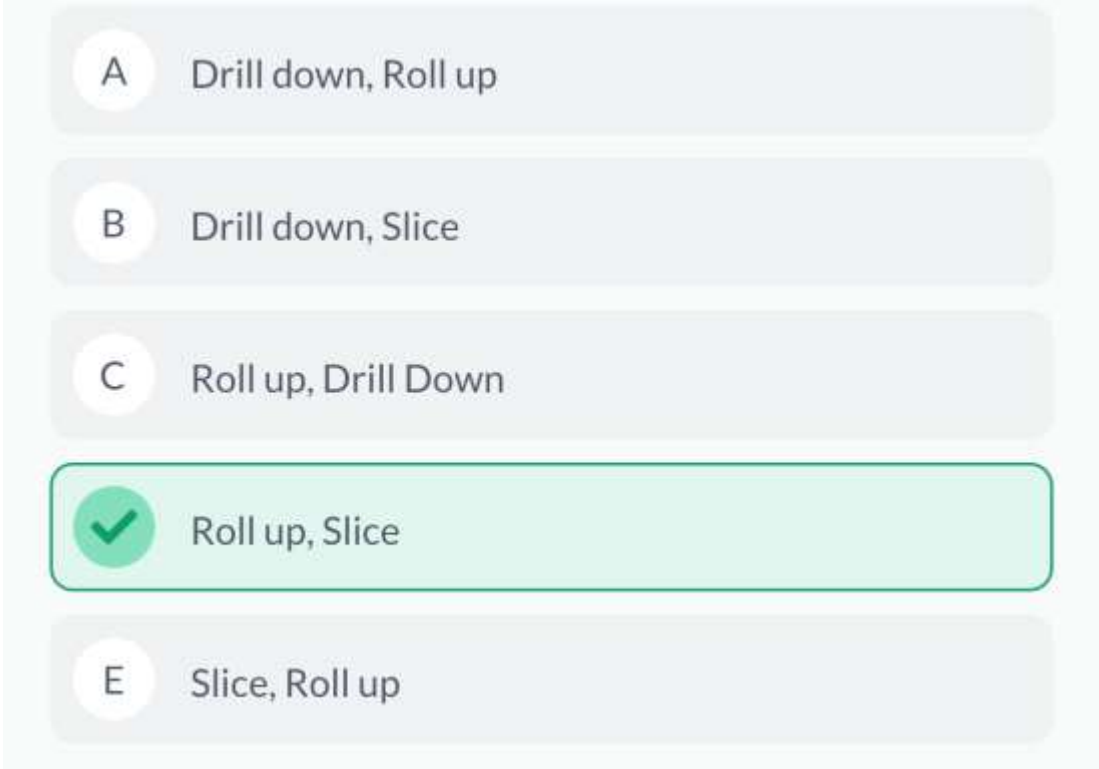


O tipo de modelagem multidimensional empregada é:

- Star Schema;
- B Snowflake Schema;
- C Fact Constellation;
- D Data Lake;

Data warehouse - OLAP

- Ao usar um cubo de dados OLAP, que permitia estudar todos os projetos de sua organização governamental, um colaborador iniciou analisando os projetos por Custo Total, Município e Mês. Percebendo que eram muitos dados, decidiu passar a olhar por Custo Total, Município e Ano. Esses dados, porém, mostravam muita diferença entre o governo atual e o antigo. Devido a isso, o colaborador resolveu pegar apenas os últimos 3 anos.
  Qual sequência correta das operações OLAP ele realizou?



Data warehouse - OLAP

Utilizando um Painel de Informações, Julia analisou a Taxa de Congestionamento de Processos Judiciais dos Estados do Brasil e depois selecionou o Estado do Rio Grande no Norte para analisar a taxa por cidade.
A operação OLAP realizada por Julia foi:

A Slice;

B Dice;

✓ Drill down;

D Drill up;

E Rotate.

Data warehouse - OLAP

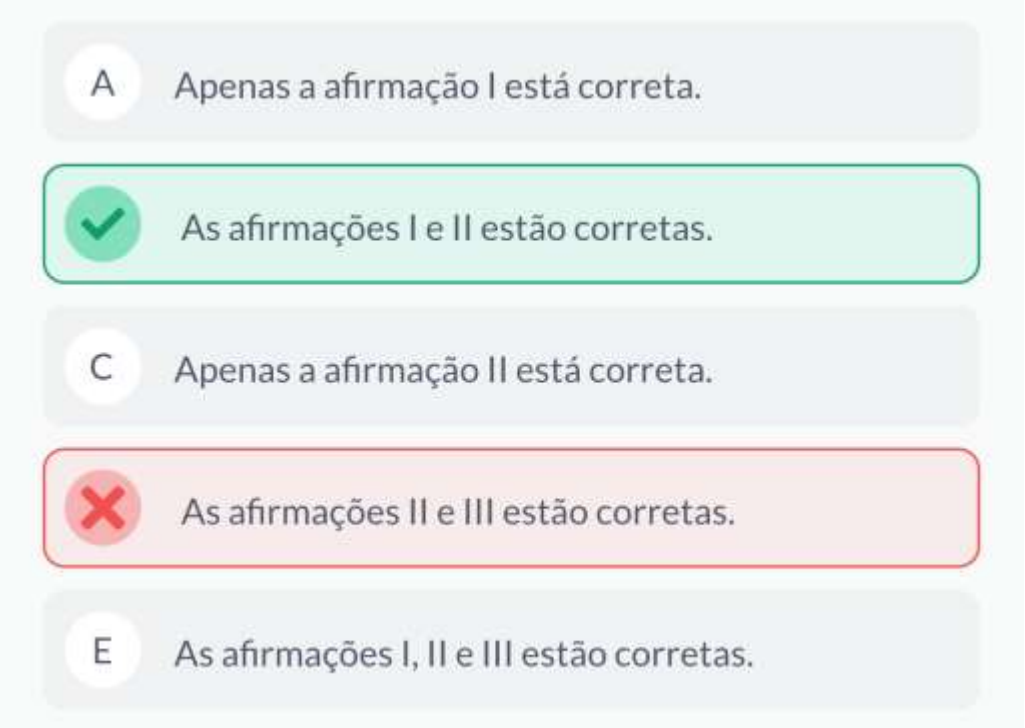
Considere as seguintes afirmações sobre OLAP (Online Analytical Processing) e Data Mining:

I. A função OLAP WINDOW é uma espécie de média móvel, sendo aplicada como uma nova cláusula nas expressões SQL (Structured Query Language) normais como: SELECT, FROM, WHERE, GROUP BY e HAVING.

II. Data Mining é um processo de análise de dados exploratório com o objetivo de procurar padrões nos dados e identificar comportamentos incomuns, visando a uma estratégia de negócio.

III. OLAP é o processo estático de criar, gerenciar, analisar e gerar relatórios sobre dados.

Assinale a alternativa CORRETA, conforme Date (2004):



Data warehouse - OLAP

Analise as afirmações a seguir sobre funcionalidades típicas de aplicações OLAP (Online Analytical Processing) de um sistema data warehouse:

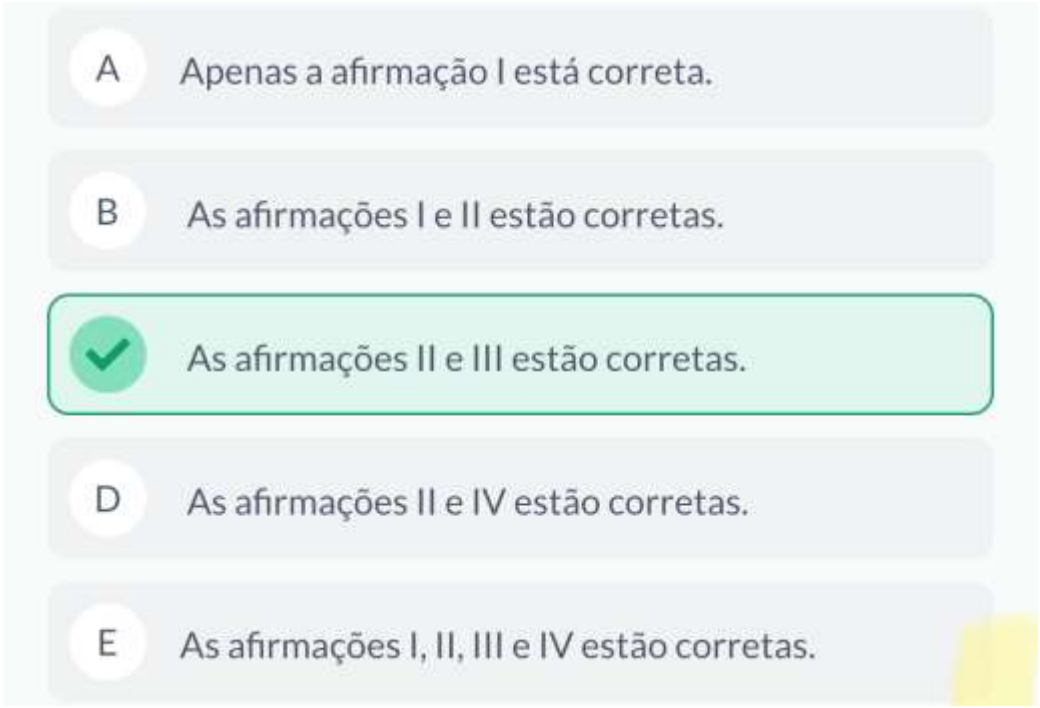
I. Roll-up são níveis de detalhamento cada vez maiores.

II. Giro é a operação de tabulação cruzada.

III. Slice e dice são operações de projeção realizadas nas dimensões.

IV. Drill-down são dados resumidos com generalização cada vez maior.

Assinale a alternativa CORRETA conforme Elmasri e Navathe (2011, p. 728):



DW – Tabelas de fato e dimensão

Considerando a tabela de fato apresentada inicialmente e as tabelas de dimensão a ela subsequentes, julgue o item a seguir, referente à modelagem dimensional de dados.

A tabela fato_processos deveria conter informações detalhadas acerca do nome do juiz e do ano de referência, para otimizar a consulta direta, eliminando-se a necessidade de dimensões.

- Certo
- **Errado (gabarito)**

DW – Tabelas de fato e dimensão

Considerando a tabela de fato apresentada inicialmente e as tabelas de dimensão a ela subsequentes, julgue o item a seguir, referente à modelagem dimensional de dados.

A granularidade da tabela fato_processos é definida em nível de evento por processo, de forma que cada registro nessa tabela representa uma movimentação judicial específica, enquanto as dimensões agregam atributos descritivos necessários para contextualizar a análise multidimensional.

- **Certo (gabarito)**
- Errado

OLAP

A diferença entre OLTP e OLAP no contexto de Business Intelligence é definida pela ideia de que

- A) OLTP e OLAP são termos intercambiáveis que se referem ao mesmo conceito de armazenamento e análise de dados.
- B) OLTP lida com transações que ocorrem simultaneamente, enquanto OLAP é voltado para análise de dados estratégicos e tomada de decisões. (gabairot)
- C) OLTP é usado para análise de dados históricos, enquanto OLAP é usado para transações em tempo real.
- D OLTP e OLAP são técnicas diferentes de modelagem de dados, sem relação direta com Business Intelligence.

OLAP

Acerca de business intelligence, ETL e OLAP, julgue o item que se segue.

A análise preditiva utiliza algoritmos estatísticos e modelos de machine learning para identificar padrões históricos nos dados e projetar tendências futuras, de forma a guiar decisões proativas e permitir a antecipação de cenários críticos para o negócio.

- **Certo (gabarito)**
- Errado

OLAP

- Associe os dois conceitos a seguir, às suas respectivas características.
- 1. OLAP
- 2. Data Lake
- ( ) Seu principal padrão de escrita é a importação em massa (ETL) ou fluxo de eventos.
- ( ) Armazena principalmente dados brutos, passando ou não por transformação.
- ( ) É caracterizada pela agregação sobre um grande número de registros, alimentando relatórios que ajudam a gestão de uma empresa a tomar decisões mais informadas.
- ( ) permite o armazenamento de grandes volumes de dados de qualquer tipo e tamanho, sendo uma alternativa aos Data Warehouses tradicionais.
- Assinale a opção que apresenta a associação correta, na ordem apresentada.
- A) 1 – 2 – 2 – 1.
- B) 2– 2 – 1 – 2.
- **C) 1 - 2 – 1 – 2. (gabarito)**
- D) 2 – 1 – 2 – 1.
- E) 1 – 1 – 2 – 2.

ROLAP

Acerca de business intelligence, ETL e OLAP, julgue o item que se segue.

O ROLAP (relational OLAP) armazena os dados exclusivamente em estruturas multidimensionais, como cubos OLAP, ignorando bancos de dados relacionais para processamento analítico.

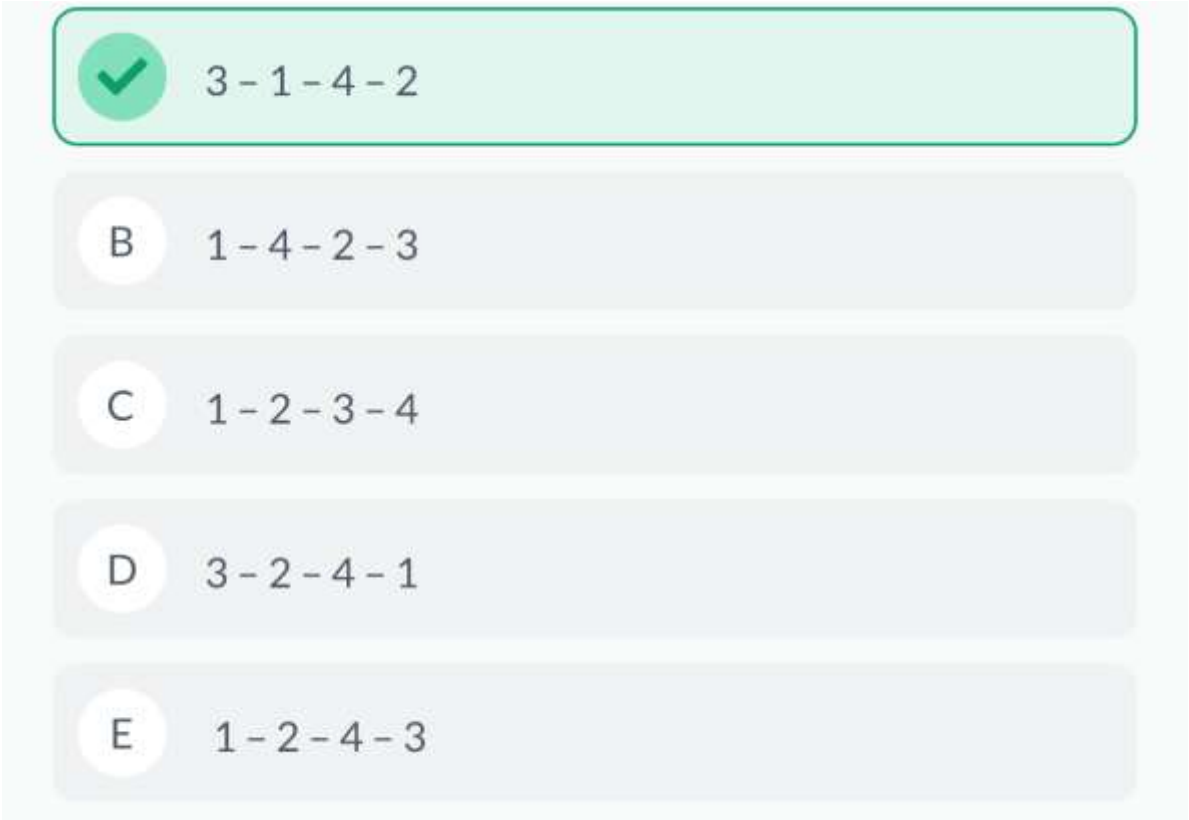
- Certo
- **Errado (gabarito)**

Data Mining

Sobre as etapas do processo de descoberta de conhecimento e mineração de dados (KDD, Knowledge Discovery and Data Mining), numere a coluna da direita de acordo com a coluna da esquerda:

- 1. Seleção de dados 2. Limpeza de dados 3.Mineração de dados 4. Avaliação
- ( ) São aplicados algoritmos para extração de características dos dados.
- ( ) O subconjunto objetivado dos dados e os atributos de interesse são identificados examinando-se o conjunto de dados bruto inteiro.
- ( ) Os padrões são apresentados para os usuários em uma forma inteligível.
- ( ) Ruído e exceções são removidos, valores de campo são transformados em unidades comuns e alguns campos são criados pela combinação de campos já existentes para facilitar a análise. Normalmente, os dados são colocados em um formato relacional, e várias tabelas podem ser combinadas em uma etapa de desnormalização.

Assinale a alternativa que apresenta a numeração CORRETA da coluna da direita, de cima para baixo, conforme Raghu e Gehrke (2011, p. 739)



Data Mining

Na Mineração de Dados, qual dos seguintes métodos é usado para identificar padrões sequenciais em conjuntos de dados temporais?

- A) Árvores de decisão.
- B)Algoritmos genéticos.
- C) Modelos de Markov ocultos (gabarito).
- D) Lista duplamente encadeada

Exemplo - ETL

- A analista de suporte Ana gerencia o ELCluster na PGM de Niterói. O ELCluster agrupa nós do servidor de busca e análise de dados Elasticsearch. O Elasticsearch do ELCluster foi configurado para realizar uma série de transformações nos dados de entrada, a fim de extrair conteúdos específicos possivelmente presentes nos dados. Essas transformações são executadas em um pipeline antes da indexação dos dados. Ana constatou um desempenho abaixo do esperado nas transformações dos dados de entrada do ELCluster e concluiu serem necessários mais nós Elasticsearch dedicados à execução de pipelines de pré-processamento de dados.
  Logo, Ana deve adicionar ao ELCluster mais nós Elasticsearch do tipo:



ETL

Acerca de business intelligence, ETL e OLAP, julgue o item que se segue.

Na transformação avançada do ETL, a derivação de dados cria novos atributos ou métricas a partir de colunas existentes, usando cálculos complexos e funções analíticas para enriquecer o dataset e suportar análises preditivas e segmentações contextuais.

- **Certo (gabarito)**
- Errado

Blockchain

- O blockchain pode ser compreendido como um sistema centralizado de registro de transações, hospedado em um banco de dados relacional. Ele é mutável e transparente, o que garante segurança nas transações bancária

